```
!pip install einops
!pip install torchinfo
# !pip install datasets

Requirement already satisfied: einops in
/usr/local/lib/python3.12/dist-packages (0.8.2)
Collecting torchinfo
  Downloading torchinfo-1.8.0-py3-none-any.whl.metadata (21 kB)
Downloading torchinfo-1.8.0-py3-none-any.whl (23 kB)
Installing collected packages: torchinfo
Successfully installed torchinfo-1.8.0
```

HW8: Diffusions & Low-Rank Adaptation (LoRA)

In this assignment, you will learn to implement diffusion model and low-rank adaptation (LoRA) from scratch using the Pytorch library.

This homework consists of two main sections:

In the first section, we introduce the diffusion model where you will be tasked to implement various components for training diffusion model, including a noise scheduler, sampling module, and model architectures. After building our these compoments, we will assemble them and train the diffusion model on the MNIST dataset.

The second part will introduce you to parameter-efficient transfer learning (PET) where LoRA will be used to transfer the MaskFormer (https://arxiv.org/abs/2107.06278), an instance segmentation model, to semantic segmentation task on a satellite-building segmentation dataset. This section will teach you how LoRA works and how to implement it from scratch using `forward_hook`.

Part 1: Diffusion Model

In this section, you will be tasked to implement each component of the diffusion model (DDPM).

Based on the components in DDPM, this section is organized into four parts :

1. **Noise Scheduler (Foward Process)**: During diffusion's forward process, the noise was gradually added, transforming the clean image x_0 into noisy image x_t w.r.t to the timestep t. To do so, the noise scheduler is used for controlling the amount of noise to be added in each timestep.

2. **Sampling (Reverse Process)**: During the revesere process, the sampling algorithm is employed to update x_t to remove the noise inside it using the noise estimation model θ.

3. **Model Architecture**: The deep learning model θ is for predicting the noise in x_t to be used in reverse process. In this homework, we will use UNet as a base architecture.

4. **Training Process**: The training process will combine these three modules to perform a learning process.

The overview of the system is shown below.

Downloading CIFAR10 Dataset

The MNIST dataset comprises of 60,000 training images at 32x32 resolution; the images are normalized to [-1, 1]. In the following, you will learn how to train diffusion on this dataset.

```
import os
import torch
import torchvision.datasets as datasets
from torch.utils.data import DataLoader
from torchvision.transforms import Compose, ToTensor, Resize
from einops import rearrange
from tqdm import tqdm
# from torch_ema import ExponentialMovingAverage

class Rescale(object):
    def __init__(self, old_range, new_range):
        self.old_range = old_range
        self.new_range = new_range

    def __call__(self, image):
        old_min, old_max = self.old_range
        new_min, new_max = self.new_range
        image -= old_min
        image *= (new_max - new_min) / (old_max - old_min)
        image += new_min
        return image

normalize_to_neg_one_to_one = Rescale((0, 1), (-1, 1))
unnormalize_to_zero_to_one = Rescale((-1, 1), (0, 1))

RESOLUTION = (32, 32)
BATCH_SIZE = 64

transform = Compose([
    Resize(RESOLUTION), lambda x: x.convert('RGB'), ToTensor(),
normalize_to_neg_one_to_one
])

training_data = datasets.MNIST(
```

```
    root="./data",
    train=True,
    download=True,
    transform=transform
)

test_data = datasets.MNIST(
    root="./data",
    train=False,
    download=True,
    transform=transform
)

train_dataloader = DataLoader(training_data, batch_size=BATCH_SIZE,
shuffle=True)
test_dataloader = DataLoader(test_data, batch_size=BATCH_SIZE,
shuffle=False)
```

```
100%|██████████| 9.91M/9.91M [00:00<00:00, 45.9MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 1.18MB/s]
100%|██████████| 1.65M/1.65M [00:00<00:00, 10.4MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 9.80MB/s]
```

These are example images from the CIFAR dataset.

```
import matplotlib.pyplot as plt
import numpy as np

fig, axs = plt.subplots(ncols=9, figsize=(15, 75))
for i in range(9):
  image, label = training_data[i]
  axs[i].imshow(rearrange(unnormalize_to_zero_to_one(image), "c h w ->
h w c"))
  axs[i].set_title(training_data.classes[label])
  axs[i].set_axis_off()
plt.show()
```

Noise Scheduler

The noise scheduler controls the acceleration of the noise added through the forward process, transfroming the clean image x_0 into noisy image x_t w.r.t. the timestep t. Typically, the noise scheduler is categorized into two classes: Variance Preserving and Variance Exploding. However,

for this homework, we focus on the variance-preserving scheduler ($\alpha_t = 1 - \beta_t$). You task is to implement linear and cosine schedulers based on the description provided below.

The noise scheduler consists of four parameters:

1. alpha (α): A numpy array that stores α_t (`alpha[i]` $= \alpha_i$).
2. alpha_cumprod ($\acute{\alpha}$): A numpy array that stores cumulative product of α (`alpha_cumprod[i]` $= \acute{\alpha}_i$; $\acute{\alpha}_t = \prod_{i=0}^{t} \alpha_i$)
3. alpha_cumprod_prev: The shifted version of alpha_cumprod such that `alpha_cumprod_prev[i]` $= \acute{\alpha}_{i-1}$ and `alpha_cumprod_prev[0] = 1`.
4. beta (β): A numpy array stores β_t (`beta[i]` $= \beta_i$).

After initializing the parameters, we are going to sample $x_t \sim p(x_t \vee x_0, t)$ where $p(x_t \vee x_0, t)$ is a normal distribution with the mean and variance defined as $\sqrt{\acute{\alpha}_t} x_0$ and $(1 - \acute{\alpha}_t) I$, respectively ($p(x_t \vee x_0, t) = N(\sqrt{\acute{\alpha}_t} x_0, (1 - \acute{\alpha}_t) I)$). When sampling from a normal distribution, it can be done using the following equation (reparameterization trick): \begin{aligned} x &\sim \mathcal{N}(x; \ \mu, \sigma^2) \ x &= \mu + \sigma z \quad z \sim \mathcal{N} (0, \mathbf{I}) \ \end{aligned}

Instruction

TODO 1: initialize parameters in the scheduler for the linear scheduler. (β is linearly space between 0.0001 and 0.02.) TODO 2: initialize parameters in the scheduler for the cosine scheduler. TODO 3: calculate mean of $p(x_t \vee x_0, t)$ ¿>$\sqrt{\acute{\alpha}_t} x_0$ TODO 4: calculate std of $p(x_t \vee x_0, t)$ ¿>$\sqrt{1 - \acute{\alpha}_t}$ TODO 5: sample $x_t \sim p(x_t \vee x_0, t)$ using a reparameterization trick in VAE

Hint: You can use `torch.cumprod` to calculate cumulative product.

```
import torch
import torch.nn.functional as F
import numpy as np

def expand_axis_like(a, b):
    """ Expands axes (at the end) of b to have the same number of axis
as a.

    Args:
        a (Tensor): A reference tensor.
        b (Tensor): A target tensor to be expanded.

    Returns:
        Expanded version of b.

    """
    assert len(a.shape) >= len(b.shape), f"The number of axis in a
must greater than b. Shape a: {a.shape}, Shape b: {b.shape}"
```

```
    n_unsqueeze = len(a.shape) - len(b.shape)
    b = b[(..., ) + (None, ) * n_unsqueeze] # unsqueeze such that it
has the same size for broadcasting.
    return b

class NoiseScheduler:
    def __init__(self, T, mode="linear"):
        """ Noise Scheduler Abstract Class

        Args:
            T (int): A maximum number of diffusion timestep.

        """
        self.T = T
        self.mode = mode
        self.init_alpha_beta()

    def init_alpha_beta(self):
        """ Initialize alpha and beta parameters based on the
scheduler. """
        if self.mode == "linear":
            # TODO 1: initialize parameters for linear scheduler
            t = np.arange(1, self.T + 1, 1)
            self.beta = 0.0001 * (1 - ((t - 1) / (self.T - 1))) + 0.02
* ((t - 1) / (self.T - 1))
            self.alpha = 1 - self.beta
            self.alpha_cumprod = np.cumprod(self.alpha)
            self.alpha_cumprod_prev = self.alpha_cumprod.copy()
            self.alpha_cumprod_prev[0] = 1.0
            self.alpha_cumprod_prev[1:] = self.alpha_cumprod[:-1]
        elif self.mode == "cosine":
            # TODO 2: initialize parameters for cosine scheduler
            t = np.arange(1, self.T + 1, 1)
            f = np.cos( ( ((t - 1) / (self.T - 1)) + 0.008 ) * np.pi /
((1 + 0.008) * 2)   ) ** 2
            f1 = f[0]
            self.alpha_cumprod = f.copy() / f1
            self.alpha_cumprod_prev = self.alpha_cumprod.copy()
            self.alpha_cumprod_prev[1:] = self.alpha_cumprod[:-1]
            self.alpha_cumprod_prev[0] = 1.0
            self.beta = 1 - (self.alpha_cumprod /
self.alpha_cumprod_prev)
            self.beta = np.clip(self.beta, a_max=0.999, a_min=None) #
its mean that no elements value is over 0.999
            self.alpha = 1 - self.beta

        else:
            raise NotImplementedError

        self.beta = torch.tensor(self.beta, dtype=torch.float32)
```

```
        self.alpha = torch.tensor(self.alpha, dtype=torch.float32)
        self.alpha_cumprod = torch.tensor(self.alpha_cumprod,
dtype=torch.float32)
        self.alpha_cumprod_prev =
torch.tensor(self.alpha_cumprod_prev, dtype=torch.float32)

    def _mean(self, x_0, t):
        """ Mean of p(x_t | x_0, t)

        Args:
            x_0 (Tensor): Clean images (Shape: (B, C, H, W))
            t (Tensor): Diffusion time-step (Shape: (B))

        Returns:
            Mean of p(x_t | x_0, t) (Shape: (B, C, H, W))

        """
        # TODO 3: calculate mean of p(x_t | x_0, t)
        # alpha_cumprod[t] has shape: (batch,)
        tmp_alpha = (self.alpha_cumprod[t.cpu()] ** 0.5)
        tmp_alpha = tmp_alpha.reshape(-1, 1, 1, 1)
        alpha_t = torch.tensor(tmp_alpha, device=x_0.device,
dtype=x_0.dtype)
        x_mean = x_0 * alpha_t
        return x_mean

    def _std(self, t):
        """ Standard deviation of p(x_t | x_0, t)

        Args:
            t (Tensor): Diffusion time-step (Shape: (B))

        Returns:
            Standard deviation of p(x_t | x_0, t) (Shape: (B))

        """
        # TODO 4: calculate standard deviation of p(x_t| x_0, t)
        std = (1 - self.alpha_cumprod[t.cpu()]) ** 0.5
        return torch.tensor(std, device=t.device)

    def marginal_prob(self, x_0, t):
        """ Marginal probability p(x_t | x_0, t)"""
        return self._mean(x_0, t), self._std(t)

    def sample_marginal_prob(self, x_0, t, noise=None):
        """ Sample x_t from p(x_t | x_0, t)

        Args:
            x_0 (Tensor): Clean images (Shape: (B, C, H, W))
            t (Tensor): Diffusion time-step (Shape: (B))
```

```
            noise (Tensor): A gaussian noise to be used in the
reparameterization trick.
                            If it is None, noise is sample from
standard normal. Default to None
        Returns:
            x_t (Shape: (B, C, H, W))

        """
        # TODO 5: sample $x_t \sim p(x_t | x_0, t)$ using a
reparameterization trick in VAE
        # Note: You can use marginal_prob function to obtain the mean
and std of p(x_t| x_0, t).
        #       If noise is provided, you must use it to sample x_t by
setting z = noise. Otherwise, z is sampled from standard normal.
        mean, std = self.marginal_prob(
            x_0, t
        )  # Compute mean and std of the marginal prob.

        if (noise is None):
            z = torch.randn_like(x_0)
        else:
            z = noise

        std = std.reshape(-1, 1, 1, 1)
        x_t = z * std + mean
        return x_t

    def prior_sampling(self, resolution, batch_size=1,
num_channels=3):
        """ Sampling Gaussian noise

        Args:
            resolution (Tuple[int]): A tuple of integer indicates
width and height of the image.
            batch_size (int, optional): A number of noise to be
generated.
            num_channels (int, optional): A number of channels in the
images.

        Returns:
            The sampling noises sample from Gaussian distribution.

        """
        return torch.randn(batch_size, num_channels, *resolution)

    def to(self, *args, **kwargs):
        """ Store the parameters on the given devices (eg. cpu, cuda)
"""
        self.alpha = self.alpha.to(*args, **kwargs)
        self.beta = self.beta.to(*args, **kwargs)
```

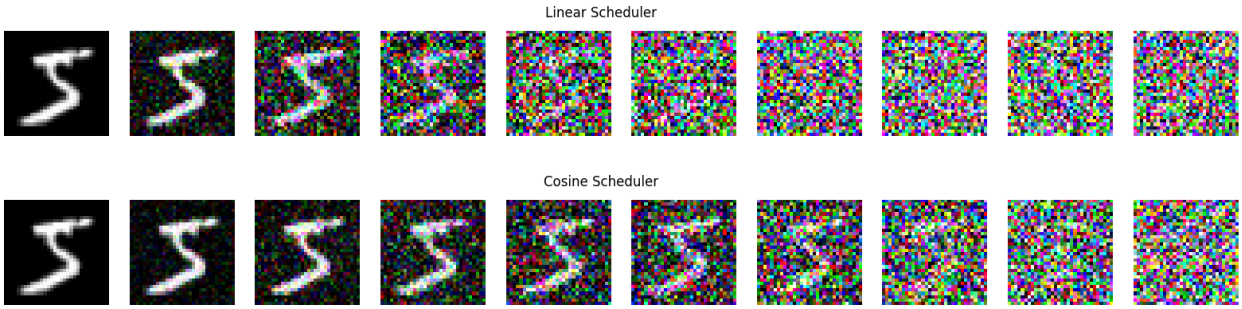
```
        self.alpha_cumprod = self.alpha_cumprod.to(*args, **kwargs)
        self.alpha_cumprod_prev = self.alpha_cumprod_prev.to(*args,
**kwargs)
        return self
```

To verify that the code is correct, we provide the following code that perform a forward pass. If done correctly, the output should be similar to this:

```
T = 1000
linear_scheduler = NoiseScheduler(T, "linear")
cosine_scheduler = NoiseScheduler(T, "cosine")

for image, label in training_data:
    fig_lin, axs_lin = plt.subplots(ncols=10, figsize=(20, 2))
    fig_cos, axs_cos = plt.subplots(ncols=10, figsize=(20, 2))

    for idx, t in enumerate(np.linspace(0, T-1, 10, dtype=int)):
        x = torch.unsqueeze(image, 0)
        t = torch.tensor([t], device=x.device)

        perturbed_data = linear_scheduler.sample_marginal_prob(
            x, t
        )
        perturbed_data =
unnormalize_to_zero_to_one(perturbed_data).clamp(0, 1)
        perturbed_data = rearrange(torch.squeeze(perturbed_data), "c h
w -> h w c").detach().cpu().numpy()
        axs_lin[idx].imshow(perturbed_data)
        axs_lin[idx].set_axis_off()

        perturbed_data = cosine_scheduler.sample_marginal_prob(
            x, t
        )
        perturbed_data =
unnormalize_to_zero_to_one(perturbed_data).clamp(0, 1)
        perturbed_data = rearrange(torch.squeeze(perturbed_data), "c h
w -> h w c").detach().cpu().numpy()
        axs_cos[idx].imshow(perturbed_data)
        axs_cos[idx].set_axis_off()
    fig_lin.suptitle('Linear Scheduler')
    fig_cos.suptitle('Cosine Scheduler')
    plt.show()
    break
```

```
/tmp/ipykernel_496/3489802820.py:80: UserWarning: To copy construct
from a tensor, it is recommended to use sourceTensor.detach().clone()
or sourceTensor.detach().clone().requires_grad_(True), rather than
torch.tensor(sourceTensor).
```

```
  alpha_t = torch.tensor(tmp_alpha, device=x_0.device,
dtype=x_0.dtype)
/tmp/ipykernel_496/3489802820.py:96: UserWarning: To copy construct
from a tensor, it is recommended to use sourceTensor.detach().clone()
or sourceTensor.detach().clone().requires_grad_(True), rather than
torch.tensor(sourceTensor).
  return torch.tensor(std, device=t.device)
```



Analyzing the noise scheduler (TODO 6)

How does linear scheduler and cosine scheduler differ from each other? Which one add noise faster?

Try varying `T`. Does x_T become a gaussian noise when `T` is small? Why is it neccessary to set `T` to be very large?

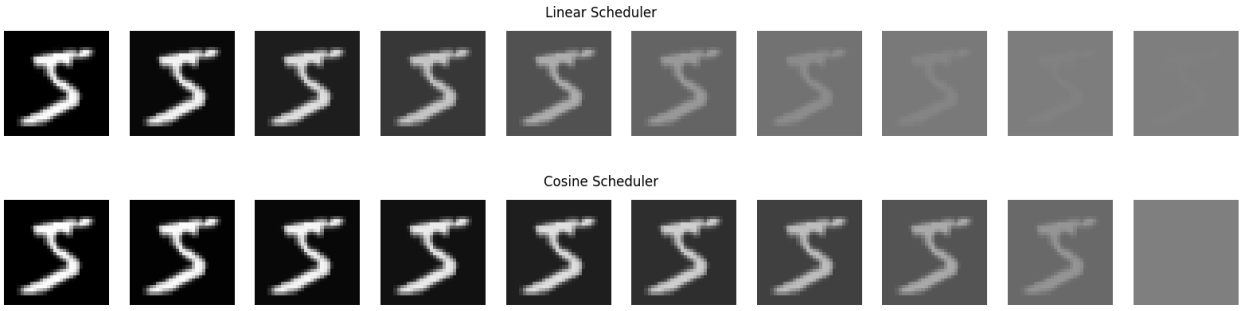
Ans. The linear scheduler tends to add noise faster in case of large `T`. In case of small T, the cosine scheduler still make the image to gaussian noise, but the linear scheduler doesn't.

This is an expected output of the following cell. If your output is not the same, please go check that the `sample_marginal_prob` uses the noise when it is given.

```
for image, label in training_data:
    fig_lin, axs_lin = plt.subplots(ncols=10, figsize=(20, 2))
    fig_cos, axs_cos = plt.subplots(ncols=10, figsize=(20, 2))

    for idx, t in enumerate(np.linspace(0, T-1, 10, dtype=int)):
        x = torch.unsqueeze(image, 0)
        t = torch.tensor([t], device=x.device)

        perturbed_data = linear_scheduler.sample_marginal_prob(
            x, t, noise=torch.zeros(x.shape)
        )
        perturbed_data =
unnormalize_to_zero_to_one(perturbed_data).clamp(0, 1)
        perturbed_data = rearrange(torch.squeeze(perturbed_data), "c h
w -> h w c").detach().cpu().numpy()
        axs_lin[idx].imshow(perturbed_data)
```

```
        axs_lin[idx].set_axis_off()

        perturbed_data = cosine_scheduler.sample_marginal_prob(
            x, t, noise=torch.zeros(x.shape)
        )
        perturbed_data =
unnormalize_to_zero_to_one(perturbed_data).clamp(0, 1)
        perturbed_data = rearrange(torch.squeeze(perturbed_data), "c h
w -> h w c").detach().cpu().numpy()
        axs_cos[idx].imshow(perturbed_data)
        axs_cos[idx].set_axis_off()
    fig_lin.suptitle('Linear Scheduler')
    fig_cos.suptitle('Cosine Scheduler')
    plt.show()
    break
```

```
/tmp/ipykernel_496/3489802820.py:80: UserWarning: To copy construct
from a tensor, it is recommended to use sourceTensor.detach().clone()
or sourceTensor.detach().clone().requires_grad_(True), rather than
torch.tensor(sourceTensor).
  alpha_t = torch.tensor(tmp_alpha, device=x_0.device,
dtype=x_0.dtype)
/tmp/ipykernel_496/3489802820.py:96: UserWarning: To copy construct
from a tensor, it is recommended to use sourceTensor.detach().clone()
or sourceTensor.detach().clone().requires_grad_(True), rather than
torch.tensor(sourceTensor).
  return torch.tensor(std, device=t.device)
```



Sampling

The reverse process update x_t to x_{t-1} based on $p_\theta(x_{t-1} \vee x_t) = N\left(\mu_\theta(x_t, t), \Sigma_\theta(x_t, t)\right)$ using θ for estimation iteratively until $t = 0$. There are several famous sampling algorithm to update x_t to x_{t-1} such as DDPM, DDIM, DPM-Solver, and etc. The DDPM sampling is the fundamental one and you are going to reimplement it.

The algorithm below is a pseudocode of DDPM sampling method. Specifically, the DDPM model define $p_\theta(x_{t-1} \vee x_t)$ as

$$p_\theta(x_{t-1} \vee x_t) \quad ¿ N(\mu_\theta, \Sigma_\theta)$$

$$\mu_\theta \quad ¿ \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{1-\alpha_t}{\sqrt{1-\acute{\alpha}_t}}\epsilon_\theta\right)$$

$$\Sigma_\theta \quad ¿ \sigma_t = \sqrt{\frac{1-\acute{\alpha}_{t-1}}{1-\acute{\alpha}_t}\beta_t}$$

However, due to the numerical instability problem, we must re-derive μ_θ such that it is written in the form of x_0 and x_t. To be specific, the estimated clean image $\tilde{x}_0$ can be derived from the estimated noise ϵ_θ and it may not dwell in the image domain $[-1, 1)$ which can incur various issues such as cumulative error, etc. Therefore, it is a crucial step to derive $\tilde{x}_0$ from ϵ_θ and clip it before updating x_t.

The following is a step to represent μ_θ in form of x_t and x_0. Since ($p(x_t \vee x_0, t) = N\left(\sqrt{\acute{\alpha}_t}x_0, (1-\acute{\alpha}_t)I\right)$), the estimated clean noise $\tilde{x}_0$ is:

$$\tilde{x}_0 \quad ¿ \frac{1}{\sqrt{\acute{\alpha}_t}}\left(x_t - \sqrt{1-\acute{\alpha}_t}\epsilon_\theta\right)$$

As $ \mu_\theta = \frac{1}{\sqrt{\alpha_t}} (x_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}t}}*lepsilon*\ theta)$ and $\epsilon_\theta = \frac{x_t - \sqrt{\acute{\alpha}_t}\tilde{x}_0}{\sqrt{1-\acute{\alpha}_t}}$ (from the equation above). $\mu_\theta $ becomes:

$$\mu_\theta \quad ¿ \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{1-\alpha_t}{1-\acute{\alpha}_t}\left(x_t - \sqrt{\acute{\alpha}_t}\tilde{x}_0\right)\right)$$

$$¿ \quad ¿ \sqrt{\alpha_t}\frac{1-\acute{\alpha}_{t-1}}{1-\acute{\alpha}_t}x_t + \sqrt{\acute{\alpha}_{t-1}}\frac{1-\alpha_t}{1-\acute{\alpha}_t}\tilde{x}_0$$

Since $p_\theta(x_{t-1} | x_t) = \mathcal{N}(\mu_\theta, \Sigma_\theta) = \mu_\theta + \sigma_t\ mathbf{z} $, x_{t-1} is

$$x_{t-1} = \sqrt{\alpha_t}\frac{1-\acute{\alpha}_{t-1}}{1-\acute{\alpha}_t}x_t + \sqrt{\acute{\alpha}_{t-1}}\frac{1-\alpha_t}{1-\acute{\alpha}_t}\tilde{x}_0 + \sigma_t z$$

Instruction

TODO 7: perform one step update (sample $x_{t-1} \sim p_\theta(x_{t-1} \vee x_t)$)

$$\mathrm{x}_{\text{mean}} = \sqrt{\alpha_t}\frac{1-\acute{\alpha}_{t-1}}{1-\acute{\alpha}_t}x_t + \sqrt{\acute{\alpha}_{t-1}}\frac{1-\alpha_t}{1-\acute{\alpha}_t}\tilde{x}_0$$

$$\sigma_t = \sqrt{\frac{1-\acute{\alpha}_{t-1}}{1-\acute{\alpha}_t}\beta_t}$$

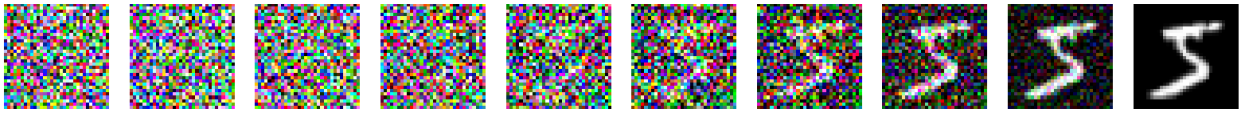
$$\mathrm{x}_{\text{update}} = \mathrm{x}_{\text{mean}} + \sigma_t z$$

```
class DDPMSampler:
    def __init__(self, pred_x0_func, schedule):
        """ Sampler Abstract class

        Args:
            pred_x0_func: A function that predicts clean image x_0
given x_t (tensor shape (B, C, H, W)) and t (tensor shape (B)).
            schedule: A diffusion scheduler contains alpha and beta
parameters.

        """
        self.pred_x0_func = pred_x0_func
        self.schedule = schedule

    def update_step(self, x_t, t, context):
        """One step update

        Update x_t to x_{t-1} following DDPM update rule.

        Args:
            x_t (torch.tensor): An image at diffusion step t.
            t (torch.tensor): A diffusion timestep.

        Returns:
            x_mean (torch.tensor): The noiseless mean of the reverse
process (not adding noise yet)
            x_update (torch.tensor): The updated image from a reverse
process (mean + noise)

        """
        # TODO 7: perform one step update (sample x_{t-1} from p_\
theta(x_{t-1} | x_t))
        # Note: This function return x_mean and x_update (the sampled
x_{t-1}).
        #       At the last step of reverse process, we use x_mean
insetad of x_update because
        #       we assume that it is noiseless.
        z = torch.randn_like(x_t) # standard gaussian
        pred_x_0 = self.pred_x0_func(x_t, t, context) # the output of
pred_x0_func is already clipped. You do not need to clip it anymore.

        # calculate the mean and std of p_\theta(x_{t-1} | x_t) where
the mean is calculate from the equation above (in form of x_0).

        alpha_t = self.schedule.alpha[t].view(-1, 1, 1, 1)
        alpha_cumprod_t = self.schedule.alpha_cumprod[t].view(-1, 1,
1, 1)
        alpha_cumprod_prev_t =
self.schedule.alpha_cumprod_prev[t].view(-1, 1, 1, 1)
        beta_t = self.schedule.beta[t].view(-1, 1, 1, 1)
        coef_x_t = (alpha_t ** 0.5) * ((1 - alpha_cumprod_prev_t) / (1
```

```
- alpha_cumprod_t))

        coef_x_0 = (alpha_cumprod_prev_t ** 0.5) * ((1 - alpha_t) / (1
- alpha_cumprod_t))
        x_mean = coef_x_t * x_t + coef_x_0 * pred_x_0
        var = ((1 - alpha_cumprod_prev_t) / (1 - alpha_cumprod_t) *
beta_t) ** 0.5
        x_update = x_mean + var * z

        return x_mean, x_update


    def sampling(self, x_T, context, return_all=False):
        """Sampling new image from prior sample

        Args:
            x_T (torch.tensor): A prior sample, sampling from Standard
Normal Distribution.
            return_all (bool): If True, return every x_t for all t.
Otherwise, only return x_0. Default to False.

        Returns:
            x_0 (torch.tensor): The generated images given prior
samples, assuming x_0 is noiseless.

        """
        x_t = x_T
        T = self.schedule.T
        reverse_process = [x_T]
        for t in reversed(range(0, T)):
            vec_t = torch.ones(x_T.shape[0], dtype=int,
device=x_T.device) * t
            x_mean, x_t = self.update_step(x_t, vec_t, context)
            # append x_t to reverse_process (Do not forget the last
step.)
            reverse_process.append(x_t if t > 0 else x_mean)

        # if return_all is True then return reverse_process (list of
x_t), otherwise return x_0
        reverse_process = torch.cat(reverse_process, dim=0) if
return_all else reverse_process[-1]
        return reverse_process  # In the last step, we assume x_0 is
noiseless.
```

For a sanity check, we will set predicted x_0 function to predict the ground truth. The following code perform a reverse process.

```
for image, label in training_data:
    fig, axs = plt.subplots(ncols=10, figsize=(20, 2))


    x = torch.unsqueeze(image, 0)
    t = torch.tensor([t], device=x.device)

    pred_gt = lambda x_t, t, context: x
    sampler = DDPMSampler(pred_gt, linear_scheduler)
    x_T = linear_scheduler.prior_sampling(RESOLUTION)

    perturbed_data = sampler.sampling(x_T, return_all=True,
context=torch.zeros(x_T.shape[0]))
    perturbed_data =
unnormalize_to_zero_to_one(perturbed_data).clamp(0, 1)

    for idx, t in enumerate(np.linspace(0, T, 10, dtype=int)):
        x_t = rearrange(torch.squeeze(perturbed_data[t]), "c h w -> h
w c").detach().cpu().numpy()

        axs[idx].imshow(x_t)
        axs[idx].set_axis_off()


    plt.show()
    break
```



Model Architecture

Due to its promising result in the computer vision field, Unet was adapted to the diffusion model to predict the noise in noisy images. In addition, the Unet model in diffusion was slightly modified to enable the controllable property by adding an attention block.

To make it simple, we divide the Unet model into 4 modules.

1. Upsample & Downsample
2. Positional Embedding
3. ResnetBlock
4. Attention

Downsample and Upsample

The downsample module is responsible for reducing the features size while the upsample module is for expanding the features back.

The upsample layer consists of two components:

1. Upsample module with the default setting and scale factor of 2.
2. Convolution module (kernel size = 3, stride = 1, and padding = 1) for extracting the feature.

On the other hand, the downsample layer is just a single convolution module (kernel size = 3, stride = 2, and padding = 1).

```
import math
import torch.nn as nn
import torch.nn.functional as F
from einops.layers.torch import Rearrange
from functools import partial

def exists(x):
    return x is not None

def default(val, d):
    if exists(val):
        return val
    return d() if callable(d) else d

def Upsample(dim, dim_out = None):
    return nn.Sequential(
        nn.Upsample(scale_factor = 2, mode = 'nearest'),
        nn.Conv2d(dim, default(dim_out, dim), 3, padding = 1)
    )

def Downsample(dim, dim_out = None):
    return nn.Sequential(
        nn.Conv2d(dim, default(dim_out, dim), 3, padding=1, stride=2)
    )
```

Sinusoidal Positional Embedding

The sinusoidal positional embedding layer is a layer used for incorporating temporal information into an embedding. Similar to the positional embedding in a transformer, it uses the following equations:

$$PE_{(timestep,2i)} = \sin\left(timestep/10000^{2i/dim}\right)$$

$$PE_{(timestep,2i+i)} = \cos\left(timestep/10000^{2i/dim}\right)$$

```
class SinusoidalPosEmb(nn.Module):
    def __init__(self, dim, theta = 10000):
        super().__init__()
        self.dim = dim
        self.theta = theta
```

```
    def forward(self, x):
        device = x.device
        half_dim = self.dim // 2
        emb = math.log(self.theta) / (half_dim - 1)
        emb = torch.exp(torch.arange(half_dim, device=device) * -emb)
        emb = x[:, None] * emb[None, :]
        emb = torch.cat((emb.sin(), emb.cos()), dim=-1)
        return emb
```

ResnetBlock

The ResnetBlock can be separated into 3 modules:

1. timestep extraction: A module is used to extract the feature in timestep embedding
2. ConvBlock1: It composes of convolution layer followed by groupnorm and activation function. In the first block, we perform an additional operation between the features from the timestep extraction module and the features after convolution and groupnorm (before performing activation function).
3. ConvBlock2: Similar to ConvBlock1 but ignore the timestep embedding
4. ConvResidual: It performs a residual connection between the previous module and the input (which is passed to convolutional layer to ensure that the dimension is aligned.).

Figure ResnetBlock

```
# building block modules

class Block(nn.Module):
    def __init__(self, dim, dim_out, groups = 8):
        super().__init__()
        self.proj = nn.Conv2d(dim, dim_out, 3, padding = 1)
        self.norm = nn.GroupNorm(groups, dim_out)
        self.act = nn.SiLU()

    def forward(self, x, scale_shift = None):
        x = self.proj(x)
        x = self.norm(x)

        if exists(scale_shift):
            x += scale_shift

        x = self.act(x)
        return x

class ResnetBlock(nn.Module):
    def __init__(self, dim, dim_out, *, time_emb_dim = None, groups =
8):
        super().__init__()
```

```
        self.mlp = nn.Sequential(
            nn.SiLU(),
            nn.Linear(time_emb_dim, dim_out)
        ) if exists(time_emb_dim) else None

        self.block1 = Block(dim, dim_out, groups = groups)
        self.block2 = Block(dim_out, dim_out, groups = groups)
        self.res_conv = nn.Conv2d(dim, dim_out, 1) if dim != dim_out
else nn.Identity()

    def forward(self, x, time_emb = None):

        scale_shift = None
        if exists(self.mlp) and exists(time_emb):
            time_emb = self.mlp(time_emb)
            time_emb = rearrange(time_emb, 'b c -> b c 1 1')
            scale_shift = time_emb

        h = self.block1(x, scale_shift = scale_shift)

        h = self.block2(h)

        return h + self.res_conv(x)
```

TODO 8: What does this line `time_emb = rearrange(time_emb, 'b c -> b c 1 1')` do in ResnetBlock? Where does it relate to the above figure?

Ans

```
At the line where timestamp extraction connect to the main line.
It expands the shape to be able to add the tensor because we
want to add this time embedding to every pixel (width and height).
```

Attention

To control the diffusion model, we insert the attention block into the unet model. The label in MNIST dataset is transform into embedding through the embedding layer and then it is passed into attention to perform a cross attention to control the output based on the the label.

The architecture of AttentionBlock:

Note: The image features is rearranged before passing through self-attention layer. We observe the spatial dimension (width and height) as a sequence length.

TODO 9: What is the output's shape of `torch.einsum("b h i d, b h j d -> b h i j", q, k)`? How is each entry in the output calculated? Given $Q \in R^{B,H,I,D}$ and $K \in R^{B,H,J,D}$.

Ans

The output of matrix multiplication:

$$Q \times K^T = C$$

where $Q \in R^{B,H,I,D}$ and $K \in R^{B,H,J,D}$

- Shape of C -> $C \in R^{B,H,I,J}$
- The entry in C -> $c_{b,h,i,j} = \sum_{d=1}^{D} q_{b,h,i,d} k_{b,h,j,d}$

The format of the answer should be similar to this: The output of matrix multiplication:

$$A \times B = C$$

where $A \in R^{m,n}$ and $B \in R^{n,p}$

- Shape of C -> $C \in R^{m,p}$
- The entry in C -> $c_{i,j} = \sum_{k=1}^{n} a_{i,k} b_{k,j}$ for $i = 1, \ldots, m$ and $j = 1, \ldots, p$

```python
class MultiHeadSelfAttention(nn.Module):
    def __init__(self, n_heads, dim):
        """ Multi-head self attention

        Args:
            n_heads (int): The number of distinct representation to
learn.
            dim (int): The number of channels (eg. the size of
embedding vector)

        """
        super().__init__()
        self.n_heads = n_heads
        self.dim = dim
        self.dim_head = dim // n_heads
        _dim = self.dim_head * n_heads
        self.to_qkv = nn.Linear(dim, 3 * _dim, bias=False)
        self.linear = nn.Linear(_dim, dim)

    def forward(self, x):
        qkv = self.to_qkv(x)
        q, k, v = tuple(rearrange(qkv, "b l (k h d) -> k b h l d",
h=self.n_heads, k=3)) # decompose to q, k, v
```

```
        # TODO 9: What does torch.einsum do?
        attention = F.softmax(torch.einsum("b h i d, b h j d -> b h i
j", q, k) / (self.dim_head ** 0.5), dim=-1)
        output = torch.einsum("b h i j, b h j d -> b h i d",
attention, v)
        output = rearrange(output, "b h l d -> b l (h d)")
        output = self.linear(output)
        return output

class MultiHeadCrossAttention(nn.Module):
    def __init__(self, n_heads, dim_emb, dim_cross):
        """ Multi-head cross attention

        Args:
            n_heads (int): The number of distinct representation to
learn.
            dim_emb (int): The number of channels in embedding for
query.
            dim_cross (int): The number of channels in embedding for
key and value.
        """
        super().__init__()
        self.n_heads = n_heads
        self.dim_emb = dim_emb
        self.dim_cross = dim_cross
        self.dim_head = dim_emb // n_heads
        _dim = self.dim_head * n_heads
        self.to_q = nn.Linear(dim_emb, _dim, bias=False)
        self.to_kv = nn.Linear(dim_cross, 2*_dim, bias=False)
        self.linear = nn.Linear(_dim, dim_emb)

    def forward(self, x, context):
        q = self.to_q(x)
        kv = self.to_kv(context)

        q = rearrange(q, "b l (h d) -> b h l d", h=self.n_heads)
        k, v = tuple(rearrange(kv, "b l (k h d) -> k b h l d", k=2,
h=self.n_heads))

        attention = F.softmax(torch.einsum("b h i d, b h j d -> b h i
j", q, k) / (self.dim_head ** 0.5), dim=-1)
        output = torch.einsum("b h i j, b h j d -> b h i d",
attention, v)
        output = rearrange(output, "b h l d -> b l (h d)")
        output = self.linear(output)
        return output

class AttentionBlock(nn.Module):
    def __init__(self, n_heads, in_channels, context_channels=128):
        super().__init__()
```

```
        self.groupnorm = nn.GroupNorm(32, in_channels, eps=1e-6)
        self.conv_input = nn.Conv2d(in_channels, in_channels,
kernel_size=1, padding=0)

        self.layernorm_1 = nn.LayerNorm(in_channels)
        self.attention_1 = MultiHeadSelfAttention(n_heads,
in_channels)
        self.layernorm_2 = nn.LayerNorm(in_channels)
        self.attention_2 = MultiHeadCrossAttention(n_heads,
in_channels, context_channels)
        self.layernorm_3 = nn.LayerNorm(in_channels)
        self.linear_geglu_1  = nn.Linear(in_channels, 4 * in_channels)
# 4 * in_channels * 2
        self.linear_geglu_2 = nn.Linear(4 * in_channels, in_channels)

        self.conv_output = nn.Conv2d(in_channels, in_channels,
kernel_size=1, padding=0)

    def forward(self, x, context):
        residue_long = x

        x = self.groupnorm(x)
        x = self.conv_input(x)

        n, c, h, w = x.shape
        x = x.view((n, c, h * w))   # (n, c, hw)
        x = x.transpose(-1, -2)  # (n, hw, c)

        residue_short = x
        x = self.layernorm_1(x)
        x = self.attention_1(x)
        x += residue_short

        residue_short = x
        x = self.layernorm_2(x)
        x = self.attention_2(x, context)
        x += residue_short

        residue_short = x
        x = self.layernorm_3(x)
        # x, gate = self.linear_geglu_1(x).chunk(2, dim=-1)
        # x = x * F.gelu(gate)
        x = self.linear_geglu_1(x)
        x = F.gelu(x)
        x = self.linear_geglu_2(x)
        x += residue_short

        x = x.transpose(-1, -2)  # (n, c, hw)
        x = x.view((n, c, h, w))    # (n, c, h, w)
```

```
        return self.conv_output(x) + residue_long
```

Unet

We combine every modules to build a Unet.

```
def cast_tuple(t, length = 1):
    if isinstance(t, tuple):
        return t
    return ((t,) * length)

def divisible_by(numer, denom):
    return (numer % denom) == 0
# model

class Unet(nn.Module):
    def __init__(
        self,
        dim,
        init_dim = None,
        out_dim = None,
        dim_mults = (1, 2, 4, 8),
        channels = 3,
        self_condition = False,
        resnet_block_groups = 8,
        sinusoidal_pos_emb_theta = 10000,
        attn_heads = 4,
        context_dim = 768,
    ):
        super().__init__()

        # determine dimensions

        self.channels = channels
        self.self_condition = self_condition
        input_channels = channels * (2 if self_condition else 1)

        init_dim = default(init_dim, dim)
        self.init_conv = nn.Conv2d(input_channels, init_dim, 7,
padding = 3)

        dims = [init_dim, *map(lambda m: dim * m, dim_mults)]
        in_out = list(zip(dims[:-1], dims[1:]))

        block_klass = partial(ResnetBlock, groups =
resnet_block_groups)

        # time embeddings
```

```
        time_dim = dim * 4


        sinu_pos_emb = SinusoidalPosEmb(dim, theta =
sinusoidal_pos_emb_theta)
        fourier_dim = dim

        self.time_mlp = nn.Sequential(
            sinu_pos_emb,
            nn.Linear(fourier_dim, time_dim),
            nn.GELU(),
            nn.Linear(time_dim, time_dim)
        )

        # attention
        num_stages = len(dim_mults)

        FullAttention = partial(AttentionBlock,
context_channels=context_dim, n_heads=attn_heads)

        # layers

        self.downs = nn.ModuleList([])
        self.ups = nn.ModuleList([])
        num_resolutions = len(in_out)

        for ind, (dim_in, dim_out) in enumerate(in_out):
            is_last = ind >= (num_resolutions - 1)

            attn_klass = FullAttention

            self.downs.append(nn.ModuleList([
                block_klass(dim_in, dim_in, time_emb_dim = time_dim),
                block_klass(dim_in, dim_in, time_emb_dim = time_dim),
                attn_klass(in_channels = dim_in),
                Downsample(dim_in, dim_out) if not is_last else
nn.Conv2d(dim_in, dim_out, 3, padding = 1)
            ]))

        mid_dim = dims[-1]
        self.mid_block1 = block_klass(mid_dim, mid_dim, time_emb_dim =
time_dim)
        self.mid_attn = FullAttention(in_channels = mid_dim)
        self.mid_block2 = block_klass(mid_dim, mid_dim, time_emb_dim =
time_dim)

        for ind, (dim_in, dim_out) in enumerate(reversed(in_out)):
            is_last = ind == (len(in_out) - 1)

            attn_klass = FullAttention
```

```
            self.ups.append(nn.ModuleList([
                block_klass(dim_out + dim_in, dim_out, time_emb_dim =
time_dim),
                block_klass(dim_out + dim_in, dim_out, time_emb_dim =
time_dim),
                attn_klass(in_channels = dim_out),
                Upsample(dim_out, dim_in) if not is_last else
nn.Conv2d(dim_out, dim_in, 3, padding = 1)
            ]))

        default_out_dim = channels
        self.out_dim = default(out_dim, default_out_dim)

        self.final_res_block = block_klass(dim * 2, dim, time_emb_dim
= time_dim)
        self.final_conv = nn.Conv2d(dim, self.out_dim, 1)

    @property
    def downsample_factor(self):
        return 2 ** (len(self.downs) - 1)

    def forward(self, x, time, context):
        assert all([divisible_by(d, self.downsample_factor) for d in
x.shape[-2:]]), f'your input dimensions {x.shape[-2:]} need to be
divisible by {self.downsample_factor}, given the unet'

        if self.self_condition:
            x_self_cond = default(x_self_cond, lambda:
torch.zeros_like(x))
            x = torch.cat((x_self_cond, x), dim = 1)

        x = self.init_conv(x)
        r = x.clone()

        t = self.time_mlp(time)

        h = []

        for block1, block2, attn, downsample in self.downs:
            x = block1(x, t)
            h.append(x)

            x = block2(x, t)
            x = attn(x, context)

            h.append(x)

            x = downsample(x)

        x = self.mid_block1(x, t)
```

```
        x = self.mid_attn(x, context)
        x = self.mid_block2(x, t)

        for block1, block2, attn, upsample in self.ups:
            x = torch.cat((x, h.pop()), dim = 1)
            x = block1(x, t)

            x = torch.cat((x, h.pop()), dim = 1)
            x = block2(x, t)
            x = attn(x, context)

            x = upsample(x)

        x = torch.cat((x, r), dim = 1)

        x = self.final_res_block(x, t)
        return self.final_conv(x)

from torchinfo import summary
model = Unet(
    dim = 32,
    dim_mults = (1, 2, 4, 8),
)


summary(model, input_size=[(32, 3, 32, 32), (32,), (32, 10, 768)],
dtypes=[torch.float, torch.long, torch.float], depth=1)

==========================================================================
==================================
Layer (type:depth-idx)                                  Output Shape
Param #
==========================================================================
==================================
Unet                                                    [32, 3, 32,
32]           --
├─Conv2d: 1-1                                           [32, 32, 32,
32]           4,736
├─Sequential: 1-2                                       [32, 128]
20,736
├─ModuleList: 1-3                                       --
2,034,912
├─ResnetBlock: 1-4                                      [32, 256, 4,
4]            1,214,208
├─AttentionBlock: 1-5                                   [32, 256, 4,
4]            1,446,144
├─ResnetBlock: 1-6                                      [32, 256, 4,
4]            1,214,208
├─ModuleList: 1-7                                       --
6,853,344
```

```
├─ResnetBlock: 1-8                                       [32, 32, 32,
32]          34,048
├─Conv2d: 1-9                                           [32, 3, 32,
32]           99
==========================================================================
==================================
Total params: 12,822,435
Trainable params: 12,822,435
Non-trainable params: 0
Total mult-adds (Units.GIGABYTES): 14.27
==========================================================================
==================================
Input size (MB): 1.38
Forward/backward pass size (MB): 836.92
Params size (MB): 51.29
Estimated Total Size (MB): 889.59
==========================================================================
==================================
```

Diffusion

In this section, we are going to assemble every components into diffusion class.

The functions in Diffusion class

- predict_x0: transforming the noise predicted from the model into $\tilde{x}_0$.
  $\tilde{x}_0 = \frac{1}{\sqrt{\acute{\alpha}_t}}\left(x_t - \sqrt{1-\acute{\alpha}_t}\epsilon_\theta\right)$
- p_losses: calculate loss for one step update (the algorithm of training process is shown below)
- forward: a forward pass of the model (The output is the noise in x_t)
- sample: perform a reverse process

Instruction

TODO 10: implement predict_x0 function TODO 11: implement p_losses function

```
import torch.nn as nn

class Diffusion(nn.Module):
    def __init__(self, model, scheduler, resolution, sampler="ddpm",
loss="mse", device="cuda"):
        super().__init__()
        self.model = model
        self.scheduler = scheduler
        self.resolution = resolution
        self.device = device
```

```
        if loss == "mse":
            self.loss_fn = nn.MSELoss()
        elif loss == "mae":
            self.loss_fn = nn.L1Loss()
        else:
            raise NotImplementedError

        if sampler == "ddpm":
            self.sampler = DDPMSampler(self.predict_x0, scheduler)
        else:
            raise NotImplementedError

    def predict_x0(self, x_t, t, context):
        noise = self.model(x_t, t, context).detach()
        # TODO 10: convert the noise into x_0
        # Note: Do not forget to clip the value of the predicted x_0
to be in the range of [-1, 1] (using torch.clamp).
        alpha_t = self.scheduler.alpha_cumprod[t].reshape(-1, 1, 1, 1)
# (batch, 1, 1, 1)
        x_0 = torch.clamp((x_t - ((1 - alpha_t) ** 0.5) * noise) /
(alpha_t ** 0.5), min=-1, max=1)
        return x_0

    def p_losses(self, x_0, context):
        # TODO 11: calculate loss of one step
        # 3 Steps:
        #   1. random a gaussian noise (z).
        #   2. sample time step uniformly from [0, T] where T is an
attribute in self.scheduler
        #   3. sample x_t using scheduler.sample_marginal_prob where
the noise come from step 1.
        #   4. predict noise give x_t, t, context using self.model
        #   5. calculate loss given predicted noise and noise from
step 1 uinsg self.loss_fn.
        # !!!! Don't forget to consider about batch dimension.
        z = torch.randn_like(x_0)
        batch = x_0.shape[0]
        t = torch.randint(0, self.scheduler.T,
(batch, )).to(self.device)
        x_t = self.scheduler.sample_marginal_prob(x_0, t, noise=z)
        predicted_noise = self.model(x_t, t, context)
        loss = self.loss_fn(predicted_noise, z)
        return loss

    def forward(self, x_t, t, context):
        return self.model(x_t, t, context)

    def sample(self, batch_size, context):
        x_T = self.scheduler.prior_sampling(self.resolution,
```

```
batch_size=batch_size, num_channels=3).to(self.device)
        sampler = self.sampler.sampling(x_T,
context).detach().cpu().numpy().squeeze()
        return sampler


    def to(self, device):
        self.device = device
        self.model.to(device)
        self.scheduler.to(device)
```

Training

After building a diffusion model, we create trainer class to train our model.

These are the important functions that you should read:

1. `_step` (TODO 12): A one step function that return loss of one step update.
2. `train_loop`: A function to train the model for one epoch.
3. `val_loop`: A valdation function
4. `sampling_image`: Sample new images using reverse process

Note: We will control the diffusion model using the `self.embedding=nn.Embedding` to transform the digit into embedding. Then the embedding is passed through the Unet model and perform the cross attention with the image features.

Instruction

TODO 12: implement `_step` function

```
class Trainer:
    def __init__(
        self,
        model,
        train_dataloader,
        val_dataloader,
        ckpt_path,
        resolution=None,
        lr=1e-4,
        device="cuda",
    ):
        self.model = model
        self.embedding = nn.Embedding(10, 768) # MNIST has 10 classes
and embedding size is 768.

        self.train_dataloader = train_dataloader
        self.val_dataloader = val_dataloader
        self.resolution = next(iter(train_dataloader))[0].shape[-2:]
if resolution == None else resolution
```

```
        self.lr = lr
        self.configure_optimizers()

        self.ckpt_path = ckpt_path
        self.device = device
        self.epoch = 0
        self.mode = "train"

        self.to(device)

    def configure_optimizers(self):
        """Construct Optimizer"""
        self.optimizer = torch.optim.Adam(self.parameters(),
lr=self.lr)

    def _step(self, batch):
        """One step forward pass

        Args:
            batch (Tuple[torch.tensor]): A mini-batch data to be
processed. The first index must contain images
                and the latter are the corresponded label. The shape
of image is (B, C, H, W),
                while the shape of label is (B).

        Returns:
            loss (torch.tensor): A loss value to be used for updating
the model.

        """
        x, context = batch
        # TODO 12: calculate the loss given batch
        # Pseudo code:
        #   1. acquire the context embedding through self.embedding
        #   2. expand dimension such that the shape of embedding is
(batch_size, 1, dimension)
        #   3. calculate loss using self.model.p_losses
        context_embedding =
self.embedding(context).reshape(x.shape[0], 1, -1)
        losses = self.model.p_losses(x, context_embedding)
        return losses

    def optimizer_step(self, loss):
        """Update model based on the given loss.

        Args:
            loss (torch.tensor): A loss value to be differentiated
(Note: must be able to compute a gradient.)

        """
```

```
        loss.backward()
        self.optimizer.step()
        self.optimizer.zero_grad()

    def train_loop(self, epoch):
        """One epoch training loop.

        Args:
            epoch (int): A current epoch.

        Returns:
            train_loss (np.array): A list of loss in each batch.

        """
        self.train()
        with tqdm(self.train_dataloader, unit="batch", leave=False) as
tbatch:
            train_loss = []
            for batch in tbatch:
                tbatch.set_description(f"Epoch {epoch+1}")
                batch = list(
                    map(lambda x: x.to(self.device), batch)
                )  # allocate data on the pre-defined device.
                loss = self._step(batch)  # forward pass and calculate
loss
                self.optimizer_step(loss)  # backward pass / updating
the parameters
                train_loss.append(loss.item())
                tbatch.set_postfix(loss=loss.item())
            train_loss = np.array(train_loss)
        return train_loss

    def val_loop(self, epoch):
        """One epoch validating loop.

        Args:
            epoch (int): A current epoch.

        Returns:
            train_loss (np.array): A list of loss in each batch.

        """
        self.eval()
        # with self.ema.average_parameters():  # Copy EMA weight to
model and restore after exiting `with`
        with tqdm(self.val_dataloader, unit="batch", leave=False) as
tbatch:
            val_loss = []
            for batch in tbatch:
                tbatch.set_description(f"Epoch {epoch+1}")
```

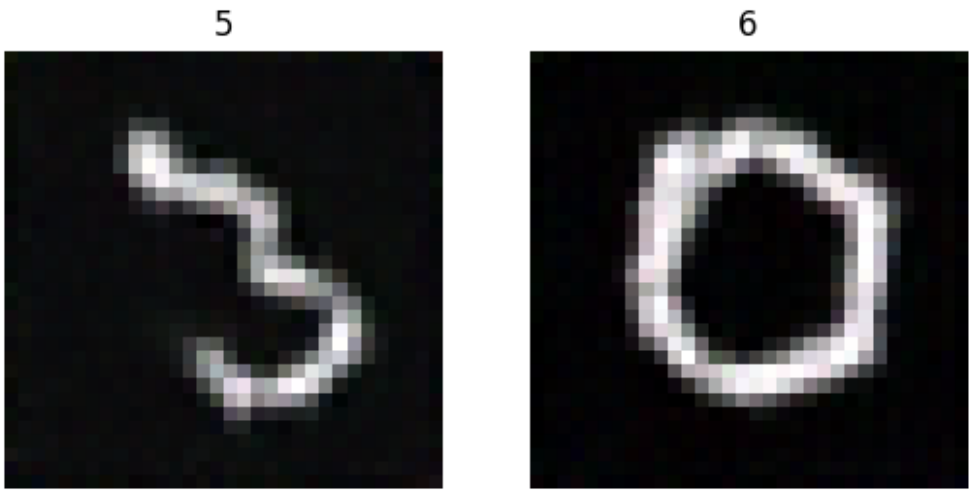
```
                batch = list(
                    map(lambda x: x.to(self.device), batch)
                )  # allocate data on the pre-defined device.
                loss = (
                    self._step(batch).detach().cpu()
                )  # forward pass and calculate loss
                val_loss.append(loss.item())
                tbatch.set_postfix(loss=loss.item())
            val_loss = np.array(val_loss)
        return val_loss

    def fit(self, epochs, num_val_sampler=2, ckpt_path=None,
resume=False, sampling_round=10):
        """Train the model

        Args:
            epochs (int): A number of epochs to be trained.

        Returns:
            train_epoch (np.array): A list of training loss in each
epoch.
            val_epoch (np.array): A list of validation loss in each
epoch.

        """
        best_val_loss = None
        ckpt_path = self.ckpt_path if ckpt_path is None else ckpt_path
        last_path = os.path.join(ckpt_path, "last_weight.ckpt")
        print("Save:", last_path)
        self.to(self.device)

        train_loss_epoch = []
        val_loss_epoch = []

        epochs = range(self.epoch, self.epoch+epochs) if resume ==
True else range(epochs)
        for epoch in epochs:
            self.epoch = epoch

            train_loss = self.train_loop(epoch) # Training Model
            val_loss = self.val_loop(epoch) # Evaluating Model

            if (epoch+1)%sampling_round == 0 or epoch == 0:
                self.sampling_image(num_val_sampler) # Generating new
images

            print(
                f"Epoch {epoch+1}: Train Loss
{train_loss.mean().item()}, Val Loss {val_loss.mean().item()}"
            )
```

```
            train_loss_epoch.append(train_loss.mean().item())
            val_loss_epoch.append(val_loss.mean().item())

            torch.cuda.empty_cache() # Clear GPU cache

        self.save(last_path)
        train_loss_epoch = np.array(train_loss_epoch)
        val_loss_epoch = np.array(val_loss_epoch)
        return train_loss_epoch, val_loss_epoch

    def sampling_image(self, num_images):
        """Sampling new images

        The sampling images was exhibited in a column format, using
matplotlib.

        Args:
            num_images (int): A number of images to be generated.

        """
        self.eval()
        fig, axs = plt.subplots(ncols=num_images)
        label = torch.randint(0, 10, (num_images,),
device=self.device)
        context = self.embedding(label).unsqueeze(1)
        sampled_images = self.model.sample(batch_size = num_images,
context=context)
        for idx_sampler in range(num_images):

axs[idx_sampler].imshow(rearrange(unnormalize_to_zero_to_one(sampled_i
mages[idx_sampler]), "c h w -> h w c"), cmap="gray")
            axs[idx_sampler].set_title(label[idx_sampler].item())
            axs[idx_sampler].set_axis_off()
        plt.show()

    def save(self, path):
        """ Save trainer state """
        path = self.ckpt_path if path is None else path
        torch.save({
            "epoch": self.epoch,
            "model_state_dict": self.model.state_dict(),
            "optimizer_state_dict": self.optimizer.state_dict(),
            "embedding": self.embedding.state_dict(),
        }, path)

    def load(self, path):
        """ Load trainer state """
        checkpoint = torch.load(path)
        self.epoch = checkpoint["epoch"]
        self.model.load_state_dict(checkpoint["model_state_dict"])
```

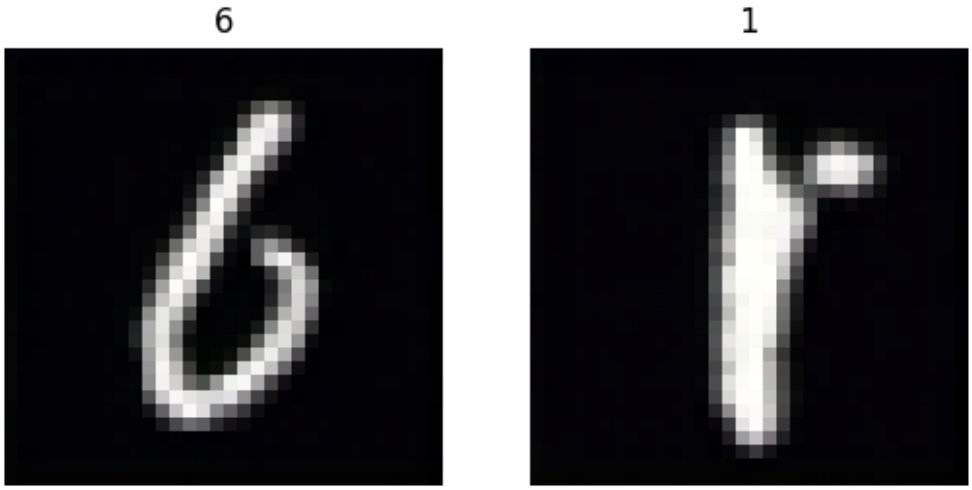
```
self.optimizer.load_state_dict(checkpoint["optimizer_state_dict"])
        self.embedding.load_state_dict(checkpoint["embedding"])

    def train(self):
        """ Set trainer to train mode """
        if self.mode != "train":
            self.model.train()
            self.mode = "train"

    def eval(self):
        """ Set trainer to evaluate mode """
        if self.mode != "eval":
            self.model.eval()
            self.mode = "eval"

    def to(self, device):
        """Set a computational device (eg. cpu or cuda).

        Args:
            device (str): A computational device to be computed on.

        """
        self.device = device
        self.model.to(device)
        self.embedding.to(device)

    def set_dataset(self, train_dataloader, val_dataloader,
resolution=None):
        """Set a new dataset.

        Args:
            train_dataloader (torch.utils.data.DataLoader): A new
training set.
            val_dataloader (torch.utils.data.DataLoader): A new
validation set.
            resolution (Tuple[int], optional): A new resolution of the
images in the given dataset. If None,
                it was set to the size of the first image. Default to
None.

        """
        self.train_dataloader = train_dataloader
        self.val_dataloader = val_dataloader
        self.resolution = next(iter(train_dataloader))[0].shape[-2:]
if resolution == None else resolution

    def parameters(self):
        return [p for p in self.model.parameters() if p.requires_grad]
```

```
model = Unet(
    dim = 64,
    dim_mults = (1, 2, 4, 8),
)

diffusion = Diffusion(
    model,
    linear_scheduler,
    resolution=(32, 32),
)

trainer_config = {
    "model": diffusion,
    "train_dataloader": train_dataloader,
    "val_dataloader": test_dataloader,
    "device": "cuda",
    "ckpt_path": "./",
    "lr": 8e-5,
}

trainer = Trainer(**trainer_config)

epoch = 10
train_losses, val_losses = trainer.fit(epoch)
```

```
Save: ./last_weight.ckpt

Epoch 1:   0%|          | 0/938 [00:00<?,
?batch/s]/tmp/ipykernel_663/1563770698.py:80: UserWarning: To copy
construct from a tensor, it is recommended to use
sourceTensor.detach().clone() or
sourceTensor.detach().clone().requires_grad_(True), rather than
torch.tensor(sourceTensor).
  alpha_t = torch.tensor(tmp_alpha, device=x_0.device,
dtype=x_0.dtype)
/tmp/ipykernel_663/1563770698.py:96: UserWarning: To copy construct
from a tensor, it is recommended to use sourceTensor.detach().clone()
or sourceTensor.detach().clone().requires_grad_(True), rather than
torch.tensor(sourceTensor).
  return torch.tensor(std, device=t.device)
```




```
Epoch 1: Train Loss 0.05937467647819663, Val Loss 0.020687597216504396


Epoch 2: Train Loss 0.017053638224495946, Val Loss
0.014834156740385636


Epoch 3: Train Loss 0.013858756559319112, Val Loss
0.012331746619455753


Epoch 4: Train Loss 0.01254485965297341, Val Loss 0.01288943228780464


Epoch 5: Train Loss 0.01119253784318818, Val Loss 0.010581108198685062


Epoch 6: Train Loss 0.010653741488944112, Val Loss 0.01056634795490154


Epoch 7: Train Loss 0.010084396269398808, Val Loss
0.010049629481924567


Epoch 8: Train Loss 0.009626837565438516, Val Loss
0.008995235764130855


Epoch 9: Train Loss 0.009257309576635486, Val Loss
0.008985543970588096
```

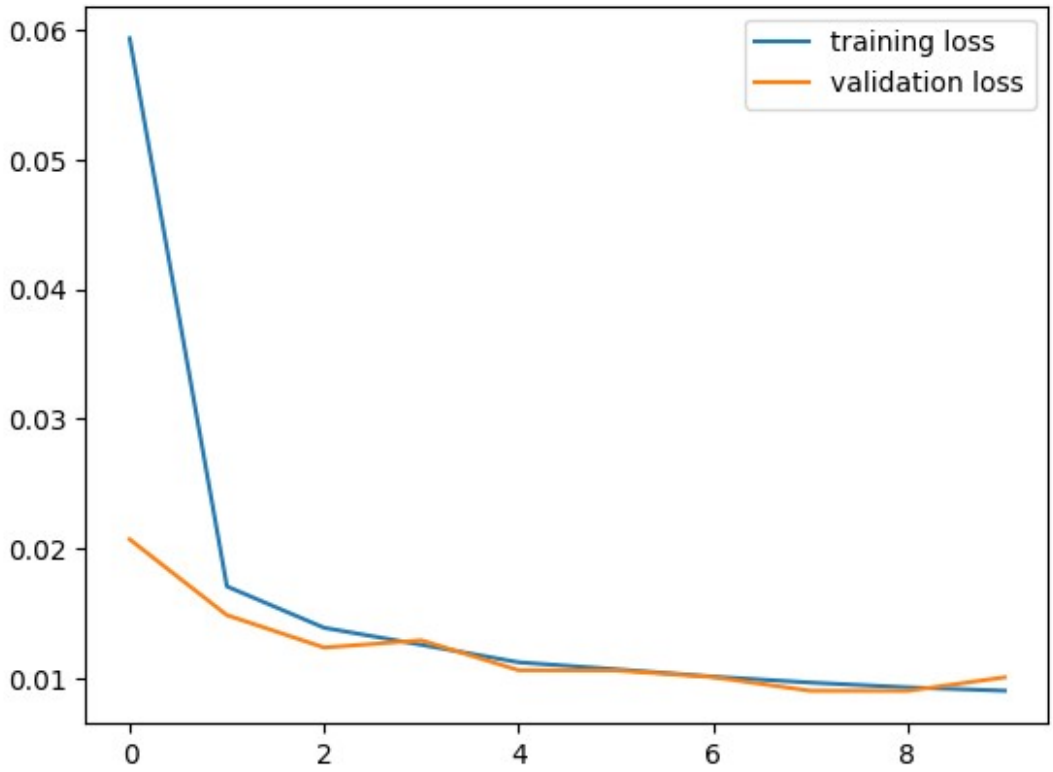



```
Epoch 10: Train Loss 0.008998869050012183, Val Loss
0.01003367988879134
```

Plot training and validation loss

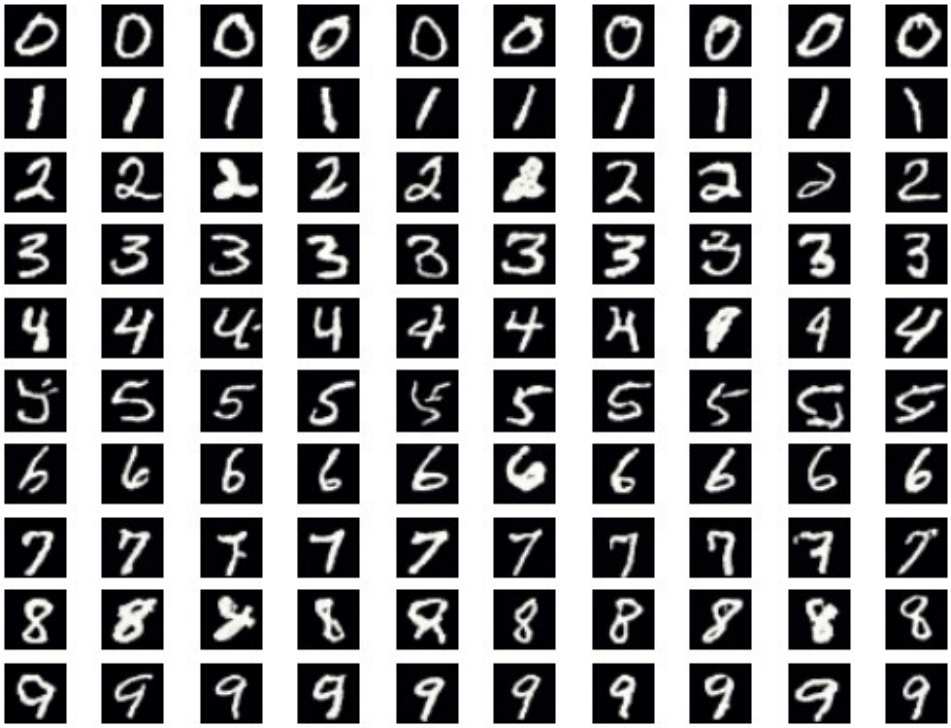
Loss of the last epoch should be around 0.0085 - 0.009.

```
plt.plot(range(epoch), train_losses, label="training loss")
plt.plot(range(epoch), val_losses, label="validation loss")
plt.legend()
plt.show()
```



Inference

```
trainer.eval()
fig, axs = plt.subplots(nrows=10, ncols=10)
for digit in range(10):
    label = torch.ones(10, device=trainer.device, dtype=torch.int) *
digit
    context = trainer.embedding(label).unsqueeze(1)
    sampled_images = trainer.model.sample(batch_size = 10,
context=context)
    for i in range(10):
        axs[digit]
[i].imshow(rearrange(unnormalize_to_zero_to_one(sampled_images[i]), "c
h w -> h w c"), cmap="gray")
        # axs[digit].set_title(label[].item())
        axs[digit][i].set_axis_off()
plt.show()
```



Part 2: Low Rank Adapter (LORA)

With the discovery of the scaling property in the deep learning model, several researchers tend to increase the size of the deep learning model to obtain the emergent property, especially in the natural language processing (NLP) field. For example, the GPT3 contains 175 billion parameters, making it impossible to fine-tune. This trend obstructs students like us from transferring these enormous foundation models on a single GPU (or small resources). Therefore, to alleviate this problem, some researchers invent new methods to fine-tune the model, called parameter-efficient transfer learning, which allows us to train large models on limited resources. Its benefits exist not only in the training process but also during deployment. After fine-tuning the model, we need to save only a small amount of parameters (LoRA weights), allowing us to deploy the foundation model to various downstream tasks using a small amount of storage. One of the prevailing methods is Low Rank Adaptation (LORA).

In this section, we are going to introduce the Low-Rank Adaptation. You are assigned to implement the LORA on the MaskTransfomer model trained by HuggingFace. The model is trained on ADE20k semantic segmentation, and we will transfer this foundation model to the satellite building segmentation dataset using the LoRA method. For the details of MaskFormer, please visit this site: https://huggingface.co/facebook/maskformer-swin-tiny-ade.

Downloading the pre-trained MaskFormer model and Satellite-Building-Segmentation Dataset

```
pip install transformers==4.40.2 huggingface_hub==0.23.0
datasets==2.19.0 torchinfo

Collecting transformers==4.40.2
  Downloading transformers-4.40.2-py3-none-any.whl.metadata (137 kB)
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0.0/138.0 kB ? eta -:--:--
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 138.0/138.0 kB 4.0 MB/s eta
0:00:00
etadata (12 kB)
Collecting datasets==2.19.0
  Downloading datasets-2.19.0-py3-none-any.whl.metadata (19 kB)
Requirement already satisfied: torchinfo in
/usr/local/lib/python3.12/dist-packages (1.8.0)
Requirement already satisfied: filelock in
/usr/local/lib/python3.12/dist-packages (from transformers==4.40.2)
(3.25.2)
Requirement already satisfied: numpy>=1.17 in
/usr/local/lib/python3.12/dist-packages (from transformers==4.40.2)
(2.0.2)
Requirement already satisfied: packaging>=20.0 in
/usr/local/lib/python3.12/dist-packages (from transformers==4.40.2)
(26.0)
Requirement already satisfied: pyyaml>=5.1 in
/usr/local/lib/python3.12/dist-packages (from transformers==4.40.2)
(6.0.3)
Requirement already satisfied: regex!=2019.12.17 in
/usr/local/lib/python3.12/dist-packages (from transformers==4.40.2)
(2025.11.3)
Requirement already satisfied: requests in
/usr/local/lib/python3.12/dist-packages (from transformers==4.40.2)
(2.32.4)
Collecting tokenizers<0.20,>=0.19 (from transformers==4.40.2)
  Downloading tokenizers-0.19.1-cp312-cp312-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (6.7 kB)
Requirement already satisfied: safetensors>=0.4.1 in
/usr/local/lib/python3.12/dist-packages (from transformers==4.40.2)
(0.7.0)
Requirement already satisfied: tqdm>=4.27 in
/usr/local/lib/python3.12/dist-packages (from transformers==4.40.2)
(4.67.3)
Requirement already satisfied: fsspec>=2023.5.0 in
/usr/local/lib/python3.12/dist-packages (from huggingface_hub==0.23.0)
(2025.3.0)
Requirement already satisfied: typing-extensions>=3.7.4.3 in
/usr/local/lib/python3.12/dist-packages (from huggingface_hub==0.23.0)
(4.15.0)
```

```
Requirement already satisfied: pyarrow>=12.0.0 in
/usr/local/lib/python3.12/dist-packages (from datasets==2.19.0)
(18.1.0)
Collecting pyarrow-hotfix (from datasets==2.19.0)
  Downloading pyarrow_hotfix-0.7-py3-none-any.whl.metadata (3.6 kB)
Requirement already satisfied: dill<0.3.9,>=0.3.0 in
/usr/local/lib/python3.12/dist-packages (from datasets==2.19.0)
(0.3.8)
Requirement already satisfied: pandas in
/usr/local/lib/python3.12/dist-packages (from datasets==2.19.0)
(2.2.2)
Requirement already satisfied: xxhash in
/usr/local/lib/python3.12/dist-packages (from datasets==2.19.0)
(3.6.0)
Requirement already satisfied: multiprocess in
/usr/local/lib/python3.12/dist-packages (from datasets==2.19.0)
(0.70.16)
Collecting fsspec>=2023.5.0 (from huggingface_hub==0.23.0)
  Downloading fsspec-2024.3.1-py3-none-any.whl.metadata (6.8 kB)
Requirement already satisfied: aiohttp in
/usr/local/lib/python3.12/dist-packages (from datasets==2.19.0)
(3.13.3)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in
/usr/local/lib/python3.12/dist-packages (from aiohttp-
>datasets==2.19.0) (2.6.1)
Requirement already satisfied: aiosignal>=1.4.0 in
/usr/local/lib/python3.12/dist-packages (from aiohttp-
>datasets==2.19.0) (1.4.0)
Requirement already satisfied: attrs>=17.3.0 in
/usr/local/lib/python3.12/dist-packages (from aiohttp-
>datasets==2.19.0) (25.4.0)
Requirement already satisfied: frozenlist>=1.1.1 in
/usr/local/lib/python3.12/dist-packages (from aiohttp-
>datasets==2.19.0) (1.8.0)
Requirement already satisfied: multidict<7.0,>=4.5 in
/usr/local/lib/python3.12/dist-packages (from aiohttp-
>datasets==2.19.0) (6.7.1)
Requirement already satisfied: propcache>=0.2.0 in
/usr/local/lib/python3.12/dist-packages (from aiohttp-
>datasets==2.19.0) (0.4.1)
Requirement already satisfied: yarl<2.0,>=1.17.0 in
/usr/local/lib/python3.12/dist-packages (from aiohttp-
>datasets==2.19.0) (1.23.0)
Requirement already satisfied: charset_normalizer<4,>=2 in
/usr/local/lib/python3.12/dist-packages (from requests-
>transformers==4.40.2) (3.4.6)
Requirement already satisfied: idna<4,>=2.5 in
/usr/local/lib/python3.12/dist-packages (from requests-
>transformers==4.40.2) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in
```

```
/usr/local/lib/python3.12/dist-packages (from requests-
>transformers==4.40.2) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in
/usr/local/lib/python3.12/dist-packages (from requests-
>transformers==4.40.2) (2026.2.25)
Requirement already satisfied: python-dateutil>=2.8.2 in
/usr/local/lib/python3.12/dist-packages (from pandas-
>datasets==2.19.0) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in
/usr/local/lib/python3.12/dist-packages (from pandas-
>datasets==2.19.0) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in
/usr/local/lib/python3.12/dist-packages (from pandas-
>datasets==2.19.0) (2025.3)
Requirement already satisfied: six>=1.5 in
/usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2-
>pandas->datasets==2.19.0) (1.17.0)
Downloading transformers-4.40.2-py3-none-any.whl (9.0 MB)
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 9.0/9.0 MB 92.5 MB/s eta
0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 401.2/401.2 kB 41.2 MB/s eta
0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 542.0/542.0 kB 51.6 MB/s eta
0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 172.0/172.0 kB 21.9 MB/s eta
0:00:00
anylinux_2_17_x86_64.manylinux2014_x86_64.whl (3.6 MB)
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3.6/3.6 MB 126.9 MB/s eta
0:00:00
ers, datasets
  Attempting uninstall: fsspec
    Found existing installation: fsspec 2025.3.0
    Uninstalling fsspec-2025.3.0:
      Successfully uninstalled fsspec-2025.3.0
  Attempting uninstall: huggingface_hub
    Found existing installation: huggingface_hub 1.7.1
    Uninstalling huggingface_hub-1.7.1:
      Successfully uninstalled huggingface_hub-1.7.1
  Attempting uninstall: tokenizers
    Found existing installation: tokenizers 0.22.2
    Uninstalling tokenizers-0.22.2:
      Successfully uninstalled tokenizers-0.22.2
  Attempting uninstall: transformers
    Found existing installation: transformers 5.0.0
    Uninstalling transformers-5.0.0:
      Successfully uninstalled transformers-5.0.0
  Attempting uninstall: datasets
    Found existing installation: datasets 4.0.0
    Uninstalling datasets-4.0.0:
      Successfully uninstalled datasets-4.0.0
```

```
ERROR: pip's dependency resolver does not currently take into account
all the packages that are installed. This behaviour is the source of
the following dependency conflicts.
gradio 5.50.0 requires huggingface-hub<2.0,>=0.33.5, but you have
huggingface-hub 0.23.0 which is incompatible.
peft 0.18.1 requires huggingface_hub>=0.25.0, but you have
huggingface-hub 0.23.0 which is incompatible.
gcsfs 2025.3.0 requires fsspec==2025.3.0, but you have fsspec 2024.3.1
which is incompatible.
sentence-transformers 5.3.0 requires transformers<6.0.0,>=4.41.0, but
you have transformers 4.40.2 which is incompatible.
diffusers 0.37.0 requires huggingface-hub<2.0,>=0.34.0, but you have
huggingface-hub 0.23.0 which is incompatible.
Successfully installed datasets-2.19.0 fsspec-2024.3.1
huggingface_hub-0.23.0 pyarrow-hotfix-0.7 tokenizers-0.19.1
transformers-4.40.2
```

```
from PIL import Image, ImageDraw
import torch.nn as nn
import torch
import torch.nn.functional as F
from transformers import MaskFormerConfig, MaskFormerModel,
MaskFormerImageProcessor, MaskFormerForInstanceSegmentation

from datasets import load_dataset

model_name = "facebook/maskformer-swin-tiny-ade"
ds = load_dataset("keremberke/satellite-building-segmentation",
name="full")
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/
_token.py:89: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your
settings tab (https://huggingface.co/settings/tokens), set it as
secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to
access public models or datasets.
  warnings.warn(
```


```
{"model_id":"7f0f67e1ff0542c792b5bd81471b595a","version_major":2,"vers
ion_minor":0}

{"model_id":"168ba21f66f040389c53a9afeb161320","version_major":2,"vers
ion_minor":0}

{"model_id":"44b8323a91c2406ca11e275876313aa5","version_major":2,"vers
ion_minor":0}
```

{"model_id":"3e861a5b8d4f422085aa0b3252604b32","version_major":2,"version_minor":0}

{"model_id":"b0f5c52499454acabea1b171f06950e0","version_major":2,"version_minor":0}

{"model_id":"d8ae0e1c70f54a168183d4e630f6e77a","version_major":2,"version_minor":0}

Preprocessing Data

```
def create_mask(data):
    image = data["image"]
    mask = Image.new(mode="RGB", size=image.size, color=(0, 0, 0))
    draw = ImageDraw.Draw(mask)

    instance_id = 1
    for category_id, polygon_point in zip(data["objects"]["category"],
data["objects"]["segmentation"]):
        if len(polygon_point) > 1:
            print(polygon_point)
            raise IndexError
        draw.polygon(polygon_point[0], fill=(category_id+1,
instance_id, 0))
        instance_id += 1
    return mask

def unnormalize_image(image, mean=[0.485, 0.456, 0.406], std=[0.229,
0.224, 0.225]):
    image = image * np.array([0.229, 0.224, 0.225])[:, None, None]
    image = image + np.array([0.485, 0.456, 0.406])[:, None, None]
    return image

from torch.utils.data import DataLoader, Dataset
import albumentations as A
import numpy as np
from PIL import Image, ImageDraw
import matplotlib.pyplot as plt
from einops import rearrange

class ImageSegmentationDataset(Dataset):
    def __init__(self, dataset, processor, transform=None):
        self.dataset = dataset
        self.processor = processor
        self.transform = transform

    def __len__(self):
        return len(self.dataset)

    def __getitem__(self, idx):
```

```
        # Convert the PIL Image to a NumPy array
        image = np.array(self.dataset[idx]["image"].convert("RGB"))

        # Get the pixel wise instance id and category id maps
        # of shape (height, width)
        instance_seg = np.array(create_mask(self.dataset[idx]))[...,
1]
        class_id_map = np.array(create_mask(self.dataset[idx]))[...,
0]
        class_labels = np.unique(class_id_map)
        # Build the instance to class dictionary
        inst2class = {}
        for label in class_labels:
            instance_ids = np.unique(instance_seg[class_id_map ==
label])
            inst2class.update({i: label for i in instance_ids})
        # Apply transforms
        if self.transform is not None:
            transformed = self.transform(image=image,
mask=instance_seg)
            (image, instance_seg) = (transformed["image"],
transformed["mask"])

            # Convert from channels last to channels first
            image = image.transpose(2,0,1)
        if class_labels.shape[0] == 1 and class_labels[0] == 0:
            # If the image has no objects then it is skipped
            inputs = self.processor([image], return_tensors="pt")
            inputs = {k:v.squeeze() for k,v in inputs.items()}
            inputs["class_labels"] = torch.tensor([0])
            inputs["mask_labels"] = torch.zeros(
                (0, inputs["pixel_values"].shape[-2],
inputs["pixel_values"].shape[-1])
            )
        else:
            # Else use process the image with the segmentation maps
            inputs = self.processor(
                [image],
                [instance_seg],
                instance_id_to_semantic_id=inst2class,
                return_tensors="pt"
            )
            inputs = {
                k:v.squeeze() if isinstance(v, torch.Tensor) else v[0]
for k,v in inputs.items()
            }
        # Return the inputs
        segmentation_mask = (inputs["mask_labels"] *
inputs["class_labels"][:, None, None]).sum(dim=0).clamp(0, 1)
```

```
        inputs["segmentation"] = segmentation_mask
        return inputs

train_val_transform = A.Compose([
    A.Resize(width=128, height=128),
])

processor = MaskFormerImageProcessor.from_pretrained(model_name)
processor.size = {'shortest_edge':  128, 'longest_edge': 2048}

train_dataset = ImageSegmentationDataset(
    ds["train"],
    processor=processor,
    transform=train_val_transform
)
val_dataset = ImageSegmentationDataset(
    ds["validation"],
    processor=processor,
    transform=train_val_transform
)
test_dataset = ImageSegmentationDataset(
    ds["test"],
    processor=processor,
    transform=train_val_transform
)
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/
file_download.py:1132: FutureWarning: `resume_download` is deprecated
and will be removed in version 1.0.0. Downloads always resume when
possible. If you want to force a new download, use
`force_download=True`.
  warnings.warn(
```

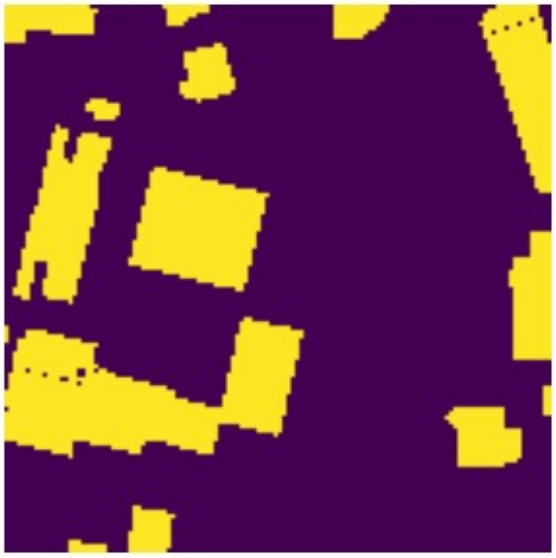

```
{"model_id":"1b32c281a4b242f08dd22e2fe08de7b5","version_major":2,"vers
ion_minor":0}
```


```
/usr/local/lib/python3.12/dist-packages/transformers/models/
maskformer/image_processing_maskformer.py:412: FutureWarning: The
`size_divisibility` argument is deprecated and will be removed in
v4.27. Please use `size_divisor` instead.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/transformers/models/maskformer
/image_processing_maskformer.py:419: FutureWarning: The `max_size`
argument is deprecated and will be removed in v4.27. Please use
size['longest_edge'] instead.
  warnings.warn(
```

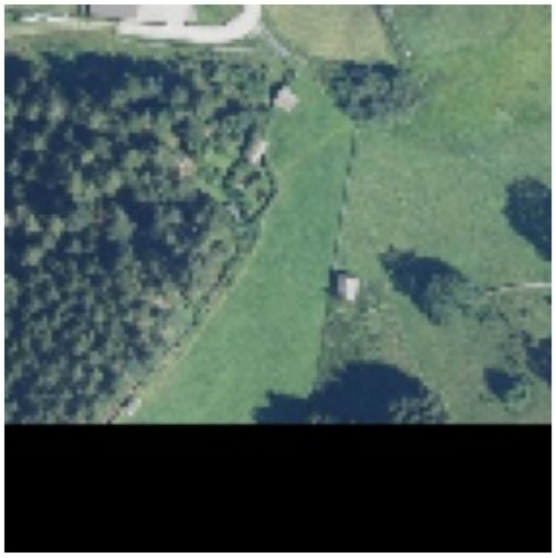
```
for idx, data in enumerate(train_dataset):
    image = unnormalize_image(data["pixel_values"])
    fig, axs = plt.subplots(ncols=2, figsize=(8, 5))
```

```
    axs[0].imshow(np.array(rearrange(image, "c h w -> h w c")))
    axs[1].imshow(np.array(data["segmentation"]))
    axs[0].set_axis_off()
    axs[1].set_axis_off()
    plt.show()

    if idx == 10:
      break
```

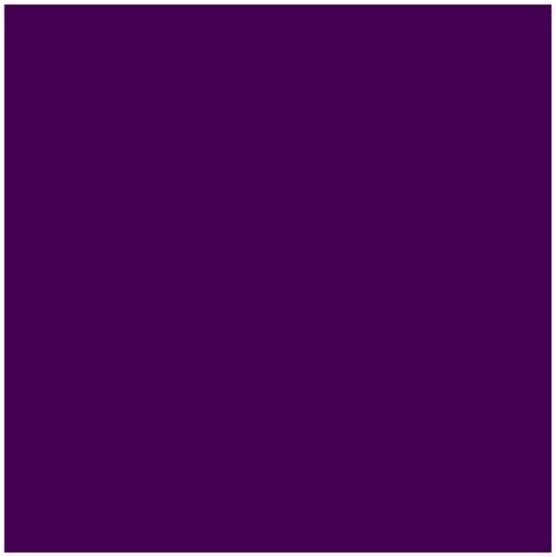
```
/tmp/ipykernel_496/1125473636.py:16: DeprecationWarning:
__array_wrap__ must accept context and return_scalar arguments
(positionally) in the future. (Deprecated NumPy 2.0)
  image = image * np.array([0.229, 0.224, 0.225])[:, None, None]
/tmp/ipykernel_496/1125473636.py:17: DeprecationWarning:
__array_wrap__ must accept context and return_scalar arguments
(positionally) in the future. (Deprecated NumPy 2.0)
  image = image + np.array([0.485, 0.456, 0.406])[:, None, None]
```

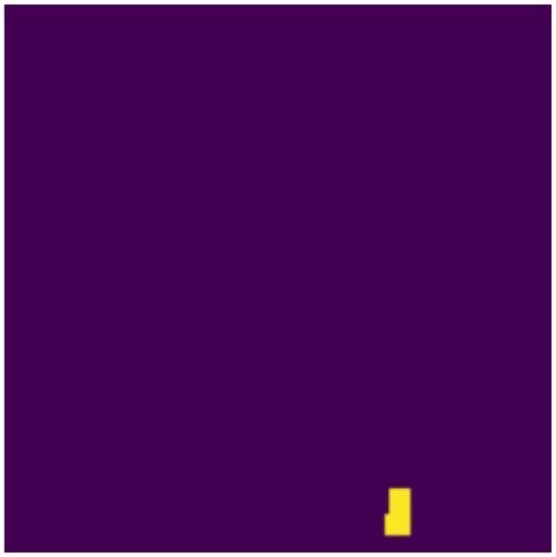


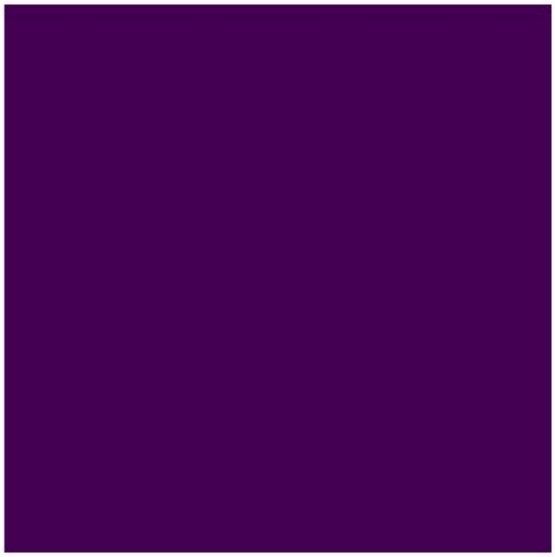
```
WARNING:matplotlib.image:Clipping input data to the valid range for
imshow with RGB data ([0..1] for floats or [0..255] for integers). Got
range [-2.288818357065736e-08..0.9411764984130859].
```

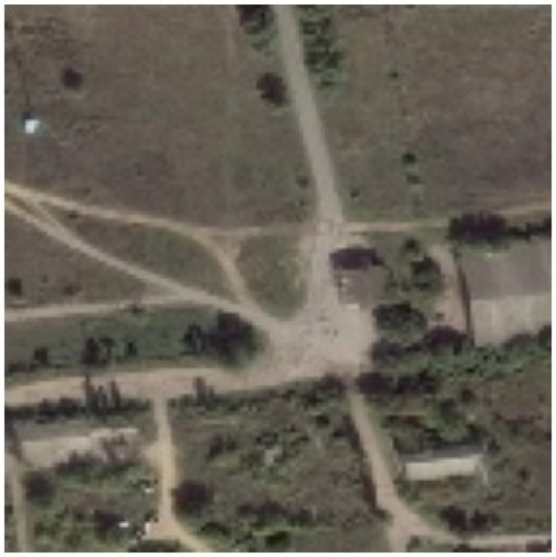


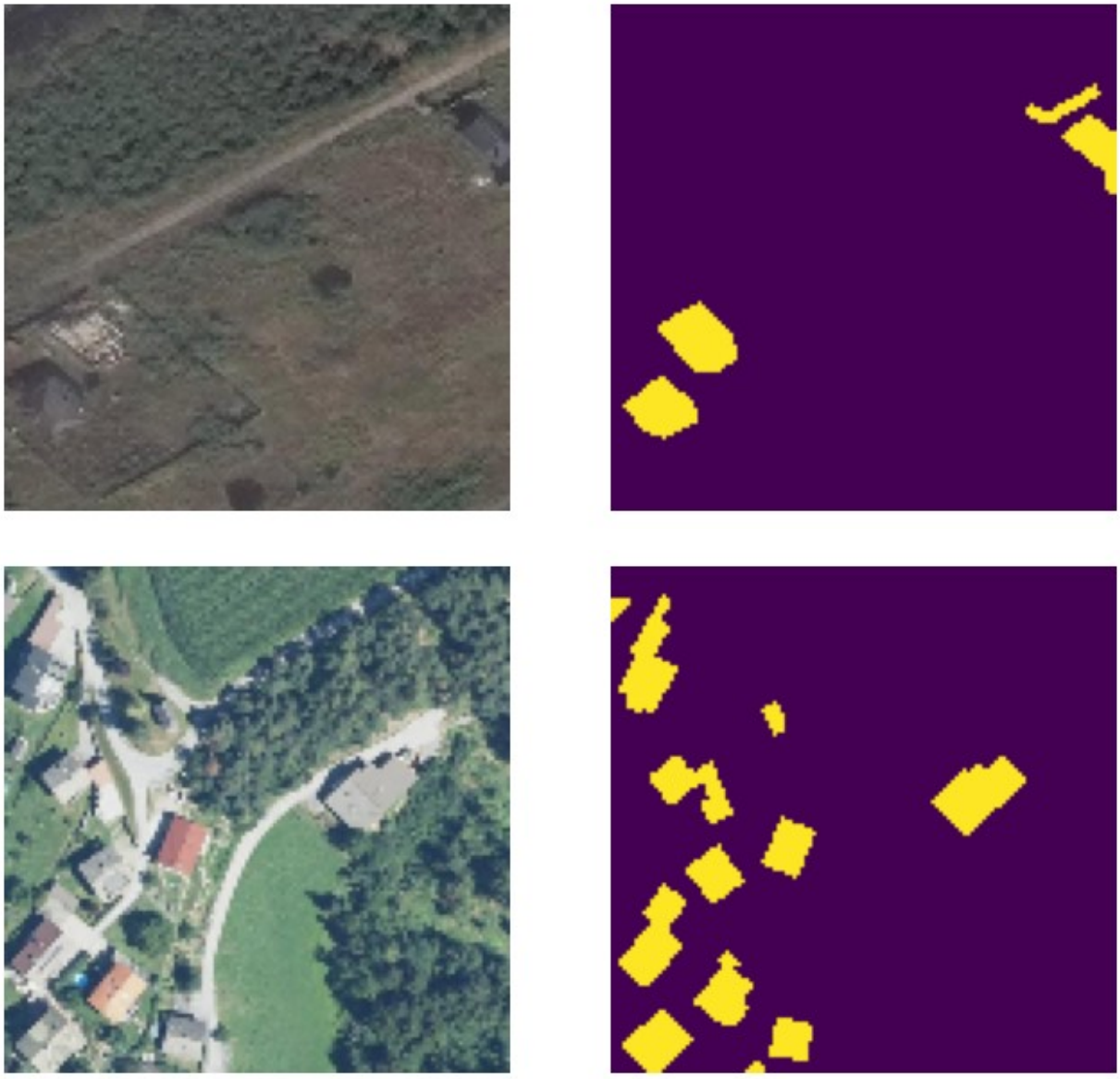
```
WARNING:matplotlib.image:Clipping input data to the valid range for
imshow with RGB data ([0..1] for floats or [0..255] for integers). Got
range [-2.288818357065736e-08..0.8078431663513184].
```











```
def collate_fn(examples):
    # Get the pixel values, pixel mask, mask labels, and class labels
    pixel_values = torch.stack([example["pixel_values"] for example in
examples])
    pixel_mask = torch.stack([example["pixel_mask"] for example in
examples])
    segmentation = torch.stack([example["segmentation"] for example in
examples])
    mask_labels = [example["mask_labels"] for example in examples]
    class_labels = [example["class_labels"] for example in examples]
    # Return a dictionary of all the collated features
    return {
        "pixel_values": pixel_values,
        "pixel_mask": pixel_mask,
```

```
        "segmentation": segmentation,
        "mask_labels": mask_labels,
        "class_labels": class_labels
    }
# Building the training and validation dataloader
train_dataloader = DataLoader(
    train_dataset,
    batch_size=32,
    shuffle=True,
    collate_fn=collate_fn
)
val_dataloader = DataLoader(
    val_dataset,
    batch_size=32,
    shuffle=False,
    collate_fn=collate_fn
)
test_dataloader = DataLoader(
    test_dataset,
    batch_size=32,
    shuffle=False,
    collate_fn=collate_fn
)
```

Model extraction

After preprocessing the dataset, we are going to extract pixel-level module from the MaskFormer model to perform semantic segmentation. Specifically, we will use only module framed inside the red rectangle.

Hint: You can use `model.named_modules()` to observe modules inside the model.

```
maskformer =
MaskFormerForInstanceSegmentation.from_pretrained(model_name)
for name, module in maskformer.named_modules():
  print(name)
```

{"model_id":"245fcd3abd8a4cedb29d8e9108d3e04c","version_major":2,"version_minor":0}

{"model_id":"5ddbe2785a054123bc5c1f6b66aa51dc","version_major":2,"version_minor":0}

```
model
model.pixel_level_module
model.pixel_level_module.encoder
model.pixel_level_module.encoder.model
```

```
model.pixel_level_module.encoder.model.embeddings
model.pixel_level_module.encoder.model.embeddings.patch_embeddings
model.pixel_level_module.encoder.model.embeddings.patch_embeddings.pro
jection
model.pixel_level_module.encoder.model.embeddings.norm
model.pixel_level_module.encoder.model.embeddings.dropout
model.pixel_level_module.encoder.model.encoder
model.pixel_level_module.encoder.model.encoder.layers
model.pixel_level_module.encoder.model.encoder.layers.0
model.pixel_level_module.encoder.model.encoder.layers.0.blocks
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.0
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.0.layer
norm_before
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.0.atten
tion
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.0.atten
tion.self
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.0.atten
tion.self.query
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.0.atten
tion.self.key
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.0.atten
tion.self.value
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.0.atten
tion.self.dropout
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.0.atten
tion.output
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.0.atten
tion.output.dense
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.0.atten
tion.output.dropout
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.0.drop_
path
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.0.layer
norm_after
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.0.inter
mediate
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.0.inter
mediate.dense
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.0.inter
mediate.intermediate_act_fn
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.0.outpu
t
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.0.outpu
t.dense
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.0.outpu
t.dropout
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.1
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.1.layer
```

```
norm_before
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.1.atten
tion
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.1.atten
tion.self
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.1.atten
tion.self.query
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.1.atten
tion.self.key
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.1.atten
tion.self.value
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.1.atten
tion.self.dropout
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.1.atten
tion.output
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.1.atten
tion.output.dense
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.1.atten
tion.output.dropout
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.1.drop_
path
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.1.layer
norm_after
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.1.inter
mediate
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.1.inter
mediate.dense
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.1.inter
mediate.intermediate_act_fn
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.1.outpu
t
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.1.outpu
t.dense
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.1.outpu
t.dropout
model.pixel_level_module.encoder.model.encoder.layers.0.downsample
model.pixel_level_module.encoder.model.encoder.layers.0.downsample.red
uction
model.pixel_level_module.encoder.model.encoder.layers.0.downsample.nor
m
model.pixel_level_module.encoder.model.encoder.layers.1
model.pixel_level_module.encoder.model.encoder.layers.1.blocks
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.0
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.0.layer
norm_before
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.0.atten
tion
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.0.atten
tion.self
```

```
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.0.attention.self.query
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.0.attention.self.key
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.0.attention.self.value
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.0.attention.self.dropout
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.0.attention.output
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.0.attention.output.dense
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.0.attention.output.dropout
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.0.drop_path
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.0.layernorm_after
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.0.intermediate
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.0.intermediate.dense
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.0.intermediate.intermediate_act_fn
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.0.output
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.0.output.dense
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.0.output.dropout
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.1
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.1.layernorm_before
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.1.attention
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.1.attention.self
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.1.attention.self.query
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.1.attention.self.key
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.1.attention.self.value
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.1.attention.self.dropout
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.1.attention.output
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.1.attention.output.dense
```

```
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.1.atten
tion.output.dropout
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.1.drop_
path
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.1.layer
norm_after
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.1.inter
mediate
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.1.inter
mediate.dense
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.1.inter
mediate.intermediate_act_fn
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.1.outpu
t
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.1.outpu
t.dense
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.1.outpu
t.dropout
model.pixel_level_module.encoder.model.encoder.layers.1.downsample
model.pixel_level_module.encoder.model.encoder.layers.1.downsample.red
uction
model.pixel_level_module.encoder.model.encoder.layers.1.downsample.nor
m
model.pixel_level_module.encoder.model.encoder.layers.2
model.pixel_level_module.encoder.model.encoder.layers.2.blocks
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.0
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.0.layer
norm_before
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.0.atten
tion
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.0.atten
tion.self
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.0.atten
tion.self.query
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.0.atten
tion.self.key
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.0.atten
tion.self.value
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.0.atten
tion.self.dropout
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.0.atten
tion.output
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.0.atten
tion.output.dense
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.0.atten
tion.output.dropout
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.0.drop_
path
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.0.layer
```

```
norm_after
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.0.inter
mediate
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.0.inter
mediate.dense
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.0.inter
mediate.intermediate_act_fn
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.0.outpu
t
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.0.outpu
t.dense
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.0.outpu
t.dropout
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.1
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.1.layer
norm_before
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.1.atten
tion
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.1.atten
tion.self
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.1.atten
tion.self.query
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.1.atten
tion.self.key
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.1.atten
tion.self.value
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.1.atten
tion.self.dropout
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.1.atten
tion.output
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.1.atten
tion.output.dense
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.1.atten
tion.output.dropout
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.1.drop_
path
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.1.layer
norm_after
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.1.inter
mediate
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.1.inter
mediate.dense
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.1.inter
mediate.intermediate_act_fn
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.1.outpu
t
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.1.outpu
t.dense
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.1.outpu
```

```
t.dropout
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.2
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.2.layer
norm_before
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.2.atten
tion
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.2.atten
tion.self
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.2.atten
tion.self.query
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.2.atten
tion.self.key
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.2.atten
tion.self.value
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.2.atten
tion.self.dropout
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.2.atten
tion.output
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.2.atten
tion.output.dense
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.2.atten
tion.output.dropout
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.2.drop_
path
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.2.layer
norm_after
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.2.inter
mediate
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.2.inter
mediate.dense
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.2.inter
mediate.intermediate_act_fn
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.2.outpu
t
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.2.outpu
t.dense
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.2.outpu
t.dropout
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.3
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.3.layer
norm_before
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.3.atten
tion
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.3.atten
tion.self
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.3.atten
tion.self.query
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.3.atten
tion.self.key
```

```
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.3.attention.self.value
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.3.attention.self.dropout
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.3.attention.output
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.3.attention.output.dense
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.3.attention.output.dropout
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.3.drop_path
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.3.layernorm_after
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.3.intermediate
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.3.intermediate.dense
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.3.intermediate.intermediate_act_fn
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.3.output
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.3.output.dense
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.3.output.dropout
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.4
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.4.layernorm_before
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.4.attention
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.4.attention.self
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.4.attention.self.query
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.4.attention.self.key
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.4.attention.self.value
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.4.attention.self.dropout
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.4.attention.output
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.4.attention.output.dense
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.4.attention.output.dropout
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.4.drop_path
```

```
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.4.layer
norm_after
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.4.inter
mediate
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.4.inter
mediate.dense
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.4.inter
mediate.intermediate_act_fn
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.4.outpu
t
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.4.outpu
t.dense
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.4.outpu
t.dropout
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.5
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.5.layer
norm_before
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.5.atten
tion
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.5.atten
tion.self
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.5.atten
tion.self.query
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.5.atten
tion.self.key
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.5.atten
tion.self.value
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.5.atten
tion.self.dropout
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.5.atten
tion.output
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.5.atten
tion.output.dense
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.5.atten
tion.output.dropout
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.5.drop_
path
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.5.layer
norm_after
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.5.inter
mediate
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.5.inter
mediate.dense
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.5.inter
mediate.intermediate_act_fn
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.5.outpu
t
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.5.outpu
t.dense
```

```
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.5.output.dropout
model.pixel_level_module.encoder.model.encoder.layers.2.downsample
model.pixel_level_module.encoder.model.encoder.layers.2.downsample.reduction
model.pixel_level_module.encoder.model.encoder.layers.2.downsample.norm
model.pixel_level_module.encoder.model.encoder.layers.3
model.pixel_level_module.encoder.model.encoder.layers.3.blocks
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.0
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.0.layernorm_before
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.0.attention
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.0.attention.self
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.0.attention.self.query
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.0.attention.self.key
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.0.attention.self.value
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.0.attention.self.dropout
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.0.attention.output
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.0.attention.output.dense
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.0.attention.output.dropout
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.0.drop_path
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.0.layernorm_after
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.0.intermediate
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.0.intermediate.dense
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.0.intermediate.intermediate_act_fn
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.0.output
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.0.output.dense
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.0.output.dropout
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.1
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.1.layernorm_before
```

```
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.1.atten
tion
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.1.atten
tion.self
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.1.atten
tion.self.query
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.1.atten
tion.self.key
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.1.atten
tion.self.value
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.1.atten
tion.self.dropout
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.1.atten
tion.output
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.1.atten
tion.output.dense
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.1.atten
tion.output.dropout
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.1.drop_
path
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.1.layer
norm_after
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.1.inter
mediate
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.1.inter
mediate.dense
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.1.inter
mediate.intermediate_act_fn
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.1.outpu
t
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.1.outpu
t.dense
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.1.outpu
t.dropout
model.pixel_level_module.encoder.model.layernorm
model.pixel_level_module.encoder.model.pooler
model.pixel_level_module.encoder.hidden_states_norms
model.pixel_level_module.encoder.hidden_states_norms.0
model.pixel_level_module.encoder.hidden_states_norms.1
model.pixel_level_module.encoder.hidden_states_norms.2
model.pixel_level_module.encoder.hidden_states_norms.3
model.pixel_level_module.decoder
model.pixel_level_module.decoder.fpn
model.pixel_level_module.decoder.fpn.stem
model.pixel_level_module.decoder.fpn.stem.0
model.pixel_level_module.decoder.fpn.stem.1
model.pixel_level_module.decoder.fpn.stem.2
model.pixel_level_module.decoder.fpn.layers
model.pixel_level_module.decoder.fpn.layers.0
model.pixel_level_module.decoder.fpn.layers.0.proj
```

```
model.pixel_level_module.decoder.fpn.layers.0.proj.0
model.pixel_level_module.decoder.fpn.layers.0.proj.1
model.pixel_level_module.decoder.fpn.layers.0.block
model.pixel_level_module.decoder.fpn.layers.0.block.0
model.pixel_level_module.decoder.fpn.layers.0.block.1
model.pixel_level_module.decoder.fpn.layers.0.block.2
model.pixel_level_module.decoder.fpn.layers.1
model.pixel_level_module.decoder.fpn.layers.1.proj
model.pixel_level_module.decoder.fpn.layers.1.proj.0
model.pixel_level_module.decoder.fpn.layers.1.proj.1
model.pixel_level_module.decoder.fpn.layers.1.block
model.pixel_level_module.decoder.fpn.layers.1.block.0
model.pixel_level_module.decoder.fpn.layers.1.block.1
model.pixel_level_module.decoder.fpn.layers.1.block.2
model.pixel_level_module.decoder.fpn.layers.2
model.pixel_level_module.decoder.fpn.layers.2.proj
model.pixel_level_module.decoder.fpn.layers.2.proj.0
model.pixel_level_module.decoder.fpn.layers.2.proj.1
model.pixel_level_module.decoder.fpn.layers.2.block
model.pixel_level_module.decoder.fpn.layers.2.block.0
model.pixel_level_module.decoder.fpn.layers.2.block.1
model.pixel_level_module.decoder.fpn.layers.2.block.2
model.pixel_level_module.decoder.mask_projection
model.transformer_module
model.transformer_module.position_embedder
model.transformer_module.queries_embedder
model.transformer_module.input_projection
model.transformer_module.decoder
model.transformer_module.decoder.layers
model.transformer_module.decoder.layers.0
model.transformer_module.decoder.layers.0.self_attn
model.transformer_module.decoder.layers.0.self_attn.k_proj
model.transformer_module.decoder.layers.0.self_attn.v_proj
model.transformer_module.decoder.layers.0.self_attn.q_proj
model.transformer_module.decoder.layers.0.self_attn.out_proj
model.transformer_module.decoder.layers.0.activation_fn
model.transformer_module.decoder.layers.0.self_attn_layer_norm
model.transformer_module.decoder.layers.0.encoder_attn
model.transformer_module.decoder.layers.0.encoder_attn.k_proj
model.transformer_module.decoder.layers.0.encoder_attn.v_proj
model.transformer_module.decoder.layers.0.encoder_attn.q_proj
model.transformer_module.decoder.layers.0.encoder_attn.out_proj
model.transformer_module.decoder.layers.0.encoder_attn_layer_norm
model.transformer_module.decoder.layers.0.fc1
model.transformer_module.decoder.layers.0.fc2
model.transformer_module.decoder.layers.0.final_layer_norm
model.transformer_module.decoder.layers.1
model.transformer_module.decoder.layers.1.self_attn
model.transformer_module.decoder.layers.1.self_attn.k_proj
```

```
model.transformer_module.decoder.layers.1.self_attn.v_proj
model.transformer_module.decoder.layers.1.self_attn.q_proj
model.transformer_module.decoder.layers.1.self_attn.out_proj
model.transformer_module.decoder.layers.1.activation_fn
model.transformer_module.decoder.layers.1.self_attn_layer_norm
model.transformer_module.decoder.layers.1.encoder_attn
model.transformer_module.decoder.layers.1.encoder_attn.k_proj
model.transformer_module.decoder.layers.1.encoder_attn.v_proj
model.transformer_module.decoder.layers.1.encoder_attn.q_proj
model.transformer_module.decoder.layers.1.encoder_attn.out_proj
model.transformer_module.decoder.layers.1.encoder_attn_layer_norm
model.transformer_module.decoder.layers.1.fc1
model.transformer_module.decoder.layers.1.fc2
model.transformer_module.decoder.layers.1.final_layer_norm
model.transformer_module.decoder.layers.2
model.transformer_module.decoder.layers.2.self_attn
model.transformer_module.decoder.layers.2.self_attn.k_proj
model.transformer_module.decoder.layers.2.self_attn.v_proj
model.transformer_module.decoder.layers.2.self_attn.q_proj
model.transformer_module.decoder.layers.2.self_attn.out_proj
model.transformer_module.decoder.layers.2.activation_fn
model.transformer_module.decoder.layers.2.self_attn_layer_norm
model.transformer_module.decoder.layers.2.encoder_attn
model.transformer_module.decoder.layers.2.encoder_attn.k_proj
model.transformer_module.decoder.layers.2.encoder_attn.v_proj
model.transformer_module.decoder.layers.2.encoder_attn.q_proj
model.transformer_module.decoder.layers.2.encoder_attn.out_proj
model.transformer_module.decoder.layers.2.encoder_attn_layer_norm
model.transformer_module.decoder.layers.2.fc1
model.transformer_module.decoder.layers.2.fc2
model.transformer_module.decoder.layers.2.final_layer_norm
model.transformer_module.decoder.layers.3
model.transformer_module.decoder.layers.3.self_attn
model.transformer_module.decoder.layers.3.self_attn.k_proj
model.transformer_module.decoder.layers.3.self_attn.v_proj
model.transformer_module.decoder.layers.3.self_attn.q_proj
model.transformer_module.decoder.layers.3.self_attn.out_proj
model.transformer_module.decoder.layers.3.activation_fn
model.transformer_module.decoder.layers.3.self_attn_layer_norm
model.transformer_module.decoder.layers.3.encoder_attn
model.transformer_module.decoder.layers.3.encoder_attn.k_proj
model.transformer_module.decoder.layers.3.encoder_attn.v_proj
model.transformer_module.decoder.layers.3.encoder_attn.q_proj
model.transformer_module.decoder.layers.3.encoder_attn.out_proj
model.transformer_module.decoder.layers.3.encoder_attn_layer_norm
model.transformer_module.decoder.layers.3.fc1
model.transformer_module.decoder.layers.3.fc2
model.transformer_module.decoder.layers.3.final_layer_norm
model.transformer_module.decoder.layers.4
```

```
model.transformer_module.decoder.layers.4.self_attn
model.transformer_module.decoder.layers.4.self_attn.k_proj
model.transformer_module.decoder.layers.4.self_attn.v_proj
model.transformer_module.decoder.layers.4.self_attn.q_proj
model.transformer_module.decoder.layers.4.self_attn.out_proj
model.transformer_module.decoder.layers.4.activation_fn
model.transformer_module.decoder.layers.4.self_attn_layer_norm
model.transformer_module.decoder.layers.4.encoder_attn
model.transformer_module.decoder.layers.4.encoder_attn.k_proj
model.transformer_module.decoder.layers.4.encoder_attn.v_proj
model.transformer_module.decoder.layers.4.encoder_attn.q_proj
model.transformer_module.decoder.layers.4.encoder_attn.out_proj
model.transformer_module.decoder.layers.4.encoder_attn_layer_norm
model.transformer_module.decoder.layers.4.fc1
model.transformer_module.decoder.layers.4.fc2
model.transformer_module.decoder.layers.4.final_layer_norm
model.transformer_module.decoder.layers.5
model.transformer_module.decoder.layers.5.self_attn
model.transformer_module.decoder.layers.5.self_attn.k_proj
model.transformer_module.decoder.layers.5.self_attn.v_proj
model.transformer_module.decoder.layers.5.self_attn.q_proj
model.transformer_module.decoder.layers.5.self_attn.out_proj
model.transformer_module.decoder.layers.5.activation_fn
model.transformer_module.decoder.layers.5.self_attn_layer_norm
model.transformer_module.decoder.layers.5.encoder_attn
model.transformer_module.decoder.layers.5.encoder_attn.k_proj
model.transformer_module.decoder.layers.5.encoder_attn.v_proj
model.transformer_module.decoder.layers.5.encoder_attn.q_proj
model.transformer_module.decoder.layers.5.encoder_attn.out_proj
model.transformer_module.decoder.layers.5.encoder_attn_layer_norm
model.transformer_module.decoder.layers.5.fc1
model.transformer_module.decoder.layers.5.fc2
model.transformer_module.decoder.layers.5.final_layer_norm
model.transformer_module.decoder.layernorm
class_predictor
mask_embedder
mask_embedder.0
mask_embedder.0.0
mask_embedder.0.1
mask_embedder.1
mask_embedder.1.0
mask_embedder.1.1
mask_embedder.2
mask_embedder.2.0
mask_embedder.2.1
matcher
criterion
```

TODO 13: extract the pixel-level module from maskformer and also replace the last layer (mask prediction) with a new convolutional layer (outchannel=1, kernel_size=3, stride=1, padding=1).

```
print(maskformer.model.pixel_level_module.decoder.mask_projection)
```

```
Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
```

```
class SegmentModel(nn.Module):
    def __init__(self, model_name):
        super().__init__()
        maskformer =
MaskFormerForInstanceSegmentation.from_pretrained(model_name)
        # TODO 13.1: extract pixel-level module and create new mask
projection
        self.pixel_level_module = maskformer.model.pixel_level_module
        self.pixel_level_module.decoder.mask_projection =
nn.Conv2d(256, 1, kernel_size=3, stride=1, padding=1)

    def forward(self, pixel_values):
        # TODO 13.2: extract pixel-level module and create new mask
projection
        # Note: Do not forget to interpolate the output to have the
same size as pixel_values (the input)
        #       we will use the bilinear interpolation with
align_corners set to False.
        out = self.pixel_level_module(pixel_values)
        y = out.decoder_last_hidden_state
        y = F.interpolate(y, pixel_values.shape[-2:], mode='bilinear',
align_corners=False)
        return y

test_input = torch.randn(1, 3, 32, 32)
out = maskformer.model.pixel_level_module(test_input)
print(type(out))
print(out)
```

```
<class
'transformers.models.maskformer.modeling_maskformer.MaskFormerPixelLev
elModuleOutput'>
MaskFormerPixelLevelModuleOutput(encoder_last_hidden_state=tensor([[[[
1.0396e+00]],

         [[ 7.4258e-01]],

         [[ 8.4024e-01]],

         [[ 7.5014e-01]],

         [[-6.9645e-01]],

         [[-4.6425e-02]],
```

```
         [[ 1.6109e-01]],

         [[ 8.1377e-01]],

         [[-6.1542e-01]],

         [[-7.9711e-01]],

         [[-7.6691e-01]],

         [[ 4.7251e-01]],

         [[-7.6071e-02]],

         [[ 9.0229e-01]],

         [[ 3.7143e-01]],

         [[-1.1326e-01]],

         [[ 2.3293e-01]],

         [[ 8.8900e-01]],

         [[ 4.1098e-02]],

         [[-2.7591e+00]],

         [[ 2.4418e-01]],

         [[ 2.4716e-01]],

         [[ 8.7296e-01]],

         [[ 5.3151e-01]],

         [[-1.0918e+00]],

         [[ 3.8225e-01]],

         [[-1.2183e+00]],

         [[-1.0209e+00]],

         [[-4.5228e-01]],

         [[-5.4901e-02]],

         [[ 3.1012e-01]],

         [[-4.0897e-01]],
```

```
          [[-1.3628e+00]],

          [[-1.2743e+00]],

          [[ 3.5841e-01]],

          [[-5.6432e-01]],

          [[ 3.7743e+00]],

          [[-1.2163e-01]],

          [[ 9.8354e-01]],

          [[ 5.6042e-01]],

          [[-7.8193e-01]],

          [[ 3.7849e-01]],

          [[-7.9590e-01]],

          [[-4.7021e-01]],

          [[ 2.4300e-01]],

          [[-4.3606e-02]],

          [[-4.0719e-01]],

          [[-1.0805e+00]],

          [[ 1.3883e+00]],

          [[-5.1549e-01]],

          [[ 2.4979e-01]],

          [[-4.7131e-02]],

          [[-4.1166e-01]],

          [[ 1.7654e-01]],

          [[ 5.7758e-01]],

          [[ 5.1876e-01]],

          [[ 6.6916e-01]],

          [[-9.6194e-01]],
```

```
          [[-1.0725e-01]],

          [[-9.9743e-01]],

          [[-3.1921e-01]],

          [[-3.2727e-01]],

          [[-1.5483e+00]],

          [[ 5.5654e-01]],

          [[ 7.6118e-01]],

          [[ 3.1493e-01]],

          [[-1.2019e+00]],

          [[-5.2693e-01]],

          [[ 1.5928e-01]],

          [[ 5.5734e-01]],

          [[ 1.6465e-01]],

          [[ 4.9032e-01]],

          [[ 1.3221e-01]],

          [[-2.0341e-01]],

          [[ 6.9480e-01]],

          [[ 3.0477e-01]],

          [[ 8.0486e-01]],

          [[ 5.1254e-01]],

          [[ 7.0972e-01]],

          [[ 5.3944e-03]],

          [[ 6.9436e-01]],

          [[-9.0408e-01]],

          [[ 8.0547e-01]],

          [[ 1.0039e-02]],
```

```
        [[ 3.8621e-01]],

        [[-8.3105e-01]],

        [[-6.2477e-01]],

        [[-1.2966e+00]],

        [[ 7.5393e-01]],

        [[-7.9346e-01]],

        [[-4.7477e-01]],

        [[ 1.1780e+00]],

        [[-1.2128e+00]],

        [[-1.2789e+00]],

        [[-6.4639e-02]],

        [[-1.6203e-01]],

        [[ 3.8454e-01]],

        [[ 2.5147e-01]],

        [[ 8.3414e-01]],

        [[-5.1476e-01]],

        [[ 4.3695e-02]],

        [[ 5.5494e-01]],

        [[ 9.7566e-01]],

        [[ 1.1357e+00]],

        [[ 1.5864e+00]],

        [[ 1.6904e-01]],

        [[-3.6866e-01]],

        [[ 2.8524e+00]],

        [[ 1.2808e-01]],

        [[ 1.1218e-01]],
```

```
          [[-3.5880e-02]],

          [[-1.0040e+00]],

          [[-8.7540e-02]],

          [[ 8.1690e-01]],

          [[ 1.7657e-01]],

          [[ 9.0669e-01]],

          [[-2.3855e-01]],

          [[-5.6915e-03]],

          [[ 5.8094e-01]],

          [[-3.7519e-01]],

          [[ 4.5805e-01]],

          [[-4.1243e-01]],

          [[-3.2896e-01]],

          [[-5.3398e-01]],

          [[-4.4666e-01]],

          [[-7.3307e-01]],

          [[ 4.6707e-01]],

          [[ 7.6375e-01]],

          [[-1.0439e+00]],

          [[-4.8341e-02]],

          [[ 1.8865e+00]],

          [[ 5.1393e-01]],

          [[-1.5169e+00]],

          [[ 1.9425e+00]],

          [[-2.2589e+00]],

          [[ 1.3388e+00]],
```

```
        [[-3.9752e-01]],
        [[ 7.0607e-01]],
        [[ 6.1381e-02]],
        [[-3.9497e-01]],
        [[-3.0330e-01]],
        [[ 1.1452e+00]],
        [[ 3.5802e-01]],
        [[-1.1399e-01]],
        [[-9.1517e-01]],
        [[-8.2931e-01]],
        [[ 1.1796e+00]],
        [[-3.2788e-01]],
        [[ 1.0075e+00]],
        [[ 3.0145e-02]],
        [[ 1.0091e+00]],
        [[ 3.2776e-01]],
        [[-2.7891e-01]],
        [[ 3.2815e-01]],
        [[-1.8963e+00]],
        [[ 3.1173e-01]],
        [[ 9.3516e-01]],
        [[-4.0856e-02]],
        [[-4.4486e-01]],
        [[-3.4227e-01]],
        [[ 9.7116e-01]],
        [[-2.7818e-02]],
```

```
          [[-3.0370e-01]],

          [[-7.8604e-02]],

          [[ 1.5724e+00]],

          [[-1.0298e-01]],

          [[-8.8117e-01]],

          [[-6.7933e-01]],

          [[-1.4946e+00]],

          [[-5.5364e-01]],

          [[-8.0874e+00]],

          [[ 3.8703e-01]],

          [[ 5.5891e-02]],

          [[-3.9710e-01]],

          [[-3.9932e-01]],

          [[-9.6503e-01]],

          [[-3.3713e-01]],

          [[ 3.1110e+00]],

          [[ 8.4494e-01]],

          [[ 9.8997e-01]],

          [[-3.5843e-01]],

          [[ 2.2381e+00]],

          [[ 3.0848e-02]],

          [[ 2.1113e-01]],

          [[-5.8806e-01]],

          [[ 5.7677e-01]],

          [[ 9.1879e-01]],

          [[-5.0391e-01]],
```

```
          [[-7.0415e-01]],

          [[ 1.9547e-01]],

          [[ 2.1832e-01]],

          [[ 4.2774e-01]],

          [[-9.4024e-02]],

          [[-2.9303e-02]],

          [[-7.1966e-01]],

          [[-8.0691e-01]],

          [[ 8.5308e-01]],

          [[ 1.8956e-01]],

          [[ 1.0898e+00]],

          [[-2.6084e-02]],

          [[-9.2474e-01]],

          [[ 1.5732e+00]],

          [[-1.9767e-01]],

          [[ 1.4483e-02]],

          [[-1.4985e-01]],

          [[ 9.6276e-01]],

          [[ 2.7496e+00]],

          [[-2.7134e-01]],

          [[ 4.6100e-02]],

          [[-2.0054e-01]],

          [[-8.2137e-01]],

          [[-1.2997e+00]],

          [[-3.7340e-01]],

          [[ 4.9063e-01]],
```

```
        [[-7.4746e-02]],
        [[-1.8574e+00]],
        [[-7.3434e-01]],
        [[ 3.2502e-01]],
        [[-5.2749e-01]],
        [[ 9.8194e-02]],
        [[-6.2028e-01]],
        [[ 4.5865e-01]],
        [[-6.9589e-02]],
        [[-3.3918e-01]],
        [[-2.7860e-02]],
        [[-4.9889e-01]],
        [[ 3.7121e-01]],
        [[ 5.7734e-01]],
        [[ 6.1912e-01]],
        [[-1.7254e+00]],
        [[-4.6467e-01]],
        [[ 8.4425e-01]],
        [[ 3.3015e-02]],
        [[-5.3579e-01]],
        [[-1.7413e-01]],
        [[-6.2475e-02]],
        [[ 6.2128e-01]],
        [[-1.2843e+00]],
        [[-1.1034e+00]],
        [[ 1.8940e-01]],
```

```
        [[ 8.9727e-02]],

        [[-8.6977e-01]],

        [[ 5.6062e-01]],

        [[ 8.0008e-01]],

        [[-1.0534e+00]],

        [[ 1.0950e-01]],

        [[-5.4206e-01]],

        [[-1.0028e-02]],

        [[ 1.1982e+00]],

        [[ 1.8676e-01]],

        [[ 6.4050e-01]],

        [[-9.6233e-02]],

        [[ 8.7530e-01]],

        [[-9.2970e-01]],

        [[-1.0328e-01]],

        [[-8.8533e-01]],

        [[-4.3594e-01]],

        [[ 8.4984e-01]],

        [[-2.9158e-01]],

        [[-7.1555e-01]],

        [[ 5.1504e-02]],

        [[ 1.1613e+00]],

        [[ 4.8343e-01]],

        [[-1.0921e+00]],

        [[-5.3911e-01]],

        [[-6.4160e-01]],
```

```
          [[-3.1625e-01]],

          [[-1.2644e+00]],

          [[ 6.4311e-02]],

          [[ 8.8005e-01]],

          [[-1.7266e-01]],

          [[ 2.1796e-01]],

          [[-7.8893e-01]],

          [[ 1.3426e+00]],

          [[-6.3419e-02]],

          [[ 1.0113e+00]],

          [[-9.6930e-01]],

          [[ 1.0877e+00]],

          [[-2.6019e-02]],

          [[-1.4405e-01]],

          [[ 7.0755e-02]],

          [[-7.3383e-03]],

          [[ 4.6919e-01]],

          [[-3.7819e-01]],

          [[ 2.7138e-01]],

          [[-1.4233e+00]],

          [[ 2.1974e-01]],

          [[-8.4231e-01]],

          [[-8.1162e-01]],

          [[ 1.0535e+00]],

          [[ 4.3651e-01]],

          [[ 3.2869e-01]],
```

```
          [[ 5.9386e-01]],

          [[ 3.5838e-01]],

          [[-6.6350e-01]],

          [[ 2.8991e-01]],

          [[ 4.6552e-01]],

          [[ 2.8700e-02]],

          [[ 7.8190e-01]],

          [[-3.8049e-01]],

          [[-7.2165e-01]],

          [[-3.2200e-01]],

          [[-2.0869e-01]],

          [[ 5.4911e-01]],

          [[-1.2012e+00]],

          [[ 1.0074e+00]],

          [[-1.6976e-01]],

          [[-2.2324e-01]],

          [[ 3.3843e-01]],

          [[-1.0585e+00]],

          [[-6.1161e-01]],

          [[-9.2049e-02]],

          [[-4.2429e-01]],

          [[ 1.1839e-01]],

          [[ 9.7702e-01]],

          [[-1.0477e+00]],

          [[-4.6017e-01]],

          [[ 1.1264e-01]],
```

```
          [[-5.8734e-01]],

          [[ 1.4294e-01]],

          [[ 3.3828e-01]],

          [[-3.0856e-01]],

          [[ 1.8618e-01]],

          [[ 2.5568e-01]],

          [[-1.8956e+00]],

          [[-1.9578e-01]],

          [[-2.4284e+00]],

          [[ 5.7550e-02]],

          [[-1.7028e-02]],

          [[-2.1382e-02]],

          [[ 1.2345e-01]],

          [[-6.2679e-01]],

          [[ 2.4064e-01]],

          [[ 6.7803e-01]],

          [[ 7.6558e-01]],

          [[ 8.4204e-01]],

          [[ 1.2522e+00]],

          [[ 2.8236e-01]],

          [[-3.8570e-01]],

          [[-4.2278e-02]],

          [[ 3.0541e-01]],

          [[-1.0115e+00]],

          [[-8.9078e-02]],

          [[ 4.9427e-01]],
```

```
          [[-1.3501e-01]],

          [[-8.6612e-01]],

          [[ 5.8747e-01]],

          [[-4.1300e-01]],

          [[ 7.0815e-01]],

          [[ 8.1941e-03]],

          [[-1.0643e-02]],

          [[ 7.4642e-01]],

          [[ 6.0004e-01]],

          [[ 8.5904e-01]],

          [[-2.2464e-01]],

          [[ 3.3859e-01]],

          [[ 4.7514e-01]],

          [[-3.3946e-01]],

          [[-1.3371e-01]],

          [[ 8.7999e-02]],

          [[ 6.1677e-01]],

          [[-4.9791e-01]],

          [[-3.8986e-01]],

          [[ 2.3107e-01]],

          [[-2.4966e-01]],

          [[ 5.3316e-01]],

          [[-1.9200e-01]],

          [[ 3.2651e-01]],

          [[-1.9328e-01]],

          [[ 7.5310e-01]],
```

```
        [[-1.1139e+00]],
        [[ 1.0925e+00]],
        [[ 1.0282e+00]],
        [[ 1.1511e+00]],
        [[-1.2433e+00]],
        [[ 1.8345e-01]],
        [[-5.6173e-01]],
        [[ 1.3667e+00]],
        [[ 6.8468e-01]],
        [[ 1.4577e+00]],
        [[ 5.6785e-01]],
        [[ 9.3249e-01]],
        [[ 3.4050e-01]],
        [[ 7.9025e-02]],
        [[-2.1326e+00]],
        [[-1.5765e-01]],
        [[-2.6123e-01]],
        [[-7.6993e-01]],
        [[-3.0334e-01]],
        [[-1.7266e+00]],
        [[ 1.3654e+00]],
        [[-7.0869e-01]],
        [[ 1.2001e+00]],
        [[ 4.5917e-01]],
        [[ 1.9861e-01]],
        [[ 2.1181e+00]],
```

```
        [[ 7.7758e-01]],
        [[ 4.2905e-01]],
        [[ 1.9526e-01]],
        [[-4.9690e-01]],
        [[ 1.1620e-01]],
        [[ 1.0096e+00]],
        [[-1.7572e+00]],
        [[-5.8704e-01]],
        [[-9.6618e-01]],
        [[ 1.0229e+00]],
        [[-1.1377e+00]],
        [[-1.0701e+00]],
        [[ 9.9784e-01]],
        [[-6.1385e-01]],
        [[ 5.6194e-01]],
        [[ 1.4020e+00]],
        [[-6.6220e-01]],
        [[-1.7797e+00]],
        [[ 3.7839e-01]],
        [[-8.3450e-01]],
        [[ 6.2484e-01]],
        [[-1.1461e+00]],
        [[ 7.6608e-01]],
        [[ 4.1342e-01]],
        [[ 5.2863e-01]],
        [[ 6.5713e-02]],
```

```
          [[-2.1713e-02]],

          [[ 1.8849e+00]],

          [[-1.0323e+00]],

          [[ 3.3744e-01]],

          [[-7.0536e-02]],

          [[-1.2008e+00]],

          [[ 1.3849e+00]],

          [[-4.4006e-01]],

          [[-1.2346e-01]],

          [[ 5.5831e-01]],

          [[-8.8061e-01]],

          [[ 9.2786e-01]],

          [[-1.8738e-02]],

          [[ 1.6363e+00]],

          [[ 4.2760e-02]],

          [[-5.8179e-02]],

          [[ 1.0106e-01]],

          [[ 4.4865e-01]],

          [[ 7.9670e-02]],

          [[ 1.0171e+00]],

          [[-3.4079e-01]],

          [[-2.5997e-01]],

          [[-3.0447e+00]],

          [[ 7.9029e-02]],

          [[ 5.1844e-01]],

          [[ 1.6214e-01]],
```

```
          [[-4.5803e-01]],

          [[-4.0750e-01]],

          [[ 1.2698e-02]],

          [[-1.1020e+00]],

          [[ 7.8430e-02]],

          [[ 9.0471e-02]],

          [[-1.1973e-01]],

          [[ 1.7629e+00]],

          [[-9.3809e-01]],

          [[ 2.6573e-01]],

          [[-2.1509e-01]],

          [[ 5.8271e-01]],

          [[-9.8841e-01]],

          [[ 1.0970e+00]],

          [[ 9.9465e-01]],

          [[ 6.2839e-01]],

          [[ 4.2887e-01]],

          [[-1.1267e-01]],

          [[ 1.0093e+00]],

          [[ 1.7117e-01]],

          [[ 5.0949e-02]],

          [[ 5.4639e-01]],

          [[-8.3234e-01]],

          [[ 9.0153e-03]],

          [[ 1.4181e-01]],

          [[ 9.8213e-01]],
```

```
          [[-6.3601e-02]],

          [[-6.6620e-02]],

          [[-3.4301e-01]],

          [[ 3.8192e-01]],

          [[-6.9915e-01]],

          [[ 6.1393e-01]],

          [[ 1.0175e+00]],

          [[-1.0992e-01]],

          [[ 1.2316e-01]],

          [[-5.3726e-01]],

          [[-5.5751e-01]],

          [[ 2.5642e-01]],

          [[ 1.2400e+00]],

          [[ 8.5095e-01]],

          [[-1.0693e+00]],

          [[-1.2429e-01]],

          [[-6.8953e-02]],

          [[-8.9871e-02]],

          [[ 6.0718e-01]],

          [[-8.4481e-01]],

          [[ 1.9610e-04]],

          [[-6.3661e-01]],

          [[-7.0944e-01]],

          [[ 1.1827e-01]],

          [[-1.1917e+00]],

          [[-1.9241e-01]],
```

```
          [[-1.5031e-01]],

          [[-9.0115e-01]],

          [[-5.9253e-01]],

          [[ 1.4643e+00]],

          [[-2.2710e+00]],

          [[-1.9479e-02]],

          [[ 4.7847e-01]],

          [[-9.2176e-01]],

          [[ 3.2350e-01]],

          [[-5.2813e-02]],

          [[ 1.7046e+00]],

          [[ 1.3062e-01]],

          [[ 2.5749e-02]],

          [[-1.1279e+00]],

          [[ 1.0637e-01]],

          [[-4.5529e-01]],

          [[-1.2004e+00]],

          [[ 3.9507e-01]],

          [[-9.6763e-01]],

          [[ 2.5189e-01]],

          [[ 9.0347e-01]],

          [[ 2.9718e-01]],

          [[-4.0329e-01]],

          [[ 9.6366e-01]],

          [[ 3.5290e-01]],

          [[ 1.1620e-04]],
```

```
          [[-1.0507e+00]],

          [[ 3.1374e-01]],

          [[-5.1042e-01]],

          [[ 5.0413e-01]],

          [[-1.6932e+00]],

          [[ 8.0011e-01]],

          [[-2.9215e-01]],

          [[ 2.1495e-01]],

          [[ 6.6121e-01]],

          [[ 8.2040e-01]],

          [[ 5.3085e-01]],

          [[-2.0360e-01]],

          [[-1.4912e-01]],

          [[ 8.7035e-02]],

          [[ 9.6777e-01]],

          [[-1.2557e+00]],

          [[-3.8028e-01]],

          [[-2.0070e-01]],

          [[ 1.1749e+00]],

          [[-5.5463e-01]],

          [[-1.4607e-01]],

          [[ 1.0779e+00]],

          [[-5.6620e-01]],

          [[ 6.1753e-02]],

          [[ 2.5509e-01]],

          [[-4.2620e-01]],
```

```
          [[ 2.2574e-01]],

          [[-1.3235e+00]],

          [[ 3.2598e-01]],

          [[-1.3641e+00]],

          [[-1.0766e+00]],

          [[ 3.9983e-01]],

          [[ 1.0082e-01]],

          [[ 4.4741e-01]],

          [[-1.5872e+00]],

          [[ 1.5052e+00]],

          [[ 7.5932e-01]],

          [[ 5.6073e-01]],

          [[ 1.0447e+00]],

          [[ 7.1650e-01]],

          [[ 4.7873e-01]],

          [[ 4.2291e-01]],

          [[-6.7641e-01]],

          [[-7.3523e-01]],

          [[-5.7374e-01]],

          [[-1.3152e+00]],

          [[-3.8365e-01]],

          [[-2.4172e-01]],

          [[-8.7745e-01]],

          [[ 4.1240e-01]],

          [[ 1.8924e-01]],

          [[-5.0354e-02]],
```

```
          [[ 8.6203e-01]],

          [[ 1.0235e+00]],

          [[ 3.5567e-01]],

          [[ 6.3870e-01]],

          [[ 1.4320e-01]],

          [[ 1.5943e-01]],

          [[ 1.9402e-01]],

          [[-6.0997e-01]],

          [[ 4.2640e-01]],

          [[-4.7674e-01]],

          [[-2.8009e-01]],

          [[-6.2149e-01]],

          [[-6.7701e-01]],

          [[ 1.3601e-01]],

          [[-1.7640e+00]],

          [[ 1.2582e+00]],

          [[ 3.8354e-01]],

          [[ 4.0615e-01]],

          [[-3.1820e-01]],

          [[-1.0412e+00]],

          [[-9.3291e-02]],

          [[ 7.5668e-01]],

          [[ 3.1545e-01]],

          [[-2.8282e-01]],

          [[-9.2497e-01]],

          [[-9.0596e-01]],
```

```
        [[-1.6577e-01]],
        [[ 1.1176e+00]],
        [[ 3.2412e-01]],
        [[-4.3120e-01]],
        [[ 9.3144e-01]],
        [[-2.8870e-01]],
        [[-5.3488e-01]],
        [[-6.5129e-01]],
        [[ 1.0436e+00]],
        [[-1.0109e+00]],
        [[-1.0601e+00]],
        [[-9.4973e-01]],
        [[-5.4740e-01]],
        [[-1.6481e-02]],
        [[ 9.7703e-01]],
        [[-8.0481e-01]],
        [[-4.2174e-01]],
        [[ 9.9040e-03]],
        [[ 3.8346e-02]],
        [[ 3.2262e-01]],
        [[ 5.4771e-01]],
        [[ 4.0770e-01]],
        [[ 4.9231e-01]],
        [[-7.8215e-01]],
        [[ 1.1961e+00]],
        [[ 2.2141e-01]],
```

```
         [[ 1.0661e+00]],

         [[ 4.6541e-01]],

         [[-1.3424e+00]],

         [[ 2.7983e-01]],

         [[-8.4100e-01]],

         [[ 3.4251e-01]],

         [[-6.6807e-01]],

         [[ 2.2765e-01]],

         [[-1.8946e-01]],

         [[ 3.9993e-01]],

         [[-1.2943e+00]],

         [[ 8.2646e-01]],

         [[-1.6720e-01]],

         [[ 7.2008e-01]],

         [[-2.2824e-01]],

         [[ 8.2099e-01]],

         [[ 2.5278e-01]],

         [[ 2.7353e-01]],

         [[ 6.7246e-02]],

         [[ 1.2372e+00]],

         [[ 2.9780e-01]],

         [[-5.7350e-01]],

         [[ 3.4988e-01]],

         [[-2.5170e-01]],

         [[ 2.5692e-01]],

         [[-2.5321e-01]],
```

```
          [[-9.1765e-02]],

          [[-6.8725e-01]],

          [[ 3.6141e-01]],

          [[ 6.8869e-01]],

          [[ 8.0303e-01]],

          [[-6.8622e-01]],

          [[-4.1416e-01]],

          [[ 2.7895e-01]],

          [[-7.8804e-01]],

          [[ 1.5793e-01]],

          [[-1.2675e+01]],

          [[ 7.2754e-01]],

          [[ 1.4143e+00]],

          [[ 7.1080e-01]],

          [[ 4.9983e-01]],

          [[-6.5552e-01]],

          [[ 4.6715e-01]],

          [[ 4.1481e-01]],

          [[-6.0357e-02]],

          [[-1.0678e+00]],

          [[-9.0345e-02]],

          [[ 1.8623e-01]],

          [[-5.2782e-01]],

          [[ 4.6967e-01]],

          [[ 1.0318e+00]],

          [[ 9.3979e-02]],
```

```
          [[-6.6527e-02]],

          [[ 1.3084e+00]],

          [[ 1.4474e+00]],

          [[ 8.8422e-01]],

          [[-1.1194e+00]],

          [[ 1.6187e+00]],

          [[-1.5331e-02]],

          [[ 1.1897e+00]],

          [[ 1.2209e+00]],

          [[-9.0233e-01]],

          [[-4.6833e-01]],

          [[ 8.6555e-01]],

          [[ 1.2011e+00]],

          [[ 4.6870e-01]],

          [[ 5.3139e-01]],

          [[-1.3784e-01]],

          [[-8.0817e-01]],

          [[ 1.0854e+00]],

          [[ 1.3130e-01]],

          [[ 3.3554e-01]],

          [[-2.8551e-01]],

          [[ 5.3264e-01]],

          [[-4.3933e-01]],

          [[ 1.0503e+00]],

          [[ 4.2251e-01]],

          [[-1.4430e-01]],
```

```
          [[-2.5980e-01]],

          [[ 4.5468e-01]],

          [[-1.6126e+00]],

          [[ 1.2292e+00]],

          [[ 3.7158e-02]],

          [[-6.5638e-01]],

          [[-4.4480e-01]],

          [[ 6.4033e-01]],

          [[-5.6244e-01]],

          [[-2.1365e-01]],

          [[ 1.6726e-01]],

          [[ 1.1575e+00]],

          [[-1.4453e-01]],

          [[-3.7077e-01]],

          [[ 1.3177e-01]],

          [[ 1.7734e-01]],

          [[ 1.3980e-01]],

          [[-5.0295e-01]],

          [[-2.1594e-01]],

          [[-1.8451e-01]],

          [[-1.9216e-01]],

          [[-2.5322e-02]],

          [[ 6.3292e-01]],

          [[-3.6142e-01]],

          [[ 2.0021e-01]],

          [[ 1.1754e+00]],
```

```
        [[-1.2190e+00]],
        [[-1.9271e-01]],
        [[ 1.9619e-02]],
        [[ 3.1101e-01]],
        [[ 1.1290e+00]],
        [[-6.0822e-01]],
        [[-6.2544e-01]],
        [[-1.1560e+00]],
        [[-5.7864e-01]],
        [[-4.5085e-01]],
        [[-1.7944e-01]],
        [[-9.8030e-01]],
        [[ 8.0431e-01]],
        [[-3.0757e-01]],
        [[ 1.4714e+00]],
        [[-7.0618e-01]],
        [[-1.4094e+00]],
        [[ 4.7485e-01]],
        [[-1.0972e-01]],
        [[-7.1466e-02]],
        [[-1.2755e+00]],
        [[ 7.9620e-01]],
        [[ 1.3601e+00]],
        [[ 2.1467e+00]],
        [[-6.5032e-02]],
        [[ 5.5756e-01]],
```

```
         [[ 3.0086e-01]],

         [[ 1.5729e-01]],

         [[-4.3832e-01]],

         [[ 5.0232e-01]],

         [[ 8.7036e-02]],

         [[-1.4385e+00]],

         [[-3.0727e-01]],

         [[ 3.2068e-01]]]], grad_fn=<ViewBackward0>),
decoder_last_hidden_state=tensor([[[[-0.0394,  0.3847,  0.3078,  ...,
0.9337,  0.6636, -0.1648],
          [ 0.5298,  0.8307,  1.1251,  ...,  0.7862,  0.7181,
0.3533],
          [ 0.6198,  1.0891,  1.1049,  ...,  1.3058,  1.2691,
0.5461],
          ...,
          [ 0.4060,  0.6057,  1.3169,  ...,  0.3140,  0.6945,
0.1056],
          [-0.3380, -0.0643,  0.5279,  ..., -0.3634,  0.4347, -
0.0271],
          [-0.0229, -0.3106, -0.1585,  ...,  0.3174,  0.0440,
0.2125]],

         [[-0.0621,  0.2151, -0.0180,  ..., -0.2318, -0.2718, -
0.1362],
          [ 0.0958,  0.0942,  0.6574,  ..., -0.0569,  0.2998,
0.3188],
          [-0.6425,  0.2165,  0.5233,  ...,  0.0278,  0.2142, -
0.0421],
          ...,
          [-0.3925,  0.4646,  0.2964,  ...,  0.6304,  1.0196,
0.3881],
          [-0.1563,  0.5484,  0.6229,  ...,  0.4538,  0.2096,
0.0550],
          [-0.1786,  0.0548,  0.3064,  ...,  0.0923,  0.0943,
0.4567]],

         [[ 0.8325,  0.5772,  0.3556,  ..., -0.4069, -0.3996,
0.0819],
          [ 0.6988,  0.9894,  0.8203,  ...,  0.5581,  0.2241,
0.3283],
          [ 0.9508,  0.7983,  0.6047,  ...,  0.0780,  0.2084,
0.2006],
```

```
          ...,
          [ 0.1536,  0.3705,  0.4615,  ..., -0.1783, -0.5781,
0.1977],
          [ 0.5916,  0.5973,  1.0063,  ..., -0.2255, -0.0198,
0.4913],
          [ 0.3740,  0.0209,  0.2521,  ..., -0.4195,  0.1072,
0.2065]],

         ...,

         [[-0.1659, -0.0436,  0.0498,  ..., -0.0490, -0.3672, -
0.4099],
          [-0.1576, -0.2579, -0.3295,  ..., -0.3392,  0.0969, -
0.4085],
          [-0.2505, -0.9568, -0.7722,  ..., -0.5459, -0.2969, -
0.2592],
          ...,
          [-0.7028, -1.3360, -1.2117,  ..., -0.3692, -0.3356, -
0.5265],
          [-0.2959, -0.9051, -0.6258,  ..., -1.0024, -0.3672, -
0.3177],
          [-0.0380, -0.2380, -0.5848,  ..., -0.4214, -0.1782, -
0.3982]],

         [[-0.4191, -0.5437, -0.2196,  ..., -0.9040, -0.6542, -
0.6674],
          [ 0.0969,  0.0256,  0.4343,  ..., -0.9223, -0.2461,
0.0603],
          [-0.3235, -0.2651, -0.5188,  ..., -0.8223, -0.5154, -
0.3702],
          ...,
          [-0.6946, -0.9204, -0.2715,  ..., -0.2781, -0.1968, -
0.2503],
          [-1.2725, -0.8006, -0.8802,  ..., -1.2366, -0.8686, -
0.3247],
          [-0.2527, -0.5396, -0.1918,  ..., -0.1606, -0.2967,
0.1243]],

         [[ 0.1176,  0.0142, -0.5594,  ..., -0.4376, -0.2191, -
0.1789],
          [-0.7128, -0.7495, -0.9717,  ..., -0.5834, -0.1726,
0.2816],
          [-0.6471, -0.8369, -0.9989,  ..., -0.8901, -0.4796,
0.1938],
          ...,
          [-0.4159, -0.8360, -1.1775,  ..., -0.6892, -0.8306, -
0.5403],
          [-0.1719, -0.6518, -1.0365,  ..., -0.3971, -0.5844, -
0.3616],
          [ 0.1820, -0.0176,  0.1687,  ..., -0.0648, -0.5647,
```

```
0.1085]]]],
       grad_fn=<ConvolutionBackward0>), encoder_hidden_states=(),
decoder_hidden_states=())
```

The number of parameters in the extracted pixel-level module should be around 31,239,099.

```
from torchinfo import summary
model = SegmentModel(model_name)

pytorch_total_params = sum(p.numel() for p in model.parameters() if
p.requires_grad)

summary(model, input_size=[(32, 3, 32, 32)], dtypes=[torch.float])
```

```
==========================================================================
==========================================================================
Layer (type:depth-idx)
Output Shape              Param #
==========================================================================
==========================================================================
SegmentModel
[32, 1, 32, 32]           --
├─MaskFormerPixelLevelModule: 1-1
--                        --
|    └─MaskFormerSwinBackbone: 2-1
[32, 96, 8, 8]            --
|    |    └─MaskFormerSwinModel: 3-1
--                        27,519,354
|    |    └─ModuleList: 3-2
--                        2,880
|    └─MaskFormerPixelDecoder: 2-2
--                        --
|    |    └─MaskFormerFPNModel: 3-3
[32, 256, 2, 2]           3,714,560
|    |    └─Conv2d: 3-4
[32, 1, 8, 8]             2,305
==========================================================================
==========================================================================
Total params: 31,239,099
Trainable params: 31,239,099
Non-trainable params: 0
Total mult-adds (Units.GIGABYTES): 2.63
==========================================================================
==========================================================================
Input size (MB): 0.39
Forward/backward pass size (MB): 335.36
Params size (MB): 124.86
Estimated Total Size (MB): 460.62
```

```
================================================================================
================================================================
```

Insert LoRA into the MaskFormer model

The concept of LoRA is that we are going to estimate the gradient (adaptation matrix) with two smaller matrices (A and B): \begin{aligned} \text{Adaptation Matrix} = B \times A \end{aligned} where $\text{Adaptation Matrix} \in R^{m \times n}$, $A \in R^{r \times n}$, and $B \in R^{m \times r}$. We could make this approximation based on the assumption that Adaptation Matrix has a rank of r. Therefore, the fine-tuned weight becomes \begin{aligned} W &= W_0 + \Delta W \ &= W_0 + \frac{\alpha}{r} BA \ \end{aligned} where W denotes the fine-tuned weight, W_0 represents pre-trained weight, ΔW is the gradient and α can be seen as a learning rate. A is initialized using a common initialization, like Kaiming initialization, during the initialization process. On the other hand, B is set to 0 such that the model's output remains the same after injecting LoRA, resulting in a stabilized training process.

To summarize, when injecting LoRA to a layer, we insert new parameters called matrix A and B and initialize them using the above description. Then, we modify the forward pass with `forward_hook` such that the output becomes \begin{aligned} h = W x + \frac{\alpha}{r} BA x \ \end{aligned} where x and h are the input and output, respectively. We recommend you read this blog to learn more about `forward_hook`.

LoRA on Linear Layer

TODO 14: initialize A and B to ones (every entry in the matrix is one), such that we can verify your forward pass after attach hook. TODO 15: implement the forward hook such that new output h is

$$h = W x + \frac{\alpha}{r} B A x$$

LoRA on Convolutional Layer

TODO 16: initialize A and B to ones (every entry in the matrix is one), such that we can verify your forward pass after attach hook. TODO 17: implement the forward hook such that new output h is

$$h = W x + \frac{\alpha}{r} B A x$$

Note: When injecting LoRA into a convolutional layer with kernel size k, the shape of matrix A and B becomes $(r \times k, \text{in features} \times k)$ and $(\text{out features} \times k, r \times k)$, respectively. It comes from the fact that we can see the weight of the convolutional layer as the weight of $k \times k$ linear layers.

Hint: When you want to declare and initialize a parameter, you can use `torch.nn.Parameter` and `torch.nn.init`, respectively.

```
import math
# Initialize LoRA and attach a hook.
```

```
def attach_lora(layer, r, lora_alpha, in_features, out_features): #
you can specify layer
    assert r > 0, "rank must greater than 0."
    # TODO 14: Declare A and B matrices and initialize A and B to
ones.
    # W should be (out_features, in_features) because it's Wx
    layer.lora_A = torch.nn.Parameter(torch.ones((r, in_features)))
    layer.lora_B = torch.nn.Parameter(torch.ones((out_features, r)))

    def hook(model, input, output):
        assert len(input) == 1, "The length of the input must be 1."
        # TODO 15: Compute adapatation matrix (BA) and modify the
forward pass.
        x = input[0].detach() # (batch, in)
        tuned = (lora_alpha / r) * (x @ (layer.lora_B @
layer.lora_A).T)
        return output + tuned

    return hook

def attach_conv_lora(layer, r, lora_alpha, in_features, out_features,
kernel_size):
    assert r > 0, "rank must greater than 0."
    # TODO 16: initialize metrix A and B in LoRA
    layer.lora_A = torch.nn.Parameter(torch.ones((r * kernel_size,
in_features * kernel_size)))
    layer.lora_B = torch.nn.Parameter(torch.ones((out_features *
kernel_size, r * kernel_size)))

    def hook(model, input, output):
        assert len(input) == 1, "The length of the input must be 1."
        # TODO 17: Compute adapatation matrix (BA) and modify the
forward pass.
        x = input[0].detach() # (batch, channel, height, width)
        ba = layer.lora_B @ layer.lora_A # (out *k, in * k)
        ba_kernel = ba.view(out_features, kernel_size, in_features,
kernel_size).permute((0, 2, 1, 3))
        tuned = F.conv2d(x, ba_kernel, stride=layer.stride,
padding=layer.padding)
        return output + (lora_alpha / r) * tuned
    return hook
```

To test your `forward_hook`, we will check the difference of the output before and after injecting the LoRA when you initialize matrices A and B with ones.

```
class DummyLinear(nn.Module):
  def __init__(self):
    super().__init__()
    self.linear = nn.Linear(10, 20)
```

```
  def forward(self, x):
    return self.linear(x)

r, lora_alpha = 1, 4
input_ = torch.arange(10, dtype=torch.float32).unsqueeze(0)
dummy_linear = DummyLinear()
output_before = dummy_linear(input_)
for name, module in dummy_linear.named_modules():
  if isinstance(module, torch.nn.modules.Linear):
    out_features, in_features = module.weight.shape
    h = module.register_forward_hook(attach_lora(module, r,
lora_alpha, in_features, out_features))
output_after = dummy_linear(input_)

if torch.all(torch.isclose(output_after - output_before, lora_alpha *
input_.sum() * torch.ones_like(output_before))):
  print("Your forward hook seems to be correct.")
else:
  print("There is something wrong with your forward hook.")
```

```
Your forward hook seems to be correct.
```

Analyze Low-rank Adaptation (TODO)

How many parameters in DummyLinear module before and after injecting LoRA?

Ans.

Before: (10 + 1) * 20 = 220

After: 220 + (10 * 1) + (20 * 1) = 250

Check LoRA in convolutional layer

```
class DummyConvolution(nn.Module):
  def __init__(self):
    super().__init__()
    self.conv = nn.Conv2d(10, 20, kernel_size=3, stride=3)

  def forward(self, x):
    return self.conv(x)

r, lora_alpha = 1, 4
input_ = torch.ones(1, 10, 9, 9) # torch.arange(10,
dtype=torch.float32).unsqueeze(0)
dummy_conv = DummyConvolution()
output_before = dummy_conv(input_)
for name, module in dummy_conv.named_modules():
  if isinstance(module, torch.nn.modules.Conv2d):
      out_features, in_features, kernel_size, _ = module.weight.shape
```

```
      assert kernel_size == _
      h = module.register_forward_hook(attach_conv_lora(module, r,
lora_alpha, in_features, out_features, kernel_size))
output_after = dummy_conv(input_)

if torch.all(torch.isclose(output_after - output_before, 1080 *
torch.ones_like(output_before))):
  print("Your forward hook seems to be correct.")
else:
  print("There is something wrong with your forward hook.")
```

```
Your forward hook seems to be correct.
```

Instruction

TODO 18-19: Change the initialization of A and B where A is initialized with Kaiming Uniform (a = sqrt(5)), and B is set to 0.

```
import math
# Initialize LoRA and attach a hook.
def attach_lora(layer, r, lora_alpha, in_features, out_features):
    assert r > 0, "rank must greater than 0."
    # TODO 18: initialize A with kaiming uniform with a = sqrt(5) and
initialize B to 0.
    layer.lora_A = torch.nn.Parameter(torch.ones((r, in_features)))
    torch.nn.init.kaiming_uniform_(layer.lora_A, a=(5 ** 0.5))
    layer.lora_B = torch.nn.Parameter(torch.zeros((out_features, r)))

    def hook(model, input, output):
        assert len(input) == 1, "The length of the input must be 1."
        # Copy from TODO 15
        x = input[0].detach() # (batch, in)
        tuned = (lora_alpha / r) * (x @ (layer.lora_B @
layer.lora_A).T)
        return output + tuned
    return hook

def attach_conv_lora(layer, r, lora_alpha, in_features, out_features,
kernel_size):
    assert r > 0, "rank must greater than 0."
    # TODO 19: initialize A with kaiming uniform with a = sqrt(5) and
initialize B to 0
    layer.lora_A = torch.nn.Parameter(torch.ones((r * kernel_size,
in_features * kernel_size)))
    torch.nn.init.kaiming_uniform_(layer.lora_A, a=(5 ** 0.5))
    layer.lora_B = torch.nn.Parameter(torch.zeros((out_features *
kernel_size, r * kernel_size)))

    def hook(model, input, output):
        assert len(input) == 1, "The length of the input must be 1."
```

```
        # COPY from TODO 17
        x = input[0].detach() # (batch, channel, height, width)
        ba = layer.lora_B @ layer.lora_A # (out *k, in * k)
        ba_kernel = ba.view(out_features, kernel_size, in_features,
kernel_size).permute((0, 2, 1, 3))
        tuned = F.conv2d(x, ba_kernel, stride=layer.stride,
padding=layer.padding)
        return output + (lora_alpha / r) * tuned
    return hook
```

We only inject LoRA into the convolutional layer in the decoder block. TODO 20: inject lora into a decoder block

```
r, lora_alpha = 1, 4
def attach_lora_to_maskformer(model, r, lora_alpha):
  hooks = []
  for name, module in model.named_modules():
      # TODO 20: inject lora into convolutional layers in the decoder
block
      # Do not forget to append registered hook to hooks
      if (("decoder" in name) and isinstance(module,
torch.nn.Conv2d)):
        # able to access variables in torch.nn.Conv2d
        in_features = module.in_channels
        out_features = module.out_channels
        kernel_size = module.kernel_size[0]

        hook = module.register_forward_hook(attach_conv_lora(module,
r, lora_alpha, in_features, out_features, kernel_size))
        hooks.append(hook)
  return hooks

hooks = attach_lora_to_maskformer(model, r, lora_alpha)
```

We freeze every layer except LoRA layer (A and B matrix), bias in the decoder, and the last layer (segment_prediction). TODO 21: freeze every layer except LoRA layer (A and B matrix), bias in the decoder, and the last layer (segment_prediction).

```
for name, p in model.named_parameters():
    # TODO 21: freeze every layer except LoRA layer, bias in the
decoder and the last layer.
    # print(name)
    if (("lora" in name) or (("mask_projection" in name) and
("decoder" in name)) or (("bias" in name) and ("decoder" in name))):
      p.requires_grad = True
    else:
      p.requires_grad = False
```

The number of learnable parameters is around 28k.

```
pytorch_total_params = sum(p.numel() for p in model.parameters() if
p.requires_grad)
print(pytorch_total_params)
```

```
30890
```

Training

```
from tqdm import tqdm

# Save on the trainable parameters
def save_trainable_params(model, path):
  trainable_state_dict = {}
  for n, p in model.named_parameters():
    if p.requires_grad:
      trainable_state_dict[n] = p

  torch.save(trainable_state_dict, path)


device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)
criteria = nn.BCEWithLogitsLoss()
# Initialize Adam optimizer
optimizer = torch.optim.Adam(model.parameters(), lr=3e-4)
# Set number of epochs and batch size
num_epochs = 10
training_losses = []
validation_losses = []
for epoch in range(num_epochs):
    print(f"Epoch {epoch} | Training")
    # Set model in training mode
    model.train()
    train_loss, val_loss = [], []
    # Training loop
    for idx, batch in enumerate(tqdm(train_dataloader)):
        # Reset the parameter gradients
        optimizer.zero_grad()

        # Forward pass
        outputs = model(batch["pixel_values"].to(device))
        # Backward propagation
        loss = criteria(outputs.squeeze(1),
batch["segmentation"].to(device))
        train_loss.append(loss.item())
        loss.backward()
        if idx % 100 == 0:
            print("  Training loss: ",
round(sum(train_loss)/len(train_loss), 6))
        # Optimization
```

```
        optimizer.step()
        torch.cuda.empty_cache()
    # Average train epoch loss
    train_loss = sum(train_loss)/len(train_loss)
    # Set model in evaluation mode
    model.eval()
    start_idx = 0
    print(f"Epoch {epoch} | Validation")
    for idx, batch in enumerate(tqdm(val_dataloader)):
        with torch.no_grad():
            # Forward pass
            outputs = model(batch["pixel_values"].to(device))
            # Get validation loss
            loss = criteria(outputs.squeeze(1),
batch["segmentation"].to(device))
            val_loss.append(loss.item())
            if idx % 50 == 0:
                print("  Validation loss: ",
round(sum(val_loss)/len(val_loss), 6))
        torch.cuda.empty_cache()
    # Average validation epoch loss
    val_loss = sum(val_loss)/len(val_loss)

    training_losses.append(train_loss)
    validation_losses.append(val_loss)
    # Print epoch losses
    print(f"Epoch {epoch} | train_loss: {train_loss} |
validation_loss: {val_loss}")

save_trainable_params(model, "./lora_weight.ckpt")
```

```
Epoch 0 | Training

  0%|          | 1/212 [00:01<04:22,  1.24s/it]

  Training loss:  0.567393

 48%|████      | 101/212 [01:12<01:13,  1.52it/s]

  Training loss:  0.319237

 95%|█████████ | 201/212 [02:25<00:07,  1.54it/s]

  Training loss:  0.277091

100%|██████████| 212/212 [02:32<00:00,  1.39it/s]

Epoch 0 | Validation

  2%|          | 1/61 [00:00<00:39,  1.52it/s]

  Validation loss:  0.228943
```

```
 84%|████████  | 51/61 [00:37<00:06,  1.58it/s]

  Validation loss:  0.207863

100%|██████████| 61/61 [00:42<00:00,  1.42it/s]

Epoch 0 | train_loss: 0.2741350771964721 | validation_loss:
0.20486473939457878
Epoch 1 | Training

  0%|          | 1/212 [00:00<02:23,  1.47it/s]

  Training loss:  0.222287

 48%|████      | 101/212 [01:09<01:33,  1.19it/s]

  Training loss:  0.215963

 95%|█████████ | 201/212 [02:24<00:06,  1.58it/s]

  Training loss:  0.213123

100%|██████████| 212/212 [02:31<00:00,  1.40it/s]

Epoch 1 | Validation

  2%|          | 1/61 [00:00<00:40,  1.48it/s]

  Validation loss:  0.207323

 84%|████████  | 51/61 [00:36<00:06,  1.58it/s]

  Validation loss:  0.193236

100%|██████████| 61/61 [00:42<00:00,  1.42it/s]

Epoch 1 | train_loss: 0.21199223926326013 | validation_loss:
0.191017178726978
Epoch 2 | Training

  0%|          | 1/212 [00:00<02:16,  1.55it/s]

  Training loss:  0.239298

 48%|████      | 101/212 [01:10<01:20,  1.38it/s]

  Training loss:  0.208237

 95%|█████████ | 201/212 [02:23<00:07,  1.50it/s]

  Training loss:  0.203805

100%|██████████| 212/212 [02:31<00:00,  1.40it/s]

Epoch 2 | Validation
```

```
  2%|▏          | 1/61 [00:00<00:39,  1.53it/s]

  Validation loss:  0.203133

 84%|████████▍  | 51/61 [00:36<00:06,  1.59it/s]

  Validation loss:  0.190165

100%|██████████| 61/61 [00:42<00:00,  1.42it/s]

Epoch 2 | train_loss: 0.20367971336785354 | validation_loss:
0.18814396345224538
Epoch 3 | Training

  0%|           | 1/212 [00:00<02:12,  1.59it/s]

  Training loss:  0.164594

 48%|████▊      | 101/212 [01:15<01:09,  1.59it/s]

  Training loss:  0.193402

 95%|█████████▍ | 201/212 [02:25<00:07,  1.56it/s]

  Training loss:  0.197006

100%|██████████| 212/212 [02:32<00:00,  1.39it/s]

Epoch 3 | Validation

  2%|▏          | 1/61 [00:00<00:38,  1.57it/s]

  Validation loss:  0.199199

 84%|████████▍  | 51/61 [00:37<00:06,  1.59it/s]

  Validation loss:  0.183012

100%|██████████| 61/61 [00:43<00:00,  1.42it/s]

Epoch 3 | train_loss: 0.19707720818103486 | validation_loss:
0.1810377003228078
Epoch 4 | Training

  0%|           | 1/212 [00:00<02:14,  1.57it/s]

  Training loss:  0.188217

 48%|████▊      | 101/212 [01:09<01:12,  1.52it/s]

  Training loss:  0.190596

 95%|█████████▍ | 201/212 [02:24<00:07,  1.47it/s]

  Training loss:  0.193292
```

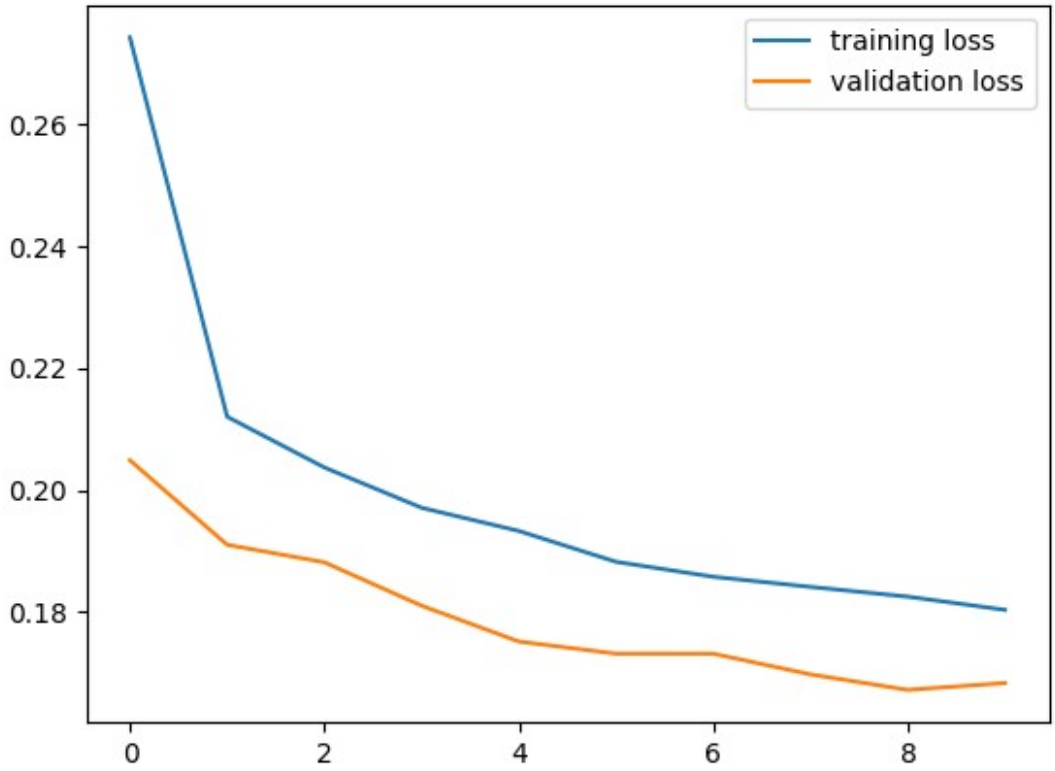
```
100%|██████████| 212/212 [02:32<00:00,  1.39it/s]

Epoch 4 | Validation

  2%|          | 1/61 [00:00<00:38,  1.56it/s]

  Validation loss:  0.194675

 84%|████████  | 51/61 [00:37<00:06,  1.58it/s]

  Validation loss:  0.177518

100%|██████████| 61/61 [00:43<00:00,  1.41it/s]

Epoch 4 | train_loss: 0.19327869030805128 | validation_loss:
0.17515653005388918
Epoch 5 | Training

  0%|          | 1/212 [00:00<02:13,  1.58it/s]

  Training loss:  0.198485

 48%|████▊     | 101/212 [01:14<01:47,  1.03it/s]

  Training loss:  0.191376

 95%|█████████▍| 201/212 [02:24<00:07,  1.50it/s]

  Training loss:  0.188403

100%|██████████| 212/212 [02:32<00:00,  1.39it/s]

Epoch 5 | Validation

  2%|          | 1/61 [00:00<00:38,  1.55it/s]

  Validation loss:  0.192021

 84%|████████  | 51/61 [00:36<00:06,  1.60it/s]

  Validation loss:  0.175485

100%|██████████| 61/61 [00:42<00:00,  1.44it/s]

Epoch 5 | train_loss: 0.18822080916110076 | validation_loss:
0.17317393874047232
Epoch 6 | Training

  0%|          | 1/212 [00:00<02:17,  1.54it/s]

  Training loss:  0.205697

 48%|████▊     | 101/212 [01:09<01:28,  1.26it/s]

  Training loss:  0.18608
```

```
 95%|█████████ | 201/212 [02:25<00:07,  1.56it/s]

  Training loss:  0.185445

100%|██████████| 212/212 [02:31<00:00,  1.40it/s]

Epoch 6 | Validation

  2%|▏         | 1/61 [00:00<00:39,  1.51it/s]

  Validation loss:  0.190734

 84%|████████  | 51/61 [00:37<00:06,  1.56it/s]

  Validation loss:  0.175292

100%|██████████| 61/61 [00:43<00:00,  1.41it/s]

Epoch 6 | train_loss: 0.18577225421959498 | validation_loss:
0.17316788178486903
Epoch 7 | Training

  0%|          | 1/212 [00:00<02:20,  1.50it/s]

  Training loss:  0.208062

 48%|████▊     | 101/212 [01:13<01:20,  1.38it/s]

  Training loss:  0.180523

 95%|█████████ | 201/212 [02:25<00:07,  1.46it/s]

  Training loss:  0.184661

100%|██████████| 212/212 [02:31<00:00,  1.40it/s]

Epoch 7 | Validation

  2%|▏         | 1/61 [00:00<00:38,  1.54it/s]

  Validation loss:  0.192219

 84%|████████  | 51/61 [00:37<00:06,  1.60it/s]

  Validation loss:  0.172276

100%|██████████| 61/61 [00:43<00:00,  1.42it/s]

Epoch 7 | train_loss: 0.18411671370267868 | validation_loss:
0.16978015481937128
Epoch 8 | Training

  0%|          | 1/212 [00:00<02:18,  1.52it/s]

  Training loss:  0.194421
```

```
 48%|████▊     | 101/212 [01:15<01:28,  1.25it/s]
  Training loss:  0.180306
 95%|█████████▍| 201/212 [02:25<00:11,  1.05s/it]
  Training loss:  0.182905
100%|██████████| 212/212 [02:31<00:00,  1.40it/s]
Epoch 8 | Validation
  2%|▏         | 1/61 [00:00<00:38,  1.56it/s]
  Validation loss:  0.187299
 84%|████████▍ | 51/61 [00:36<00:06,  1.60it/s]
  Validation loss:  0.169739
100%|██████████| 61/61 [00:42<00:00,  1.43it/s]
Epoch 8 | train_loss: 0.1825173795926121 | validation_loss:
0.16724423636667063
Epoch 9 | Training
  0%|          | 1/212 [00:00<02:11,  1.61it/s]
  Training loss:  0.16004
 48%|████▊     | 101/212 [01:11<01:12,  1.53it/s]
  Training loss:  0.183953
 95%|█████████▍| 201/212 [02:23<00:07,  1.55it/s]
  Training loss:  0.180117
100%|██████████| 212/212 [02:31<00:00,  1.40it/s]
Epoch 9 | Validation
  2%|▏         | 1/61 [00:00<00:39,  1.54it/s]
  Validation loss:  0.193304
 84%|████████▍ | 51/61 [00:37<00:06,  1.58it/s]
  Validation loss:  0.171017
100%|██████████| 61/61 [00:43<00:00,  1.42it/s]
Epoch 9 | train_loss: 0.18035345542121609 | validation_loss:
0.16836768681885766
```

Plot training and validation loss.

```
plt.plot(range(num_epochs), training_losses, label="training loss")
plt.plot(range(num_epochs), validation_losses, label="validation
loss")
plt.legend()
plt.show()
```




```
test_loss = []
for idx, batch in enumerate(tqdm(test_dataloader)):
    with torch.no_grad():
        outputs = model(batch["pixel_values"].to(device))
        loss = criteria(outputs.squeeze(1),
batch["segmentation"].to(device))
        test_loss.append(loss.item())
    torch.cuda.empty_cache()
# Average validation epoch loss
test_loss = sum(test_loss)/len(test_loss)
print("Test Loss:", test_loss)
```

```
100%|██████████| 31/31 [00:21<00:00,  1.41it/s]

Test Loss: 0.16807981580495834
```

Inference

After fine-tuning the model, we need to load the model back for inference. First, we load the pre-trained weight. Next, we attach LoRA to the foundation model and then load the LoRA's weight.

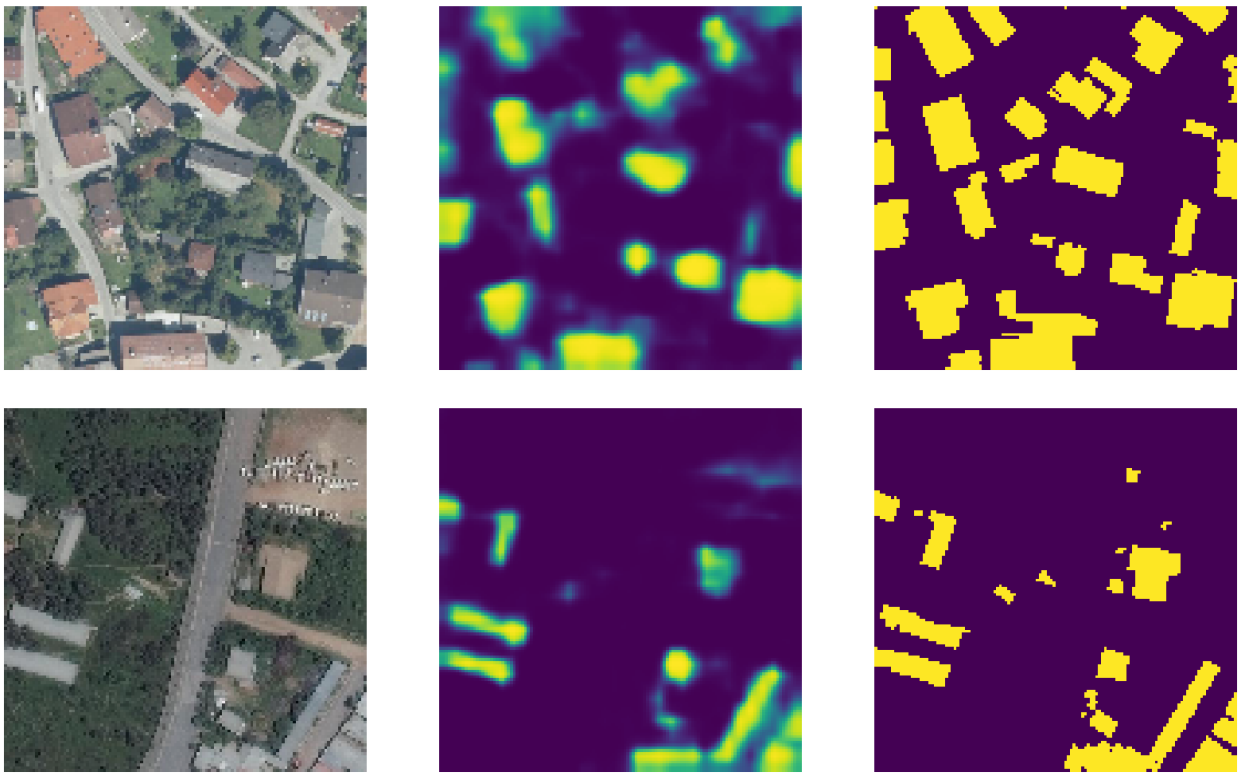
```
model = SegmentModel(model_name)
hooks = attach_lora_to_maskformer(model, r, lora_alpha)

model.load_state_dict(torch.load("./lora_weight.ckpt"), strict=False)
model.cuda()

test_loss = []
for idx, batch in enumerate(tqdm(test_dataloader)):
    with torch.no_grad():
        outputs = model(batch["pixel_values"].to(device))
        loss = criteria(outputs.squeeze(1),
batch["segmentation"].to(device))
        test_loss.append(loss.item())
    torch.cuda.empty_cache()
# Average validation epoch loss
test_loss = sum(test_loss)/len(test_loss)
print("Test Loss:", test_loss)
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/
file_download.py:1132: FutureWarning: `resume_download` is deprecated
and will be removed in version 1.0.0. Downloads always resume when
possible. If you want to force a new download, use
`force_download=True`.
  warnings.warn(
100%|██████████| 31/31 [00:22<00:00,  1.41it/s]

Test Loss: 0.16807981580495834
```

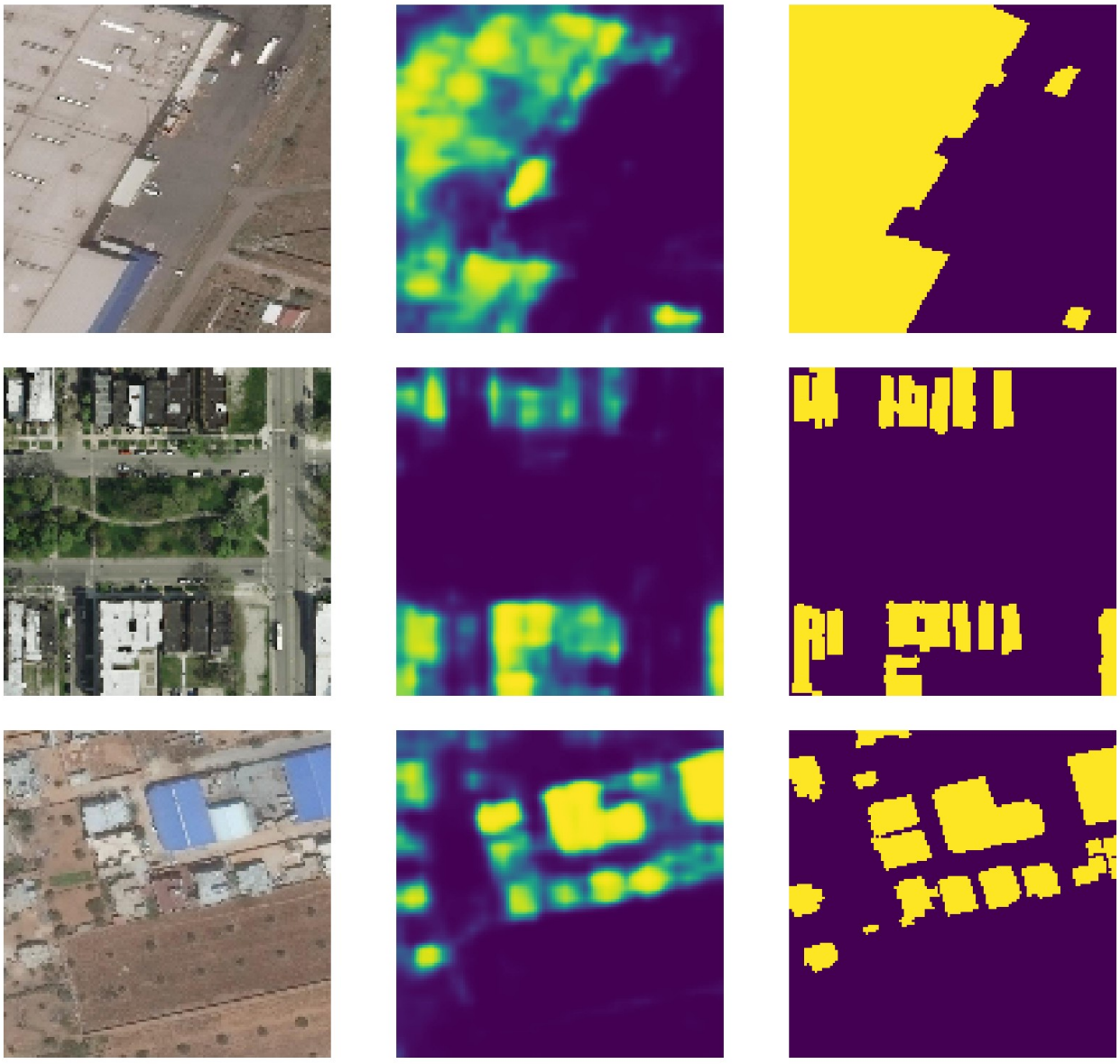
```
test_dataloader = DataLoader(
    test_dataset,
    batch_size=1,
    shuffle=False,
    collate_fn=collate_fn
)
num_images = 10
model.cuda()
for idx, batch in enumerate(test_dataloader):
    image = unnormalize_image(batch["pixel_values"][0])
    outputs = model(batch["pixel_values"].to(device))
    outputs = F.sigmoid(outputs).cpu().detach().numpy()
```

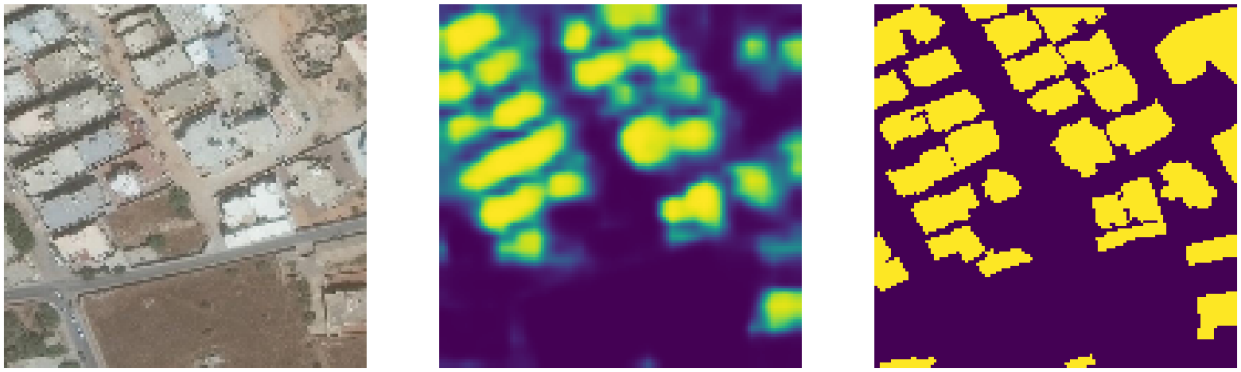
```
    mask = batch["segmentation"][0]

    fig, axs = plt.subplots(ncols=3, figsize=(20, 10))
    axs[0].imshow(np.array(rearrange(image, "c h w -> h w c")))
    axs[1].imshow(outputs[0].squeeze())
    axs[2].imshow(np.array(mask))
    axs[0].set_axis_off()
    axs[1].set_axis_off()
    axs[2].set_axis_off()
    plt.show()

    if idx >= num_images:
        break
```

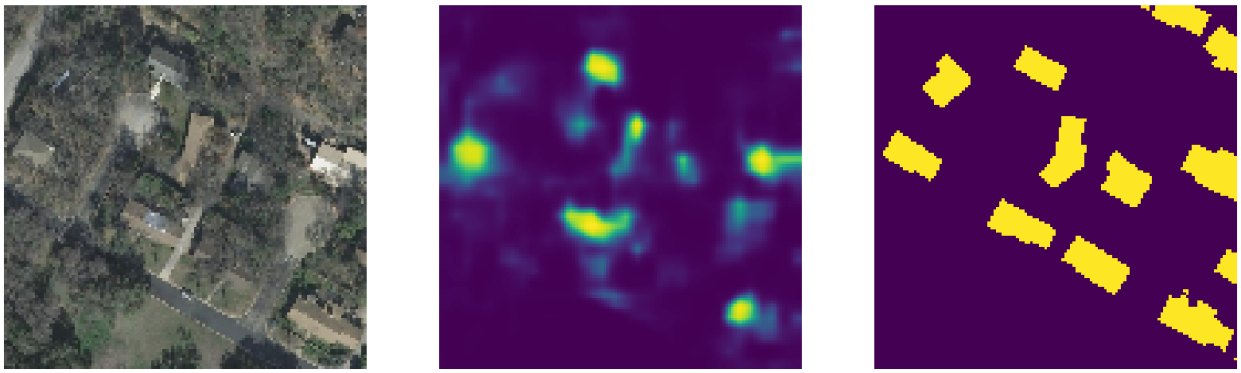
```
/tmp/ipykernel_1607/1125473636.py:16: DeprecationWarning:
__array_wrap__ must accept context and return_scalar arguments
(positionally) in the future. (Deprecated NumPy 2.0)
  image = image * np.array([0.229, 0.224, 0.225])[:, None, None]
/tmp/ipykernel_1607/1125473636.py:17: DeprecationWarning:
__array_wrap__ must accept context and return_scalar arguments
(positionally) in the future. (Deprecated NumPy 2.0)
  image = image + np.array([0.485, 0.456, 0.406])[:, None, None]
```



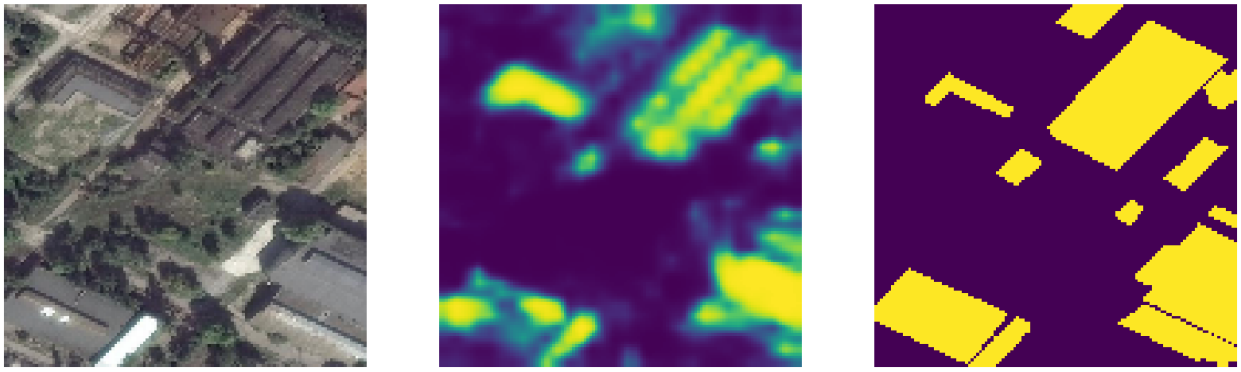
```
WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.01960783576965336..1.0000000610351563].
```

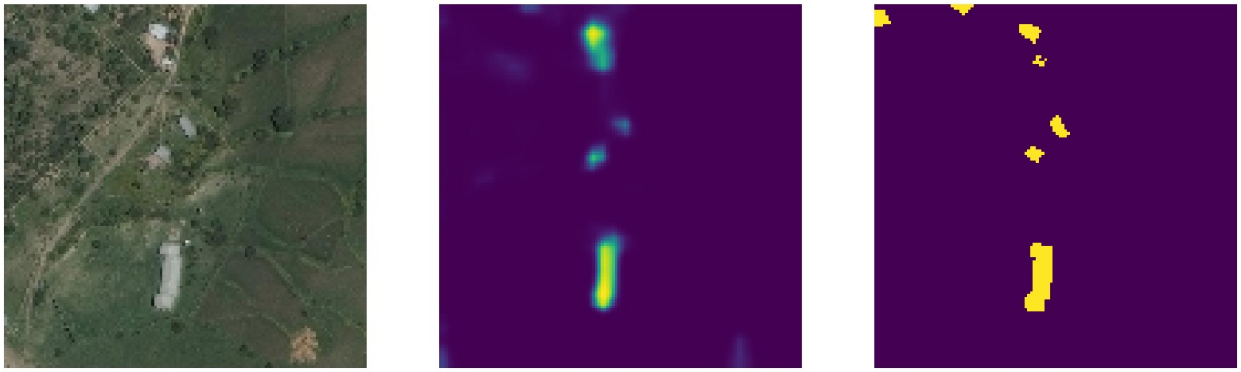




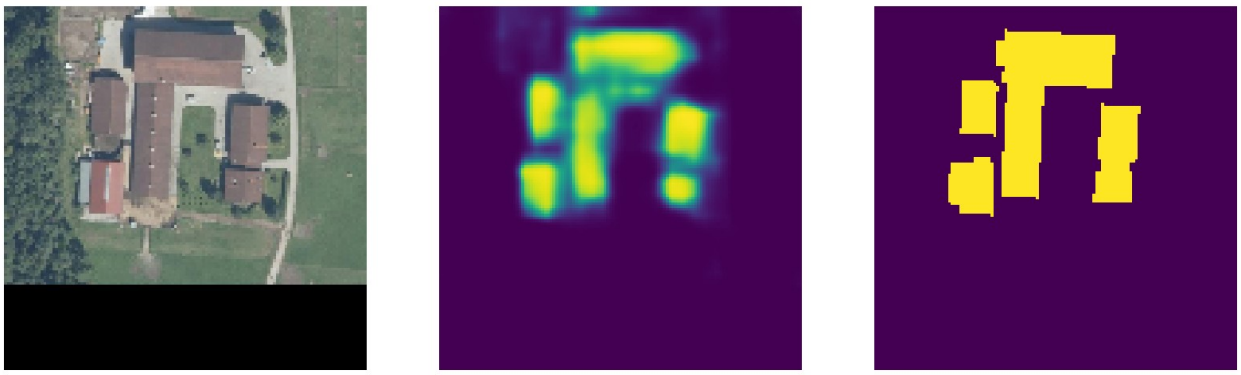


```
WARNING:matplotlib.image:Clipping input data to the valid range for
imshow with RGB data ([0..1] for floats or [0..255] for integers). Got
range [0.05098038780689235..1.0000000610351563].
```

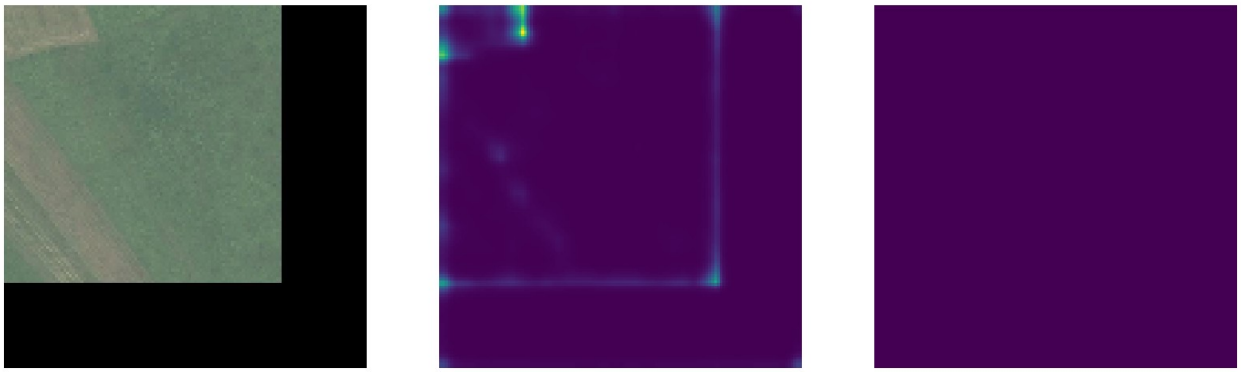




```
WARNING:matplotlib.image:Clipping input data to the valid range for
imshow with RGB data ([0..1] for floats or [0..255] for integers). Got
range [-2.288818357065736e-08..0.9921568689346314].
```



```
WARNING:matplotlib.image:Clipping input data to the valid range for
imshow with RGB data ([0..1] for floats or [0..255] for integers). Got
range [-2.288818357065736e-08..0.6313725719451905].
```



(Optional) Part 3: Low-rank Adaptation on Diffusion

In this section, we combine sections 1 and 2. Instead of controlling the diffusion with digits, we will regulate it with text by using CLIP, trained to align the text and image embedding in the same space.

To make it even more interesting, we fill color into the MNIST dataset and use the prompt to generate the desired images. In addition, we color only digits 0 to 4 while leaving digits 5-9 to their original color (white).

Preprocessing

First, we create Dataset and declare six colors, including red, green, blue, purple, yellow and white. Then we define the corresponding prompt for each image.

```
import random
from PIL import Image
import numpy as np
from torchvision import datasets
from torchvision.transforms import Compose, ToTensor, Resize
from torch.utils.data import Dataset

COLOR = {
    "red": (1, 0, 0),
    "green": (0, 1, 0),
    "blue": (0, 0, 1),
    "purple": (1, 0, 1),
    "yellow": (1, 1, 0),
    "white": (1, 1, 1)
}

def fill_color(img, color):
    """ Filled color into the image.

    Args:
        img (PIL.Image): A pillow image to be filled.
        color (str): A color to be used.
    Returns:
        The modified image (PIL.Image)
    """
    r, g, b = COLOR[color]
    data = np.array(img)   # "data" is a height x width x 4 numpy
array
    data[..., 0] *= r
    data[..., 1] *= g
    data[..., 2] *= b
    data = data.astype(np.uint8)
```

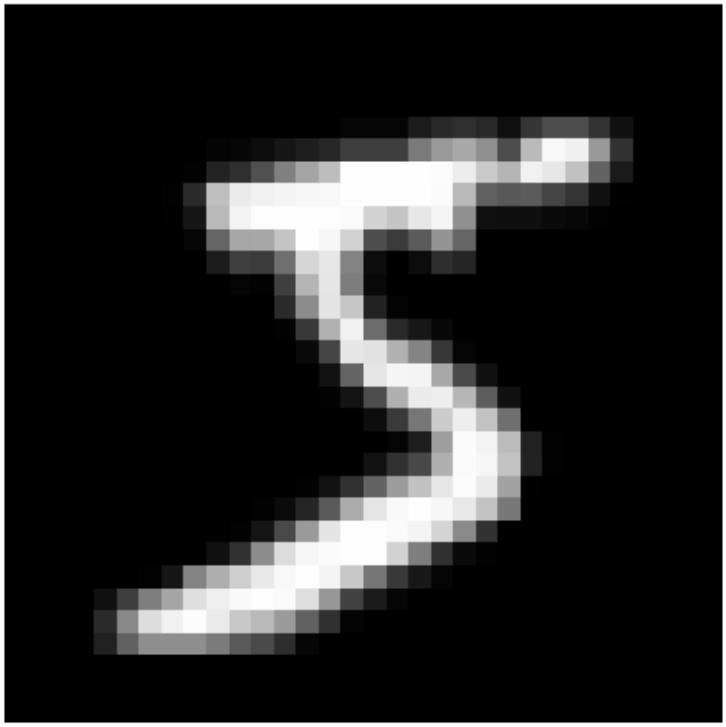
```
    img = Image.fromarray(data)
    return img

class MNISTCaptionDatset(Dataset):
    def __init__(self, tokenizer, color=False, *args, **kwargs):
        """ Initialize MNIST dataset with text label.

        Args:
            color (bool): If set to true, the digit is filled with
random color. Default to False.
        """
        assert kwargs.get("transform") is None, "Do not support
transform function."
        self.data = datasets.MNIST(*args, **kwargs)
        self.tokenizer = tokenizer
        self.color = color
        self.classes = list(map(lambda text_label: text_label.split()
[-1], self.data.classes))
        # Default transform function
        self.transform = Compose([
            ToTensor(), Resize((32, 32)), normalize_to_neg_one_to_one
        ])

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        img, label = self.data[idx]
        img = img.convert('RGB')
        color = ""
        if self.color and label in list(range(0, 5)):
            color = random.choice(list(COLOR.keys()))
            color = "" if color == "white" else color
            if color != "":
                img = fill_color(img, color)
        img = self.transform(img)

        text_label = " ".join([self.classes[label], color]).strip()
        input_ids = self.tokenizer(text_label, padding="max_length",
truncation=True, max_length=self.tokenizer.model_max_length,
return_tensors="pt").input_ids[0]
        return img, input_ids

from transformers import CLIPTextModel, CLIPTokenizer

mnist_kwargs = {
    "root":"./data",
    "download":True,
}
tokenizer = CLIPTokenizer.from_pretrained("CompVis/stable-diffusion-
```

```
v1-4", subfolder="tokenizer", revision=None)
text_encoder = CLIPTextModel.from_pretrained("CompVis/stable-
diffusion-v1-4", subfolder="text_encoder", revision=None)

train_data = MNISTCaptionDatset(tokenizer, color=True, train=True,
**mnist_kwargs)
test_data = MNISTCaptionDatset(tokenizer, color=True, train=False,
**mnist_kwargs)

train_dataloader = DataLoader(train_data, batch_size=128)
test_dataloader = DataLoader(test_data, batch_size=128)
```

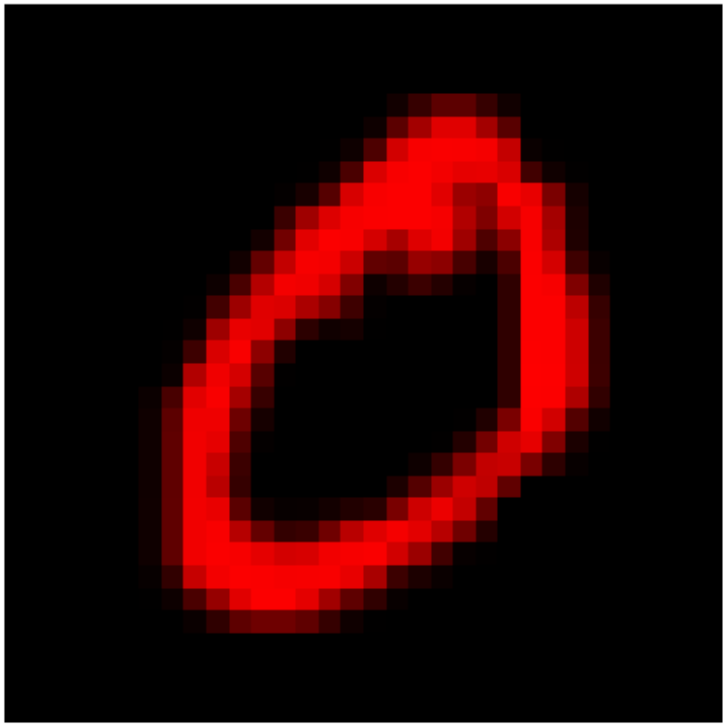

```
{"model_id":"71b16b68fd064a68a53f9ccbfa648994","version_major":2,"vers
ion_minor":0}

{"model_id":"b181ca27d28d480fa04bec5a8f6d9f39","version_major":2,"vers
ion_minor":0}

{"model_id":"ca4c842f3a394cc9994c0118d1d9a16f","version_major":2,"vers
ion_minor":0}

{"model_id":"e59403861b2a464daaf196851e2eea3e","version_major":2,"vers
ion_minor":0}

{"model_id":"a428ec9a4d104fad964ad0e733be72ad","version_major":2,"vers
ion_minor":0}

{"model_id":"8e3a2b64dd9d4751a9be4e9fd13beb0c","version_major":2,"vers
ion_minor":0}
```


```
for i in range(10):
    image, label = train_data[i]
    text = tokenizer.decode(label, skip_special_tokens=True)
    plt.imshow(rearrange(unnormalize_to_zero_to_one(image), "c h w ->
h w c"))
    plt.title("Prompt: " + text)
    plt.axis(False)
    plt.show()
```

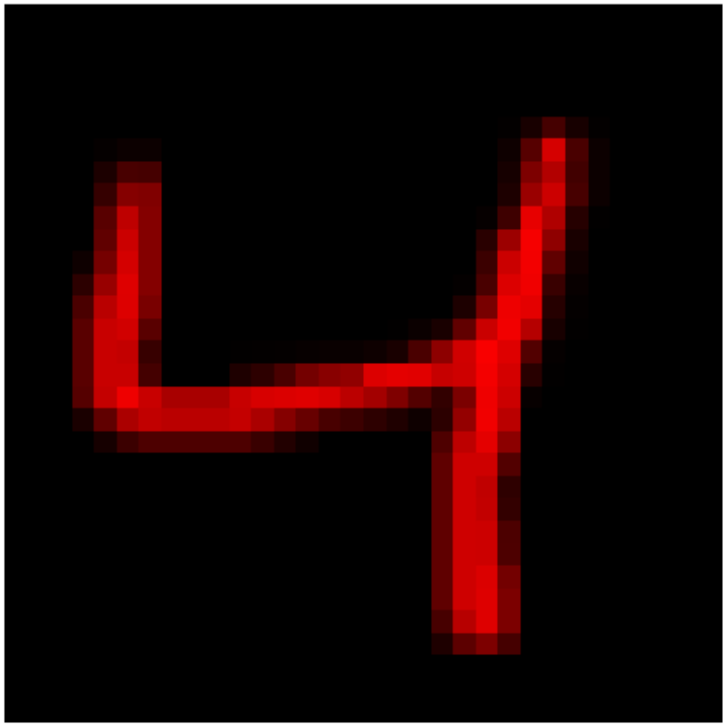
Prompt: five



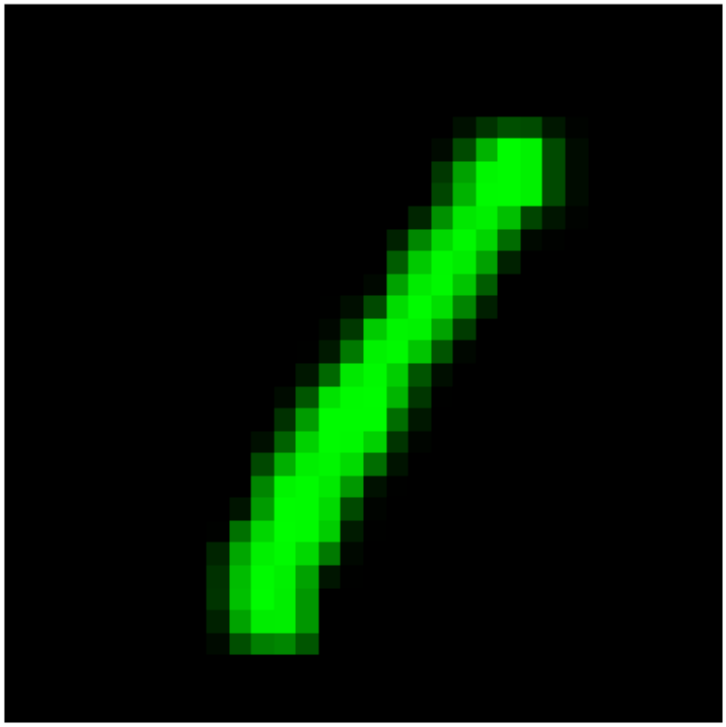
Prompt: zero red



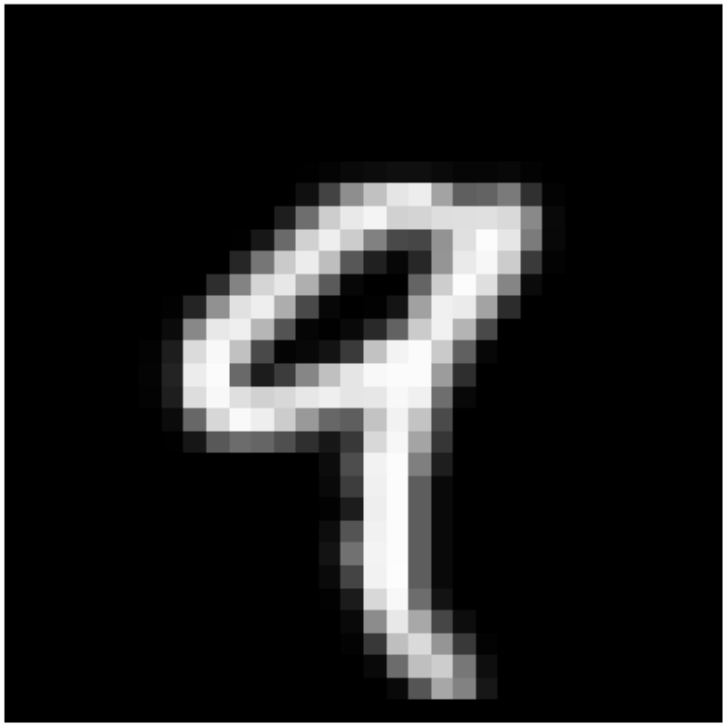
Prompt: four red



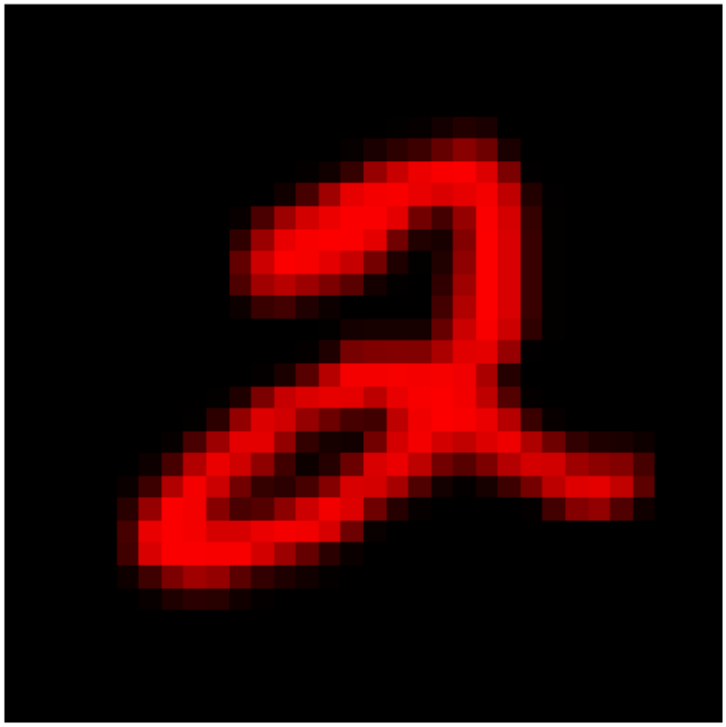
Prompt: one green



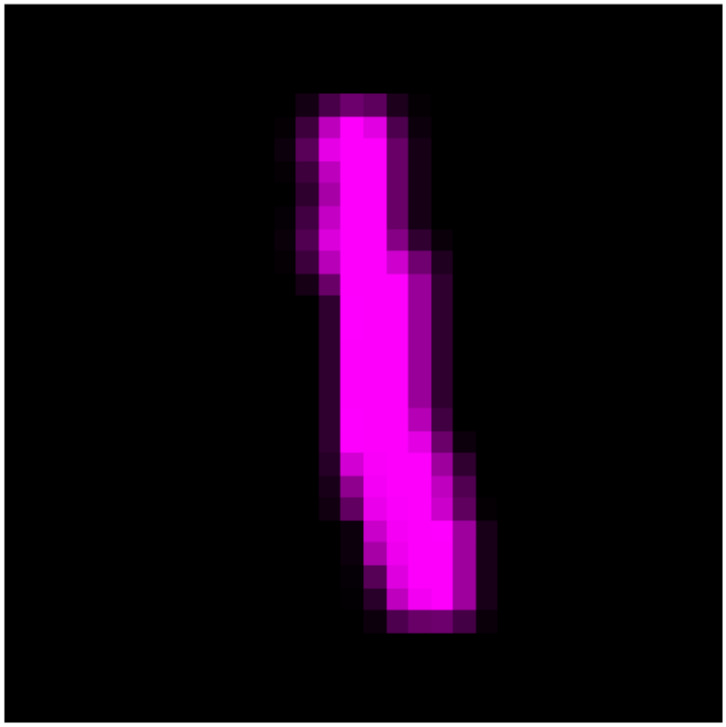
Prompt: nine



Prompt: two red

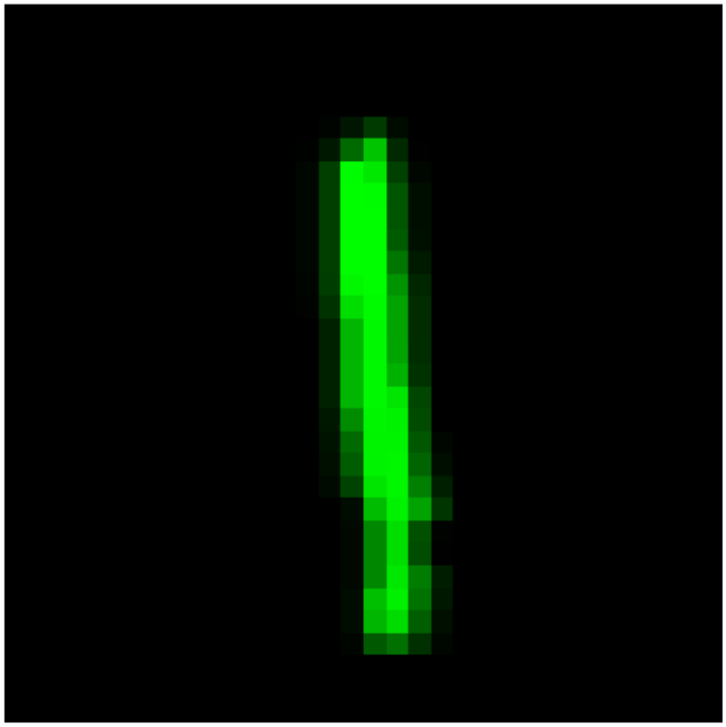


Prompt: one purple

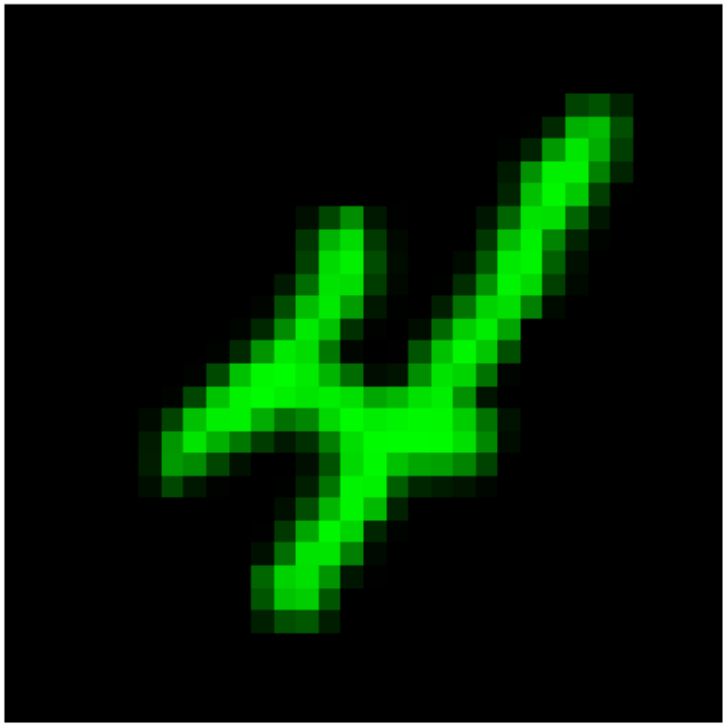


Prompt: three yellow

Prompt: one green



Prompt: four green



Load Model

After preprocessing the data, we load the diffusion model and then inject the LoRA.

```
unet = Unet(
    dim = 64,
    dim_mults = (1, 2, 4, 8),
)

model = Diffusion(
    unet,
    linear_scheduler,
    resolution=(32, 32),
)

checkpoint = torch.load("./lora_weight.ckpt")
model.load_state_dict(checkpoint, strict=False)
```

```
_IncompatibleKeys(missing_keys=['model.init_conv.weight',
'model.init_conv.bias', 'model.time_mlp.1.weight',
'model.time_mlp.1.bias', 'model.time_mlp.3.weight',
'model.time_mlp.3.bias', 'model.downs.0.0.mlp.1.weight',
'model.downs.0.0.mlp.1.bias', 'model.downs.0.0.block1.proj.weight',
'model.downs.0.0.block1.proj.bias',
'model.downs.0.0.block1.norm.weight',
'model.downs.0.0.block1.norm.bias',
'model.downs.0.0.block2.proj.weight',
'model.downs.0.0.block2.proj.bias',
'model.downs.0.0.block2.norm.weight',
'model.downs.0.0.block2.norm.bias', 'model.downs.0.1.mlp.1.weight',
'model.downs.0.1.mlp.1.bias', 'model.downs.0.1.block1.proj.weight',
'model.downs.0.1.block1.proj.bias',
'model.downs.0.1.block1.norm.weight',
'model.downs.0.1.block1.norm.bias',
'model.downs.0.1.block2.proj.weight',
'model.downs.0.1.block2.proj.bias',
'model.downs.0.1.block2.norm.weight',
'model.downs.0.1.block2.norm.bias',
'model.downs.0.2.groupnorm.weight', 'model.downs.0.2.groupnorm.bias',
'model.downs.0.2.conv_input.weight',
'model.downs.0.2.conv_input.bias',
'model.downs.0.2.layernorm_1.weight',
'model.downs.0.2.layernorm_1.bias',
'model.downs.0.2.attention_1.to_qkv.weight',
'model.downs.0.2.attention_1.linear.weight',
'model.downs.0.2.attention_1.linear.bias',
'model.downs.0.2.layernorm_2.weight',
'model.downs.0.2.layernorm_2.bias',
'model.downs.0.2.attention_2.to_q.weight',
'model.downs.0.2.attention_2.to_kv.weight',
```

```
'model.downs.0.2.attention_2.linear.weight',
'model.downs.0.2.attention_2.linear.bias',
'model.downs.0.2.layernorm_3.weight',
'model.downs.0.2.layernorm_3.bias',
'model.downs.0.2.linear_geglu_1.weight',
'model.downs.0.2.linear_geglu_1.bias',
'model.downs.0.2.linear_geglu_2.weight',
'model.downs.0.2.linear_geglu_2.bias',
'model.downs.0.2.conv_output.weight',
'model.downs.0.2.conv_output.bias', 'model.downs.0.3.0.weight',
'model.downs.0.3.0.bias', 'model.downs.1.0.mlp.1.weight',
'model.downs.1.0.mlp.1.bias', 'model.downs.1.0.block1.proj.weight',
'model.downs.1.0.block1.proj.bias',
'model.downs.1.0.block1.norm.weight',
'model.downs.1.0.block1.norm.bias',
'model.downs.1.0.block2.proj.weight',
'model.downs.1.0.block2.proj.bias',
'model.downs.1.0.block2.norm.weight',
'model.downs.1.0.block2.norm.bias', 'model.downs.1.1.mlp.1.weight',
'model.downs.1.1.mlp.1.bias', 'model.downs.1.1.block1.proj.weight',
'model.downs.1.1.block1.proj.bias',
'model.downs.1.1.block1.norm.weight',
'model.downs.1.1.block1.norm.bias',
'model.downs.1.1.block2.proj.weight',
'model.downs.1.1.block2.proj.bias',
'model.downs.1.1.block2.norm.weight',
'model.downs.1.1.block2.norm.bias',
'model.downs.1.2.groupnorm.weight', 'model.downs.1.2.groupnorm.bias',
'model.downs.1.2.conv_input.weight',
'model.downs.1.2.conv_input.bias',
'model.downs.1.2.layernorm_1.weight',
'model.downs.1.2.layernorm_1.bias',
'model.downs.1.2.attention_1.to_qkv.weight',
'model.downs.1.2.attention_1.linear.weight',
'model.downs.1.2.attention_1.linear.bias',
'model.downs.1.2.layernorm_2.weight',
'model.downs.1.2.layernorm_2.bias',
'model.downs.1.2.attention_2.to_q.weight',
'model.downs.1.2.attention_2.to_kv.weight',
'model.downs.1.2.attention_2.linear.weight',
'model.downs.1.2.attention_2.linear.bias',
'model.downs.1.2.layernorm_3.weight',
'model.downs.1.2.layernorm_3.bias',
'model.downs.1.2.linear_geglu_1.weight',
'model.downs.1.2.linear_geglu_1.bias',
'model.downs.1.2.linear_geglu_2.weight',
'model.downs.1.2.linear_geglu_2.bias',
'model.downs.1.2.conv_output.weight',
'model.downs.1.2.conv_output.bias', 'model.downs.1.3.0.weight',
```

```
'model.downs.1.3.0.bias', 'model.downs.2.0.mlp.1.weight',
'model.downs.2.0.mlp.1.bias', 'model.downs.2.0.block1.proj.weight',
'model.downs.2.0.block1.proj.bias',
'model.downs.2.0.block1.norm.weight',
'model.downs.2.0.block1.norm.bias',
'model.downs.2.0.block2.proj.weight',
'model.downs.2.0.block2.proj.bias',
'model.downs.2.0.block2.norm.weight',
'model.downs.2.0.block2.norm.bias', 'model.downs.2.1.mlp.1.weight',
'model.downs.2.1.mlp.1.bias', 'model.downs.2.1.block1.proj.weight',
'model.downs.2.1.block1.proj.bias',
'model.downs.2.1.block1.norm.weight',
'model.downs.2.1.block1.norm.bias',
'model.downs.2.1.block2.proj.weight',
'model.downs.2.1.block2.proj.bias',
'model.downs.2.1.block2.norm.weight',
'model.downs.2.1.block2.norm.bias',
'model.downs.2.2.groupnorm.weight', 'model.downs.2.2.groupnorm.bias',
'model.downs.2.2.conv_input.weight',
'model.downs.2.2.conv_input.bias',
'model.downs.2.2.layernorm_1.weight',
'model.downs.2.2.layernorm_1.bias',
'model.downs.2.2.attention_1.to_qkv.weight',
'model.downs.2.2.attention_1.linear.weight',
'model.downs.2.2.attention_1.linear.bias',
'model.downs.2.2.layernorm_2.weight',
'model.downs.2.2.layernorm_2.bias',
'model.downs.2.2.attention_2.to_q.weight',
'model.downs.2.2.attention_2.to_kv.weight',
'model.downs.2.2.attention_2.linear.weight',
'model.downs.2.2.attention_2.linear.bias',
'model.downs.2.2.layernorm_3.weight',
'model.downs.2.2.layernorm_3.bias',
'model.downs.2.2.linear_geglu_1.weight',
'model.downs.2.2.linear_geglu_1.bias',
'model.downs.2.2.linear_geglu_2.weight',
'model.downs.2.2.linear_geglu_2.bias',
'model.downs.2.2.conv_output.weight',
'model.downs.2.2.conv_output.bias', 'model.downs.2.3.0.weight',
'model.downs.2.3.0.bias', 'model.downs.3.0.mlp.1.weight',
'model.downs.3.0.mlp.1.bias', 'model.downs.3.0.block1.proj.weight',
'model.downs.3.0.block1.proj.bias',
'model.downs.3.0.block1.norm.weight',
'model.downs.3.0.block1.norm.bias',
'model.downs.3.0.block2.proj.weight',
'model.downs.3.0.block2.proj.bias',
'model.downs.3.0.block2.norm.weight',
'model.downs.3.0.block2.norm.bias', 'model.downs.3.1.mlp.1.weight',
'model.downs.3.1.mlp.1.bias', 'model.downs.3.1.block1.proj.weight',
```

```
'model.downs.3.1.block1.proj.bias',
'model.downs.3.1.block1.norm.weight',
'model.downs.3.1.block1.norm.bias',
'model.downs.3.1.block2.proj.weight',
'model.downs.3.1.block2.proj.bias',
'model.downs.3.1.block2.norm.weight',
'model.downs.3.1.block2.norm.bias',
'model.downs.3.2.groupnorm.weight', 'model.downs.3.2.groupnorm.bias',
'model.downs.3.2.conv_input.weight',
'model.downs.3.2.conv_input.bias',
'model.downs.3.2.layernorm_1.weight',
'model.downs.3.2.layernorm_1.bias',
'model.downs.3.2.attention_1.to_qkv.weight',
'model.downs.3.2.attention_1.linear.weight',
'model.downs.3.2.attention_1.linear.bias',
'model.downs.3.2.layernorm_2.weight',
'model.downs.3.2.layernorm_2.bias',
'model.downs.3.2.attention_2.to_q.weight',
'model.downs.3.2.attention_2.to_kv.weight',
'model.downs.3.2.attention_2.linear.weight',
'model.downs.3.2.attention_2.linear.bias',
'model.downs.3.2.layernorm_3.weight',
'model.downs.3.2.layernorm_3.bias',
'model.downs.3.2.linear_geglu_1.weight',
'model.downs.3.2.linear_geglu_1.bias',
'model.downs.3.2.linear_geglu_2.weight',
'model.downs.3.2.linear_geglu_2.bias',
'model.downs.3.2.conv_output.weight',
'model.downs.3.2.conv_output.bias', 'model.downs.3.3.weight',
'model.downs.3.3.bias', 'model.ups.0.0.mlp.1.weight',
'model.ups.0.0.mlp.1.bias', 'model.ups.0.0.block1.proj.weight',
'model.ups.0.0.block1.proj.bias', 'model.ups.0.0.block1.norm.weight',
'model.ups.0.0.block1.norm.bias', 'model.ups.0.0.block2.proj.weight',
'model.ups.0.0.block2.proj.bias', 'model.ups.0.0.block2.norm.weight',
'model.ups.0.0.block2.norm.bias', 'model.ups.0.0.res_conv.weight',
'model.ups.0.0.res_conv.bias', 'model.ups.0.1.mlp.1.weight',
'model.ups.0.1.mlp.1.bias', 'model.ups.0.1.block1.proj.weight',
'model.ups.0.1.block1.proj.bias', 'model.ups.0.1.block1.norm.weight',
'model.ups.0.1.block1.norm.bias', 'model.ups.0.1.block2.proj.weight',
'model.ups.0.1.block2.proj.bias', 'model.ups.0.1.block2.norm.weight',
'model.ups.0.1.block2.norm.bias', 'model.ups.0.1.res_conv.weight',
'model.ups.0.1.res_conv.bias', 'model.ups.0.2.groupnorm.weight',
'model.ups.0.2.groupnorm.bias', 'model.ups.0.2.conv_input.weight',
'model.ups.0.2.conv_input.bias', 'model.ups.0.2.layernorm_1.weight',
'model.ups.0.2.layernorm_1.bias',
'model.ups.0.2.attention_1.to_qkv.weight',
'model.ups.0.2.attention_1.linear.weight',
'model.ups.0.2.attention_1.linear.bias',
'model.ups.0.2.layernorm_2.weight', 'model.ups.0.2.layernorm_2.bias',
```

```
'model.ups.0.2.attention_2.to_q.weight',
'model.ups.0.2.attention_2.to_kv.weight',
'model.ups.0.2.attention_2.linear.weight',
'model.ups.0.2.attention_2.linear.bias',
'model.ups.0.2.layernorm_3.weight', 'model.ups.0.2.layernorm_3.bias',
'model.ups.0.2.linear_geglu_1.weight',
'model.ups.0.2.linear_geglu_1.bias',
'model.ups.0.2.linear_geglu_2.weight',
'model.ups.0.2.linear_geglu_2.bias',
'model.ups.0.2.conv_output.weight', 'model.ups.0.2.conv_output.bias',
'model.ups.0.3.1.weight', 'model.ups.0.3.1.bias',
'model.ups.1.0.mlp.1.weight', 'model.ups.1.0.mlp.1.bias',
'model.ups.1.0.block1.proj.weight', 'model.ups.1.0.block1.proj.bias',
'model.ups.1.0.block1.norm.weight', 'model.ups.1.0.block1.norm.bias',
'model.ups.1.0.block2.proj.weight', 'model.ups.1.0.block2.proj.bias',
'model.ups.1.0.block2.norm.weight', 'model.ups.1.0.block2.norm.bias',
'model.ups.1.0.res_conv.weight', 'model.ups.1.0.res_conv.bias',
'model.ups.1.1.mlp.1.weight', 'model.ups.1.1.mlp.1.bias',
'model.ups.1.1.block1.proj.weight', 'model.ups.1.1.block1.proj.bias',
'model.ups.1.1.block1.norm.weight', 'model.ups.1.1.block1.norm.bias',
'model.ups.1.1.block2.proj.weight', 'model.ups.1.1.block2.proj.bias',
'model.ups.1.1.block2.norm.weight', 'model.ups.1.1.block2.norm.bias',
'model.ups.1.1.res_conv.weight', 'model.ups.1.1.res_conv.bias',
'model.ups.1.2.groupnorm.weight', 'model.ups.1.2.groupnorm.bias',
'model.ups.1.2.conv_input.weight', 'model.ups.1.2.conv_input.bias',
'model.ups.1.2.layernorm_1.weight', 'model.ups.1.2.layernorm_1.bias',
'model.ups.1.2.attention_1.to_qkv.weight',
'model.ups.1.2.attention_1.linear.weight',
'model.ups.1.2.attention_1.linear.bias',
'model.ups.1.2.layernorm_2.weight', 'model.ups.1.2.layernorm_2.bias',
'model.ups.1.2.attention_2.to_q.weight',
'model.ups.1.2.attention_2.to_kv.weight',
'model.ups.1.2.attention_2.linear.weight',
'model.ups.1.2.attention_2.linear.bias',
'model.ups.1.2.layernorm_3.weight', 'model.ups.1.2.layernorm_3.bias',
'model.ups.1.2.linear_geglu_1.weight',
'model.ups.1.2.linear_geglu_1.bias',
'model.ups.1.2.linear_geglu_2.weight',
'model.ups.1.2.linear_geglu_2.bias',
'model.ups.1.2.conv_output.weight', 'model.ups.1.2.conv_output.bias',
'model.ups.1.3.1.weight', 'model.ups.1.3.1.bias',
'model.ups.2.0.mlp.1.weight', 'model.ups.2.0.mlp.1.bias',
'model.ups.2.0.block1.proj.weight', 'model.ups.2.0.block1.proj.bias',
'model.ups.2.0.block1.norm.weight', 'model.ups.2.0.block1.norm.bias',
'model.ups.2.0.block2.proj.weight', 'model.ups.2.0.block2.proj.bias',
'model.ups.2.0.block2.norm.weight', 'model.ups.2.0.block2.norm.bias',
'model.ups.2.0.res_conv.weight', 'model.ups.2.0.res_conv.bias',
'model.ups.2.1.mlp.1.weight', 'model.ups.2.1.mlp.1.bias',
'model.ups.2.1.block1.proj.weight', 'model.ups.2.1.block1.proj.bias',
```

```
'model.ups.2.1.block1.norm.weight', 'model.ups.2.1.block1.norm.bias',
'model.ups.2.1.block2.proj.weight', 'model.ups.2.1.block2.proj.bias',
'model.ups.2.1.block2.norm.weight', 'model.ups.2.1.block2.norm.bias',
'model.ups.2.1.res_conv.weight', 'model.ups.2.1.res_conv.bias',
'model.ups.2.2.groupnorm.weight', 'model.ups.2.2.groupnorm.bias',
'model.ups.2.2.conv_input.weight', 'model.ups.2.2.conv_input.bias',
'model.ups.2.2.layernorm_1.weight', 'model.ups.2.2.layernorm_1.bias',
'model.ups.2.2.attention_1.to_qkv.weight',
'model.ups.2.2.attention_1.linear.weight',
'model.ups.2.2.attention_1.linear.bias',
'model.ups.2.2.layernorm_2.weight', 'model.ups.2.2.layernorm_2.bias',
'model.ups.2.2.attention_2.to_q.weight',
'model.ups.2.2.attention_2.to_kv.weight',
'model.ups.2.2.attention_2.linear.weight',
'model.ups.2.2.attention_2.linear.bias',
'model.ups.2.2.layernorm_3.weight', 'model.ups.2.2.layernorm_3.bias',
'model.ups.2.2.linear_geglu_1.weight',
'model.ups.2.2.linear_geglu_1.bias',
'model.ups.2.2.linear_geglu_2.weight',
'model.ups.2.2.linear_geglu_2.bias',
'model.ups.2.2.conv_output.weight', 'model.ups.2.2.conv_output.bias',
'model.ups.2.3.1.weight', 'model.ups.2.3.1.bias',
'model.ups.3.0.mlp.1.weight', 'model.ups.3.0.mlp.1.bias',
'model.ups.3.0.block1.proj.weight', 'model.ups.3.0.block1.proj.bias',
'model.ups.3.0.block1.norm.weight', 'model.ups.3.0.block1.norm.bias',
'model.ups.3.0.block2.proj.weight', 'model.ups.3.0.block2.proj.bias',
'model.ups.3.0.block2.norm.weight', 'model.ups.3.0.block2.norm.bias',
'model.ups.3.0.res_conv.weight', 'model.ups.3.0.res_conv.bias',
'model.ups.3.1.mlp.1.weight', 'model.ups.3.1.mlp.1.bias',
'model.ups.3.1.block1.proj.weight', 'model.ups.3.1.block1.proj.bias',
'model.ups.3.1.block1.norm.weight', 'model.ups.3.1.block1.norm.bias',
'model.ups.3.1.block2.proj.weight', 'model.ups.3.1.block2.proj.bias',
'model.ups.3.1.block2.norm.weight', 'model.ups.3.1.block2.norm.bias',
'model.ups.3.1.res_conv.weight', 'model.ups.3.1.res_conv.bias',
'model.ups.3.2.groupnorm.weight', 'model.ups.3.2.groupnorm.bias',
'model.ups.3.2.conv_input.weight', 'model.ups.3.2.conv_input.bias',
'model.ups.3.2.layernorm_1.weight', 'model.ups.3.2.layernorm_1.bias',
'model.ups.3.2.attention_1.to_qkv.weight',
'model.ups.3.2.attention_1.linear.weight',
'model.ups.3.2.attention_1.linear.bias',
'model.ups.3.2.layernorm_2.weight', 'model.ups.3.2.layernorm_2.bias',
'model.ups.3.2.attention_2.to_q.weight',
'model.ups.3.2.attention_2.to_kv.weight',
'model.ups.3.2.attention_2.linear.weight',
'model.ups.3.2.attention_2.linear.bias',
'model.ups.3.2.layernorm_3.weight', 'model.ups.3.2.layernorm_3.bias',
'model.ups.3.2.linear_geglu_1.weight',
'model.ups.3.2.linear_geglu_1.bias',
'model.ups.3.2.linear_geglu_2.weight',
```

```
'model.ups.3.2.linear_geglu_2.bias',
'model.ups.3.2.conv_output.weight', 'model.ups.3.2.conv_output.bias',
'model.ups.3.3.weight', 'model.ups.3.3.bias',
'model.mid_block1.mlp.1.weight', 'model.mid_block1.mlp.1.bias',
'model.mid_block1.block1.proj.weight',
'model.mid_block1.block1.proj.bias',
'model.mid_block1.block1.norm.weight',
'model.mid_block1.block1.norm.bias',
'model.mid_block1.block2.proj.weight',
'model.mid_block1.block2.proj.bias',
'model.mid_block1.block2.norm.weight',
'model.mid_block1.block2.norm.bias',
'model.mid_attn.groupnorm.weight', 'model.mid_attn.groupnorm.bias',
'model.mid_attn.conv_input.weight', 'model.mid_attn.conv_input.bias',
'model.mid_attn.layernorm_1.weight',
'model.mid_attn.layernorm_1.bias',
'model.mid_attn.attention_1.to_qkv.weight',
'model.mid_attn.attention_1.linear.weight',
'model.mid_attn.attention_1.linear.bias',
'model.mid_attn.layernorm_2.weight',
'model.mid_attn.layernorm_2.bias',
'model.mid_attn.attention_2.to_q.weight',
'model.mid_attn.attention_2.to_kv.weight',
'model.mid_attn.attention_2.linear.weight',
'model.mid_attn.attention_2.linear.bias',
'model.mid_attn.layernorm_3.weight',
'model.mid_attn.layernorm_3.bias',
'model.mid_attn.linear_geglu_1.weight',
'model.mid_attn.linear_geglu_1.bias',
'model.mid_attn.linear_geglu_2.weight',
'model.mid_attn.linear_geglu_2.bias',
'model.mid_attn.conv_output.weight',
'model.mid_attn.conv_output.bias', 'model.mid_block2.mlp.1.weight',
'model.mid_block2.mlp.1.bias', 'model.mid_block2.block1.proj.weight',
'model.mid_block2.block1.proj.bias',
'model.mid_block2.block1.norm.weight',
'model.mid_block2.block1.norm.bias',
'model.mid_block2.block2.proj.weight',
'model.mid_block2.block2.proj.bias',
'model.mid_block2.block2.norm.weight',
'model.mid_block2.block2.norm.bias',
'model.final_res_block.mlp.1.weight',
'model.final_res_block.mlp.1.bias',
'model.final_res_block.block1.proj.weight',
'model.final_res_block.block1.proj.bias',
'model.final_res_block.block1.norm.weight',
'model.final_res_block.block1.norm.bias',
'model.final_res_block.block2.proj.weight',
'model.final_res_block.block2.proj.bias',
```

```
'model.final_res_block.block2.norm.weight',
'model.final_res_block.block2.norm.bias',
'model.final_res_block.res_conv.weight',
'model.final_res_block.res_conv.bias', 'model.final_conv.weight',
'model.final_conv.bias'],
unexpected_keys=['pixel_level_module.decoder.fpn.stem.0.lora_A',
'pixel_level_module.decoder.fpn.stem.0.lora_B',
'pixel_level_module.decoder.fpn.stem.1.bias',
'pixel_level_module.decoder.fpn.layers.0.proj.0.lora_A',
'pixel_level_module.decoder.fpn.layers.0.proj.0.lora_B',
'pixel_level_module.decoder.fpn.layers.0.proj.1.bias',
'pixel_level_module.decoder.fpn.layers.0.block.0.lora_A',
'pixel_level_module.decoder.fpn.layers.0.block.0.lora_B',
'pixel_level_module.decoder.fpn.layers.0.block.1.bias',
'pixel_level_module.decoder.fpn.layers.1.proj.0.lora_A',
'pixel_level_module.decoder.fpn.layers.1.proj.0.lora_B',
'pixel_level_module.decoder.fpn.layers.1.proj.1.bias',
'pixel_level_module.decoder.fpn.layers.1.block.0.lora_A',
'pixel_level_module.decoder.fpn.layers.1.block.0.lora_B',
'pixel_level_module.decoder.fpn.layers.1.block.1.bias',
'pixel_level_module.decoder.fpn.layers.2.proj.0.lora_A',
'pixel_level_module.decoder.fpn.layers.2.proj.0.lora_B',
'pixel_level_module.decoder.fpn.layers.2.proj.1.bias',
'pixel_level_module.decoder.fpn.layers.2.block.0.lora_A',
'pixel_level_module.decoder.fpn.layers.2.block.0.lora_B',
'pixel_level_module.decoder.fpn.layers.2.block.1.bias',
'pixel_level_module.decoder.mask_projection.weight',
'pixel_level_module.decoder.mask_projection.bias',
'pixel_level_module.decoder.mask_projection.lora_A',
'pixel_level_module.decoder.mask_projection.lora_B'])
```

Insert LoRA into diffusion model

Which modules should you inject the LoRA? (OT 1) Ans.

Of course, the diffusion module. Why? Because the CLIP already knows the meaning of the colour, but the U-Net, which do the diffusion, does not. So, we inject the LoRA into U-Net module. To be more specific, we inject into cross-attention layers which helps the model to get right attention to the encoded colours, and also convolution layer to make it generate more precise.

Instruction

OT 2: inject LoRA into diffusion model

```
r = 8
hooks = []
for name, module in model.named_modules():
    # OT 2: inject LoRA
    if (("attention_2" in name) and isinstance(module, nn.Linear)):
```

```
      hook_fn = attach_lora(
            module,
            r,
            lora_alpha,
            in_features=module.in_features,
            out_features=module.out_features
        )
      handle = module.register_forward_hook(hook_fn)
      hooks.append(handle)
      print(f"Attached LoRA to: {name}")

print(f"\nTotal LoRA Hooks Registered: {len(hooks)}")
```

```
Attached LoRA to: model.downs.0.2.attention_2.to_q
Attached LoRA to: model.downs.0.2.attention_2.to_kv
Attached LoRA to: model.downs.0.2.attention_2.linear
Attached LoRA to: model.downs.1.2.attention_2.to_q
Attached LoRA to: model.downs.1.2.attention_2.to_kv
Attached LoRA to: model.downs.1.2.attention_2.linear
Attached LoRA to: model.downs.2.2.attention_2.to_q
Attached LoRA to: model.downs.2.2.attention_2.to_kv
Attached LoRA to: model.downs.2.2.attention_2.linear
Attached LoRA to: model.downs.3.2.attention_2.to_q
Attached LoRA to: model.downs.3.2.attention_2.to_kv
Attached LoRA to: model.downs.3.2.attention_2.linear
Attached LoRA to: model.ups.0.2.attention_2.to_q
Attached LoRA to: model.ups.0.2.attention_2.to_kv
Attached LoRA to: model.ups.0.2.attention_2.linear
Attached LoRA to: model.ups.1.2.attention_2.to_q
Attached LoRA to: model.ups.1.2.attention_2.to_kv
Attached LoRA to: model.ups.1.2.attention_2.linear
Attached LoRA to: model.ups.2.2.attention_2.to_q
Attached LoRA to: model.ups.2.2.attention_2.to_kv
Attached LoRA to: model.ups.2.2.attention_2.linear
Attached LoRA to: model.ups.3.2.attention_2.to_q
Attached LoRA to: model.ups.3.2.attention_2.to_kv
Attached LoRA to: model.ups.3.2.attention_2.linear
Attached LoRA to: model.mid_attn.attention_2.to_q
Attached LoRA to: model.mid_attn.attention_2.to_kv
Attached LoRA to: model.mid_attn.attention_2.linear

Total LoRA Hooks Registered: 27
```

After inserting LoRA, we freeze the model and train only LoRA.

Instruction

OT 3: freeze model and train only matrics A and B, and bias.

```
for n, p in model.named_parameters():
    # OT 3: freeze model and train only matrics A and B, and bias.
    if (("lora" in n) or (("attention_2" in n) and ("bias" in n))):
      p.requires_grad = True
    else:
      p.requires_grad = False

pytorch_total_params = sum(p.numel() for p in model.parameters() if
p.requires_grad)
print(pytorch_total_params)
```

```
152512
```

Training

```
class Trainer:
    def __init__(
        self,
        model,
        tokenizer,
        text_encoder,
        train_dataloader,
        val_dataloader,
        ckpt_path,
        resolution=None,
        lr=1e-4,
        device="cuda",
    ):
        self.model = model
        self.tokenizer = tokenizer
        self.text_encoder = text_encoder

        self.train_dataloader = train_dataloader
        self.val_dataloader = val_dataloader
        self.resolution = next(iter(train_dataloader))[0].shape[-2:]
if resolution == None else resolution

        self.lr = lr
        self.configure_optimizers()

        self.ckpt_path = ckpt_path
        self.device = device
        self.epoch = 0
        self.mode = "train"

        self.to(device)

    def configure_optimizers(self):
        """Construct Optimizer"""
        self.optimizer = torch.optim.Adam(self.parameters(),
```

```
lr=self.lr)

    def _step(self, batch):
        """One step forward pass

        Args:
            batch (Tuple[torch.tensor]): A mini-batch data to be
processed. The first index must contain images
                and the latter are the corresponded label. The shape
of image is (B, C, H, W),
                while the shape of label is (B).

        Returns:
            loss (torch.tensor): A loss value to be used for updating
the model.

        """
        x, context = batch
        # TODO 12: calculate the loss given batch
        # Pseudo code:
        #   1. acquire the context embedding through self.embedding
        #   2. expand dimension such that the shape of embedding is
(batch_size, 1, dimension)
        #   3. calculate loss using self.model.p_losses
        context = self.text_encoder(context, return_dict=False)[0]
        losses = self.model.p_losses(x, context)
        return losses

    def optimizer_step(self, loss):
        """Update model based on the given loss.

        Args:
            loss (torch.tensor): A loss value to be differentiated
(Note: must be able to compute a gradient.)

        """
        loss.backward()
        self.optimizer.step()
        self.optimizer.zero_grad()

    def train_loop(self, epoch):
        """One epoch training loop.

        Args:
            epoch (int): A current epoch.

        Returns:
            train_loss (np.array): A list of loss in each batch.

        """
```

```
        self.train()
        with tqdm(self.train_dataloader, unit="batch", leave=False) as
tbatch:
            train_loss = []
            for batch in tbatch:
                tbatch.set_description(f"Epoch {epoch+1}")
                batch = list(
                    map(lambda x: x.to(self.device), batch)
                )  # allocate data on the pre-defined device.
                loss = self._step(batch)  # forward pass and calculate
loss
                self.optimizer_step(loss)  # backward pass / updating
the parameters
                train_loss.append(loss.item())
                tbatch.set_postfix(loss=loss.item())
            train_loss = np.array(train_loss)
        return train_loss

    def val_loop(self, epoch):
        """One epoch validating loop.

        Args:
            epoch (int): A current epoch.

        Returns:
            train_loss (np.array): A list of loss in each batch.

        """
        self.eval()
        # with self.ema.average_parameters():  # Copy EMA weight to
model and restore after exiting `with`
        with tqdm(self.val_dataloader, unit="batch", leave=False) as
tbatch:
            val_loss = []
            for batch in tbatch:
                tbatch.set_description(f"Epoch {epoch+1}")
                batch = list(
                    map(lambda x: x.to(self.device), batch)
                )  # allocate data on the pre-defined device.
                loss = (
                    self._step(batch).detach().cpu()
                )  # forward pass and calculate loss
                val_loss.append(loss.item())
                tbatch.set_postfix(loss=loss.item())
            val_loss = np.array(val_loss)
        return val_loss

    def fit(self, epochs, num_val_sampler=2, ckpt_path=None,
resume=False, sampling_round=10):
        """Train the model
```

```
        Args:
            epochs (int): A number of epochs to be trained.

        Returns:
            train_epoch (np.array): A list of training loss in each
epoch.
            val_epoch (np.array): A list of validation loss in each
epoch.

        """
        best_val_loss = None
        ckpt_path = self.ckpt_path if ckpt_path is None else ckpt_path
        last_path = os.path.join(ckpt_path, "last_weight.ckpt")
        print("Save:", last_path)
        self.to(self.device)

        train_loss_epoch = []
        val_loss_epoch = []

        epochs = range(self.epoch, self.epoch+epochs) if resume ==
True else range(epochs)
        for epoch in epochs:
            self.epoch = epoch

            train_loss = self.train_loop(epoch) # Training Model
            val_loss = self.val_loop(epoch) # Evaluating Model

            if (epoch+1)%sampling_round == 0 or epoch == 0:
                self.sampling_image(num_val_sampler) # Generating new
images

            print(
                f"Epoch {epoch+1}: Train Loss
{train_loss.mean().item()}, Val Loss {val_loss.mean().item()}"
            )
            train_loss_epoch.append(train_loss.mean().item())
            val_loss_epoch.append(val_loss.mean().item())

            torch.cuda.empty_cache() # Clear GPU cache

        self.save(last_path)
        train_loss_epoch = np.array(train_loss_epoch)
        val_loss_epoch = np.array(val_loss_epoch)
        return train_loss_epoch, val_loss_epoch

    def sampling_image(self, num_images):
        """Sampling new images

        The sampling images was exhibited in a column format, using
```

```
matplotlib.

        Args:
            num_images (int): A number of images to be generated.

        """
        self.eval()
        fig, axs = plt.subplots(ncols=num_images)
        text_label = random.choices(["zero", "one", "two", "three",
"four", "five", "six", "seven", "eight", "nine"], k=num_images)
        text_label = list(map(lambda x: " ".join([x,
random.choice(list(COLOR.keys()))]), text_label))
        input_ids = self.tokenizer(text_label, padding="max_length",
truncation=True, max_length=self.tokenizer.model_max_length,
return_tensors="pt").input_ids.to(self.device)
        context = self.text_encoder(input_ids, return_dict=False)[0]
        sampled_images = self.model.sample(batch_size = num_images,
context=context)#
        for idx_sampler in range(num_images):

axs[idx_sampler].imshow(rearrange(unnormalize_to_zero_to_one(sampled_i
mages[idx_sampler]), "c h w -> h w c"), cmap="gray")
            axs[idx_sampler].set_axis_off()
            axs[idx_sampler].set_title(text_label[idx_sampler])
        plt.show()

    def save(self, path):
        """ Save trainer state """
        path = self.ckpt_path if path is None else path
        torch.save({
            "epoch": self.epoch,
            "model_state_dict": self.model.state_dict(),
            "optimizer_state_dict": self.optimizer.state_dict(),
            "text_encoder": self.text_encoder.state_dict(),
        }, path)

    def load(self, path):
        """ Load trainer state """
        checkpoint = torch.load(path)
        self.epoch = checkpoint["epoch"]
        self.model.load_state_dict(checkpoint["model_state_dict"])

self.optimizer.load_state_dict(checkpoint["optimizer_state_dict"])
        self.text_encoder.load_state_dict(checkpoint["text_encoder"])

    def train(self):
        """ Set trainer to train mode """
        if self.mode != "train":
            self.model.train()
            self.mode = "train"
```

```
    def eval(self):
        """ Set trainer to evaluate mode """
        if self.mode != "eval":
            self.model.eval()
            self.mode = "eval"

    def to(self, device):
        """Set a computational device (eg. cpu or cuda).

        Args:
            device (str): A computational device to be computed on.

        """
        self.device = device
        self.model.to(device)
        self.text_encoder.to(device)

    def set_dataset(self, train_dataloader, val_dataloader,
resolution=None):
        """Set a new dataset.

        Args:
            train_dataloader (torch.utils.data.DataLoader): A new
training set.
            val_dataloader (torch.utils.data.DataLoader): A new
validation set.
            resolution (Tuple[int], optional): A new resolution of the
images in the given dataset. If None,
                it was set to the size of the first image. Default to
None.

        """
        self.train_dataloader = train_dataloader
        self.val_dataloader = val_dataloader
        self.resolution = next(iter(train_dataloader))[0].shape[-2:]
if resolution == None else resolution

    def parameters(self):
        return [p for p in self.model.parameters() if p.requires_grad]

text_encoder = CLIPTextModel.from_pretrained("CompVis/stable-
diffusion-v1-4", subfolder="text_encoder", revision=None)

text_encoder.requires_grad_(False)

trainer_config = {
    "model": model,
    "tokenizer": tokenizer,
    "text_encoder": text_encoder,
```

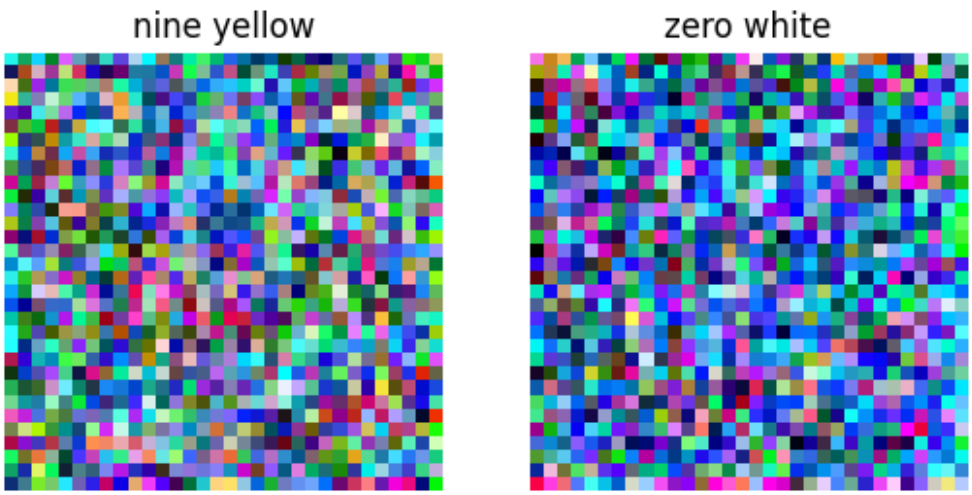
```
    "train_dataloader": train_dataloader,
    "resolution": RESOLUTION,
    "val_dataloader": test_dataloader,
    "device": "cuda",
    "ckpt_path": "./weights/",
    "lr": 3e-4,
}

trainer = Trainer(**trainer_config)
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/
file_download.py:1132: FutureWarning: `resume_download` is deprecated
and will be removed in version 1.0.0. Downloads always resume when
possible. If you want to force a new download, use
`force_download=True`.
  warnings.warn(
```

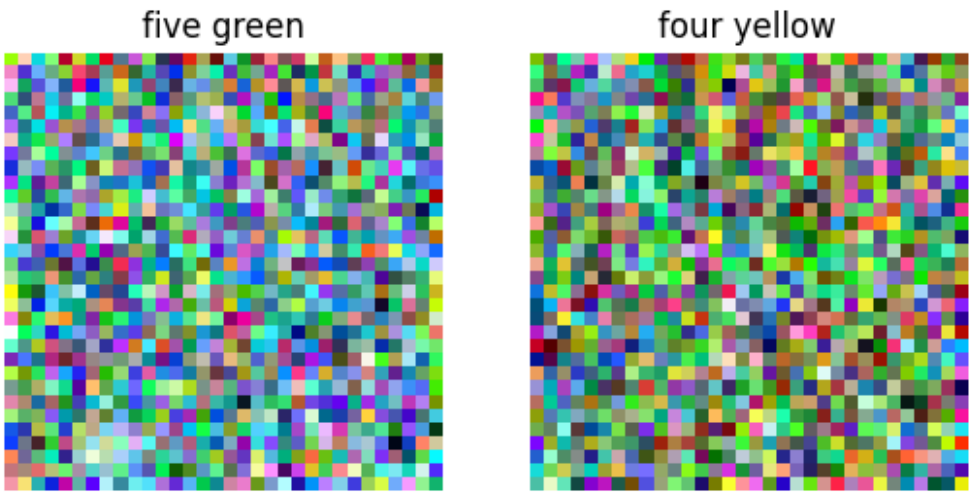
```
epoch = 10
train_losses, val_losses = trainer.fit(epoch)
```

```
Save: ./weights/last_weight.ckpt
```

```
Epoch 1:   0%|          | 0/469 [00:00<?,
?batch/s]/tmp/ipykernel_496/3489802820.py:80: UserWarning: To copy
construct from a tensor, it is recommended to use
sourceTensor.detach().clone() or
sourceTensor.detach().clone().requires_grad_(True), rather than
torch.tensor(sourceTensor).
  alpha_t = torch.tensor(tmp_alpha, device=x_0.device,
dtype=x_0.dtype)
/tmp/ipykernel_496/3489802820.py:96: UserWarning: To copy construct
from a tensor, it is recommended to use sourceTensor.detach().clone()
or sourceTensor.detach().clone().requires_grad_(True), rather than
torch.tensor(sourceTensor).
  return torch.tensor(std, device=t.device)
```

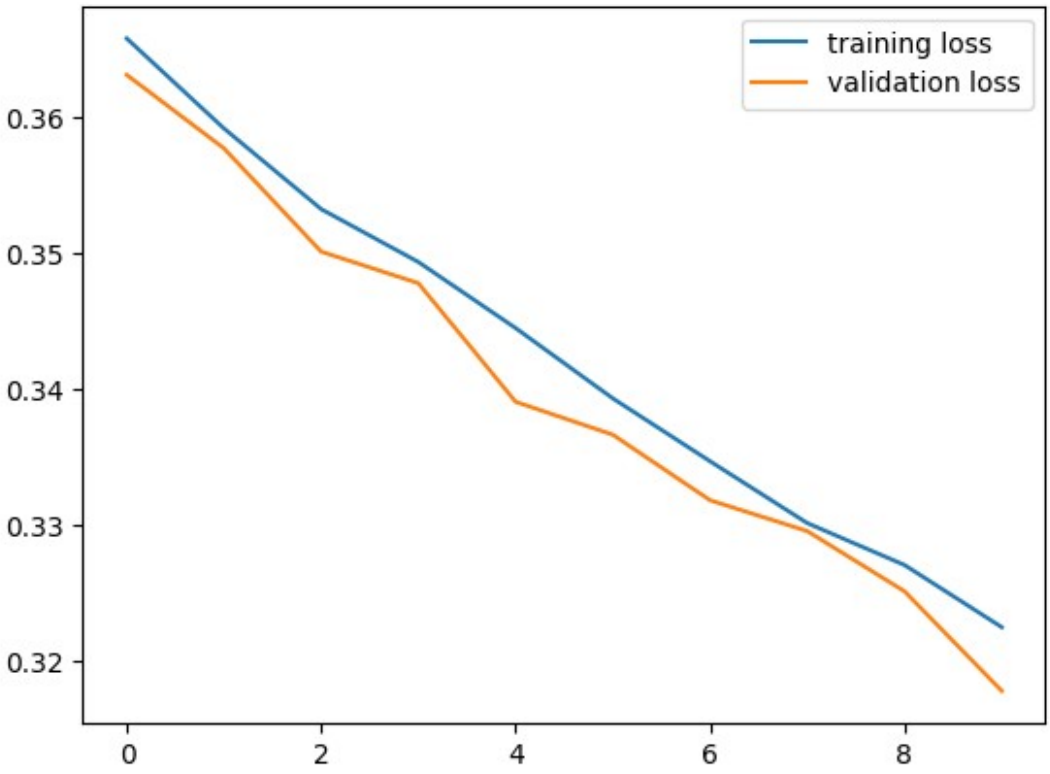



```
Epoch 1: Train Loss 0.36582132265257683, Val Loss 0.36313630319848844

Epoch 2: Train Loss 0.35919204195425203, Val Loss 0.3577250023431416

Epoch 3: Train Loss 0.3532488065233617, Val Loss 0.35011519850054873

Epoch 4: Train Loss 0.3493512990251025, Val Loss 0.3477894192254996

Epoch 5: Train Loss 0.34448441920249956, Val Loss 0.33905290236955954

Epoch 6: Train Loss 0.3393041999228219, Val Loss 0.3366271219676054

Epoch 7: Train Loss 0.33467312814838596, Val Loss 0.33180289328852786

Epoch 8: Train Loss 0.3301168827614042, Val Loss 0.32952695104140267

Epoch 9: Train Loss 0.32703016739664303, Val Loss 0.3250933874257003
```




```
Epoch 10: Train Loss 0.3224511665067693, Val Loss 0.3177584504024892
```

```
plt.plot(range(epoch), train_losses, label="training loss")
plt.plot(range(epoch), val_losses, label="validation loss")
plt.legend()
plt.show()
```



Inference

Although the diffusion model is not trained to color the digit 5-9, it can generate the colored digit 5-9, achieving zero-shot generation capability.

```
fig, axs = plt.subplots(ncols=10, nrows=10, figsize=(10, 10))
fig.tight_layout(pad=2)
text_label = ["zero", "one", "two", "three", "four", "five", "six",
"seven", "eight", "nine"]

generated_images = []
for iter in range(10):
    mod_text_label = list(map(lambda x: " ".join([x,
random.choice(list(COLOR.keys()))]), text_label))
    input_ids = tokenizer(mod_text_label, padding="max_length",
truncation=True, max_length=tokenizer.model_max_length,
return_tensors="pt").input_ids.to("cuda")
    context = text_encoder(input_ids, return_dict=False)[0]
    sampled_images = trainer.model.sample(batch_size = 10,
context=context)#
    generated_images.append(sampled_images)
    for idx_sampler in range(10):
        axs[iter]
[idx_sampler].imshow(rearrange(unnormalize_to_zero_to_one(sampled_imag
es[idx_sampler]), "c h w -> h w c"), cmap="gray")
        axs[iter][idx_sampler].set_axis_off()
        axs[iter][idx_sampler].set_title(mod_text_label[idx_sampler])
plt.show()
```

OT 4. Does our model have any other zero-shot generation capability?

Answer here

Hint: Is it possible to generate other colors?

Possible. Imagine when the diffusion only learns the colour "red" and "yellow", and it has not learn "yellow" image. But, CLIP understands that "orange" is semantically located between "red" and "yellow". So, the diffusion can learn to output the color between "red" and "yellow" which is "orange". It might not be accurate as we input the "orange" colour as training dataset, but it is possible.

Conclusion

Congratulations! You have successfully built the DDPM and Low-Rank Adaptation. Fortunately, in the real world, there are pre-built libraries ready to be used without implementing both from scratch. For diffusion, we have `diffusers` library providing all the necessary modules for the diffusion pipeline, as well as LoRA, we can use `peft` library to inject LoRA into the model by declaring injected modules in the LoRAConfig. For more details, please visit the HuggingFace website.